Neural Networks 206 (2027) 109839

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Posterior misclassification rate-based adaptive composite neighbor oversampling for imbalanced learning

Xianyou Zhong [a,b], Yifeng Zheng [a,b,*], Wenjie Zhang [a,b], Baoya Wei [a,b], Weiqiang Liu [a,b]

[a] School of Computer Science, Minnan Normal University, Zhangzhou, 363000, China
[b] Key Laboratory of Data Science and Intelligence Application, Fujian Province University, Zhangzhou, 363000, China

ARTICLE INFO

Keywords:
Imbalanced classification
Oversampling approach
Misclassification rate
Adaptive neighbor
Triangular interpolation

ABSTRACT

Class imbalance refers to a substantial disparity in sample sizes across classes, which can hinder minority class learning and degrade classification performance. Existing oversampling methods often rely on fixed-size neighborhoods for linear interpolation, making sample generation sensitive to borderline and noisy samples. Therefore, we propose the posterior misclassification rate-guided adaptive composite neighbor synthetic minority over-sampling technique (RAN-SMOTE). First, an adaptive misclassification rate estimation strategy is designed using heterogeneous base learners to characterize the sample distribution and remove noisy samples. Second, the posterior misclassification rate is incorporated to adaptively adjust the neighborhood size and select reliable minority neighbors, thereby reducing the interference from borderline samples. Finally, a probabilistic neighbor selection strategy is developed based on composite scores, and Beta distribution-guided triangular interpolation is employed to generate diverse and reliable synthetic samples. Experiments on 31 imbalanced datasets demonstrate that RAN-SMOTE improves classification performance compared with competing methods. Additional label-noise experiments on five datasets demonstrate that the proposed method exhibits good robustness at noise rates of 5% and 10%.

1. Introduction

In real-world applications, class imbalance is one of the key tasks in practical applications such as medical diagnosis (Salmi et al., 2024; Siddavatam & Shinde, 2026), fraud detection (Dal Pozzolo et al., 2018), and fault diagnosis (Chen et al., 2024a; Lin & Jamrus, 2024). The class with fewer samples in the data space is usually referred to as the minority class. Although minority samples are less frequent, they often represent more critical or high-risk cases, leading to higher misclassification costs. Conversely, the majority class contains more samples and usually represents common events or states. For example, in medical diagnosis, patients often account for only a small proportion of the population, but accurately identifying them is the core objective of the application. Therefore, machine learning can be used to assist in identifying patients and improve diagnostic efficiency and accuracy. The model tends to be biased toward the majority class during learning, thereby reducing its ability to identify minority samples (Guo et al., 2017). To alleviate the above problem, researchers have explored approaches from two perspectives: improving model learning mechanisms and adjusting the data distribution, which can be categorized into algorithm-level methods (Rezvani et al., 2024; Yu et al., 2016; Zhou & Liu, 2006) and data-level methods (Chawla et al., 2002; Hu et al., 2026; Zhou & Zou, 2025).

Algorithm-level methods aim to adjust the learning mechanisms to learn the features and decision boundaries of minority samples, thereby reducing the prediction bias toward the majority class. Such as cost-sensitive learning (Sun et al., 2007; Zhou & Liu, 2006), classification strategy adjustment (Yu et al., 2016; Zhang et al., 2020), and kernel learning (Rezvani et al., 2024). Cost-sensitive learning alleviates majority class bias by assigning higher misclassification costs to minority class errors (Sun et al., 2007). Unlike cost-sensitive learning, classification strategy adjustment modifies decision rules or thresholds to balance class-wise prediction tendencies and improve the recognition of minority samples under the default decision strategy (Zhang et al., 2020). From the perspective of feature representation, kernel learning maps samples into a suitable feature space to better characterize minority class distributions and decision boundaries (Rezvani et al., 2024). In addition to kernel space mapping, complemented subspace representation also offers a novel approach to imbalanced classification. The CSCR-W (Complemented subspace-based weighted collaborative representation model) (Li et al., 2024b) incorporates complemented subspace

* Corresponding author at: School of Computer Science, Minnan Normal University, Zhangzhou, 363000, China.
E-mail address: zyf@mnnu.edu.cn (Y. Zheng).

https://doi.org/10.1016/j.neunet.2026.109839
Received 18 June 2026; Received in revised form 12 September 2026; Accepted 29 October 2026
Available online 3 November 2026
0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

regularization into collaborative representation and adaptively determines class weights according to the original class distribution to enhance minority class representation. Subsequently, AWLICSR (Imbalanced complemented subspace representation with adaptive weight learning) (Li et al., 2024c) further integrates intra-class and inter-class density information to learn sample-level weights, assigning greater importance to minority samples while retaining a closed-form solution. In addition to these complemented subspace-based collaborative representation methods, the LSIBLS (Imbalanced broad learning system with label relaxation and sample weight adaptation) (Li et al., 2025a) combines genetic programming-based feature optimization, label relaxation guided by pairwise label relationships, and density-based sample weighting to mitigate the model's bias toward the majority class. However, the effectiveness of these approaches depends on the suitability of their weighting strategies, feature representations, and learning mechanisms for the data distribution, which may limit their applicability across different learning scenarios.

Data-level methods alleviate class imbalance by adjusting the sample distribution in the data space. They are relatively simple to implement and do not depend on specific classification models (He & Garcia, 2009). According to how the data distribution is adjusted, these methods can be broadly divided into undersampling and oversampling. Undersampling methods balance the class distribution by removing part of the majority samples, which may lead to the loss of informative majority class information (Sun et al., 2024b). In contrast, oversampling methods balance the data distribution by increasing minority samples, thereby helping the model better learn minority class characteristics. Random oversampling is one of the simplest oversampling methods. It balances the data distribution by directly duplicating existing minority samples, but this may increase the risk of overfitting. Instead of directly duplicating minority samples, Chawla et al. proposed SMOTE (Synthetic Minority Over-sampling Technique) (Chawla et al., 2002), which generates new minority samples by interpolating between minority samples and their nearest neighbors, thereby improving sample diversity. However, noisy samples in the original data may also be selected for interpolation, causing noise to spread to neighboring regions (Matharaarachchi et al., 2024; Sáez et al., 2015; Tao et al., 2024).

To address these issues, researchers have proposed methods such as RN-SMOTE (Reduced Noise-SMOTE) (Arafa et al., 2022) and SMOTE-kTLNN (a hybrid re-sampling method based on SMOTE and a two-layer nearest neighbor classifier) (Sun et al., 2024a) to identify and remove noise introduced during sample synthesis. However, these methods usually perform noise filtering after oversampling, rather than preventing noisy samples from participating in the synthesis process. Once noisy samples participate in interpolation, noise may be propagated into synthetic samples and become difficult to detect during subsequent filtering. To prevent noisy samples from participating in synthesis, Cai et al. proposed LSMOTE (Link-based Synthetic Minority Oversampling Technique) (Cai et al., 2024), which removes noisy samples before oversampling. However, its reliance on local-neighbor information may make noise identification unreliable when noisy and normal samples are highly mixed, or the data distribution is complex. In addition to noise removal, identifying minority borderline samples is important for characterizing decision boundaries. To better characterize decision boundary regions, Borderline-SMOTE (Han et al., 2005) focuses on minority samples near class boundaries, whereas ADASYN (Adaptive Synthetic Sampling) (He et al., 2008) further emphasizes hard-to-learn minority samples through adaptive sample allocation. However, using a fixed k-nearest neighbor mechanism to identify borderline or hard-to-learn minority samples may ignore variations in local structure and sample density, thereby reducing the effectiveness of borderline sample identification and new sample generation (Zhu et al., 2016). Beyond sample reliability and neighbor selection, the interpolation mechanism also plays an important role in determining the distribution and diversity of synthetic samples. Most interpolation-based oversampling methods generate samples through linear interpolation, which can partly reduce the risk of falling into majority class regions but still confines synthetic samples to local line-segment spaces between original samples. Therefore, these methods may fail to capture the complex underlying distribution of minority samples (Dai et al., 2025).

In summary, existing oversampling methods have improved classifier performance through noise suppression, neighbor selection, decision boundary enhancement, and diversity-oriented sample generation. Nevertheless, their effectiveness is still limited by unreliable noise filtering, fixed neighborhood dependence, and restricted sample diversity caused by linear interpolation. Therefore, an oversampling method that jointly considers sample reliability, adaptive neighbor selection, and diverse sample generation is needed.

To this end, this paper proposes RAN-SMOTE. First, the method estimates posterior misclassification rates of minority samples using heterogeneous base learners and an adaptive repetition mechanism based on prediction probability stability. Noisy minority samples are removed according to the misclassification rates before synthesis. Next, for each retained minority sample, a candidate neighbor set is constructed using Euclidean distance, after which the posterior misclassification rate is used to determine an adaptive neighborhood size and select reliable neighbors through a composite score combining distance and misclassification risk. Posterior misclassification rates are further used to assign sampling weights to minority samples. For sample generation, two reliable neighbors are probabilistically selected, and a Beta distribution-based triangular interpolation strategy is employed to synthesize new minority samples.

The main contributions of this paper are as follows:

- An adaptive posterior misclassification rate estimation strategy based on heterogeneous base learners is developed to characterize minority samples in terms of misclassification risk and identify noisy samples prior to oversampling.
- An adaptive neighbor selection mechanism is proposed to integrate Euclidean distance with the posterior misclassification rate and adjust the neighborhood size according to sample-level misclassification risk, thereby enabling the selection of reliable neighbors under varying local-density conditions.
- A misclassification rate-guided triangular interpolation mechanism based on the Beta distribution is introduced to enhance the diversity of synthetic minority samples.
- Experiments on 31 public imbalanced datasets demonstrate the classification effectiveness of RAN-SMOTE, while label noise experiments on five datasets evaluate its robustness under different noise rates.

The remainder of this paper is organized as follows. Section 2 introduces related work on imbalanced learning. Section 3 describes the working principle of the proposed method. Section 4 presents the experimental results and provides the corresponding analysis. Section 5 concludes the paper and discusses future research directions.

2. Related work

In imbalanced learning, reducing the bias of classification models and improving the recognition of the minority class have become active research topics (Chen et al., 2024b; Hida & Nascimento, 2026). Oversampling methods have been widely studied because they alleviate class imbalance by enriching the distribution of minority samples and can be easily applied to different classifiers. However, many interpolation-based oversampling methods generate new samples through local linear interpolation based on k-nearest neighbors. Under complex data distributions, they often fail to capture the distributional characteristics of minority samples. This weakens their ability to identify noisy samples, thereby affecting the subsequent neighbor selection and sample generation processes. This may cause sensitivity to neighbor selection, insufficient synthetic sample diversity, increased class overlap, and blurred

decision boundaries (Elreedy & Atiya, 2019; Idwan et al., 2025). To address these issues, researchers have proposed many improved oversampling methods by focusing on sample distribution feature identification, neighbor selection optimization, and sample generation mechanism design.

For sample distribution characterization, Zhang et al. proposed IW-SMOTE (Instance Weighted SMOTE) (Zhang et al., 2022), which estimates minority sample importance through indirect distribution exploration and sample weighting, and preferentially oversamples difficult but non-noisy minority instances. This method mainly focuses on sample-level learning difficulty, while its ability to evaluate the quality of candidate generation regions between samples remains relatively limited. To further incorporate distributional information into the oversampling process, Wang and Liu proposed ADDPC-SMOTE (an oversampling algorithm based on adaptive density difference peak clustering and spatial distribution entropy) (Wang & Liu, 2023), which characterizes minority sample distributions using density difference peak clustering and spatial distribution entropy. However, it mainly focuses on density and entropy-based distribution characterization and provides a limited assessment of the quality of candidate generation regions between minority samples. Furthermore, Qiu et al. proposed VS-SMOTE (an oversampling method based on high-value space selection) (Qiu et al., 2025), which characterizes candidate generation spaces by selecting high-value regions between minority samples for synthetic sample generation. However, its evaluation mainly focuses on the local value of candidate regions, while the overall distribution structure and global representativeness of minority samples are not fully considered.

In practical imbalanced datasets, imbalance may exist not only between classes but also within the minority class, where minority samples can be densely distributed in some regions while remaining sparse in others. Such intra-class density variation makes it difficult for a fixed k-nearest neighbor strategy to adapt to different local structures, thereby reducing the reliability of neighbor selection. To address this issue, Yi et al. proposed ASN-SMOTE (a synthetic minority oversampling method with adaptive qualified synthesizer selection) (Yi et al., 2022), which improves neighbor selection by adaptively selecting qualified minority samples for synthesis. However, this filtering strategy may be overly conservative and may exclude boundary minority samples that contain useful discriminative information. To improve the adaptability of neighbor parameters to different data distributions, Li et al. proposed AMSCO (Adaptive Multi-objective Swarm Crossover Optimization) (Li et al., 2018), which uses swarm intelligence optimization to adaptively determine neighbor parameters according to the overall data distribution. However, its optimization process may introduce high computational cost, and the resulting global parameter is still insufficient to capture local structural differences among minority samples. To overcome the limitation of using a single global neighbor parameter, Guan et al. proposed ENaN-SMOTE (Extended Natural Neighbor-based SMOTE) (Guan et al., 2023), which uses the Extended Natural Neighborhood to construct sample-specific neighbor sets. However, its local neighborhood-based construction may still be affected by noisy samples or severe class overlap.

When generating synthetic samples, oversampling methods are often limited by local linear interpolation in terms of sample diversity. Traditional interpolation-based methods generate synthetic samples mainly along line segments between minority sample pairs, leading to similar sample patterns and limited diversity. To overcome the limitation of local linear interpolation, Douzas et al. proposed G-SMOTE (Geometric SMOTE) (Douzas & Bacao, 2019), which expands the geometric generation region around minority samples to improve the diversity of synthetic samples. However, in regions with severe class overlap, the expanded generation region may be difficult to control precisely and may introduce noisy samples. To further expand the sample generation space, Li et al. proposed MSEO (Multi-vector Stochastic Exploration Oversampling) (Li et al., 2024a), which expands the generation space through multi-vector random exploration to improve the diversity of synthetic samples. However, its reliance on random exploration may cause generated samples to deviate from the true minority class distribution in regions with class overlap or strong noise interference. To expand the generation range while strengthening the association between the generation process and the original minority class structure, Li et al. proposed ISMOTE (Improved SMOTE) (Li et al., 2025b), which improves the constraints on the sample generation space by introducing adaptive perturbation around reference samples between original minority samples. However, although this strategy expands the generation range and increases sample diversity, the perturbation may disturb the original data structure and cause generated samples to deviate from the minority class distribution. In addition to directly expanding the generation space, some methods attempt to make sample generation more targeted through region partitioning mechanisms. Wang et al. proposed HS-SMOTE (an oversampling method for multiple dynamic interpolations based on a regular hexagon scoring mechanism) (Wang et al., 2025), which combines a regular hexagon scoring mechanism with multiple dynamic interpolations to make synthetic sample generation more targeted and diverse. However, because the scoring regions are constructed in a space reduced using principal component analysis (PCA), some structural information may be lost, causing inaccurate sample associations and affecting the distribution quality of generated samples.

Unlike the above methods, this paper integrates sample distribution characterization, adaptive selection of reliable neighbors, and diverse synthetic sample generation to address these issues, as summarized in Table 1. The next section introduces the algorithm procedure and implementation details.

3. Model construction

Most existing noise identification mechanisms are based on the local consistency assumption of k-nearest neighbors and usually rely on distance metrics to describe the neighborhood structure of samples. However, local neighborhood structures may become unstable in complex data scenarios, making noise identification unreliable. For example, minority samples near the decision boundary may be misidentified as noise when their neighborhoods contain many majority samples, as shown in Fig. 1(a), which may cause the loss of important boundary information. Similarly, interpolation-based oversampling methods usually construct local neighborhoods using distance-based k-nearest neighbors. However, in imbalanced data, minority samples often have uneven local densities, making a fixed neighborhood size difficult to adapt to different local structures, as shown in Fig. 1(b). A large k may introduce samples from other clusters and generate synthetic samples in majority class regions, whereas a small k may produce overly similar samples in dense minority regions, increasing the risk of overfitting.

In addition, traditional linear interpolation methods usually generate new samples along the line segments between minority sample pairs, which may lead to similar synthetic samples and insufficient diversity in sparse minority regions. If the interpolation range is simply expanded to improve diversity, the generated samples may deviate from the true distribution of minority samples and may even introduce or propagate noise. To address these problems, this paper proposes RAN-SMOTE for binary imbalanced data. The overall framework of the proposed method is shown in Fig. 2.

3.1. Basic definitions

To provide a unified description of the proposed method, the imbalanced classification dataset is denoted as $D = \{(x_i, y_i)\}_{i=1}^{n}$, where n is the total number of samples, $x_i = [x_i^1, x_i^2, \ldots, x_i^t] \in \mathbb{R}^t$ represents the t-dimensional feature vector of the ith sample, and $y_i \in \{0, 1\}$ denotes its class label. In the binary imbalanced classification scenario considered in this paper, the minority class is denoted by $y = 1$, while

Table 1
Comparison of different oversampling methods.

Method	Identify the sample distribution	Adaptive neighbor selection	Diverse sample generation
IW-SMOTE (2022)	✓	×	×
ADDPC-SMOTE (2023)	✓	×	×
VS-SMOTE (2025)	✓	×	✓
ASN-SMOTE (2022)	×	✓	×
AMSCO (2017)	×	✓	×
ENaN-SMOTE (2023)	✓	✓	×
G-SMOTE (2019)	×	×	✓
MSEO (2024)	×	×	✓
ISMOTE (2025)	×	×	✓
HS-SMOTE (2024)	✓	×	✓
RAN-SMOTE (ours)	✓	✓	✓

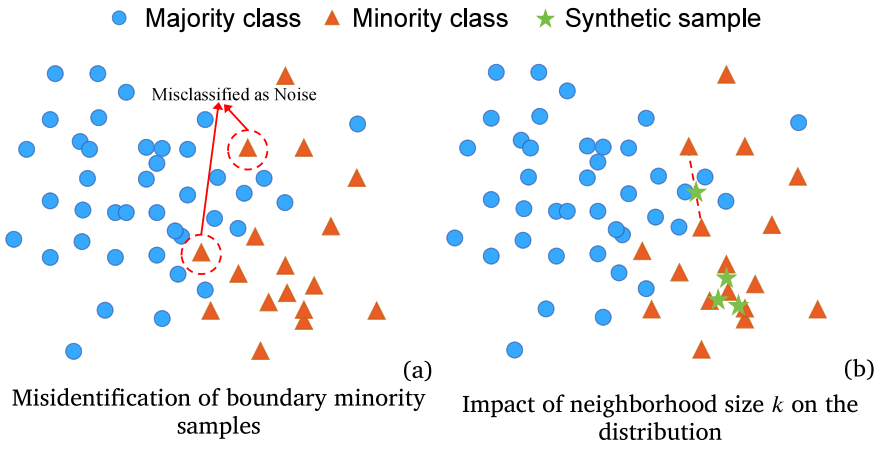


Fig. 1. An example of noise misidentification and neighborhood size limitations.

the majority class is denoted by $y = 0$. Accordingly, the minority sample set and majority sample set are defined as $X_{\min} = \{x_i \mid y_i = 1\}$ and $X_{\text{maj}} = \{x_i \mid y_i = 0\}$, respectively, and their corresponding sample sizes are denoted as $n_{\min} = |X_{\min}|$ and $n_{\text{maj}} = |X_{\text{maj}}|$. The imbalance ratio (IR) of the dataset is then defined as $IR = n_{\text{maj}}/n_{\min}$.

3.2. Sample posterior misclassification rate

In imbalanced learning, samples are usually divided into different types. Among them, borderline samples and noisy samples have a high proportion of samples from other classes in their neighborhoods and are prone to being misclassified. To robustly estimate the posterior probability that each sample is misclassified in imbalanced and noisy scenarios, this section proposes an estimation method for the sample posterior misclassification rate.

To ensure that the misclassification rate is calculated under the same data distribution, stratified K-fold cross-validation is used to divide the dataset into K folds. One fold is used as the validation fold, and the remaining folds are used as the training folds. To adapt to different data scales and reduce computational cost, the number of folds is adaptively determined by Eq. (1):

$$K = \min\left(\max\left(2, \left\lceil \frac{n_{\min}}{5} \right\rceil\right), 5\right). \tag{1}$$

Then, L heterogeneous base learners are trained on each training fold and applied to the corresponding validation fold to obtain the probability estimate p_{il} for each sample. When the probability estimates of a sample contain values close to 1 or 0, directly using the arithmetic mean in the probability space to calculate the misclassification rate can be strongly affected by extreme values. Therefore, the probability estimates of each sample are mapped to the logit space before arithmetic averaging. Specifically, the bounded probability space is transformed into an unbounded real-valued space by Eq. (2). To avoid numerical instability, probability estimates are clipped to $\tilde{p}_{il} = \min(\max(p_{il}, \epsilon), 1-\epsilon)$ before the logit transformation, where ϵ is a small positive constant. The arithmetic mean is then calculated in this space using Eq. (3). The result is mapped back to the probability space by Eq. (4).

$$z_{il} = \text{logit}(\tilde{p}_{il}) = \ln\left(\tilde{p}_{il}/(1-\tilde{p}_{il})\right). \tag{2}$$

$$z_i = \frac{1}{L}\sum_{l=1}^{L} z_{il}, \tag{3}$$

$$p_i = \text{logistic}(z_i) = (1+\exp(-z_i))^{-1}. \tag{4}$$

Here, z_{il} denotes the logit-space value of the estimated probability for the ith sample produced by the lth heterogeneous base learner. z_i denotes the average logit-space value of the ith sample over the L base learners, and p_i denotes the final value mapped back to the probability space.

Example 1. Suppose that the probability estimates of a sample obtained from four base classifiers are 0.6, 0.9, 0.9, and 0.95. Direct arithmetic averaging in the probability space gives 0.8375, which is affected by the smaller value. In the logit space, these probabilities are mapped to 0.405, 2.197, 2.197, and 2.944 by Eq. (2). The average value calculated by Eq. (3) is 1.936, which is then mapped back to the probability space by Eq. (4), yielding 0.874. This example shows that averaging in the logit space can reduce the interference caused by outlier predictions from individual base classifiers and better reflect the overall prediction trend of the base classifiers.

Due to the uncertainty caused by random splitting, cross-validation results may vary across different runs. Therefore, repeated stratified K-fold cross-validation is adopted to estimate the misclassification rate. To reduce the influence of randomness, the number of repetitions is set to at least min_R = 5. Meanwhile, considering that some samples

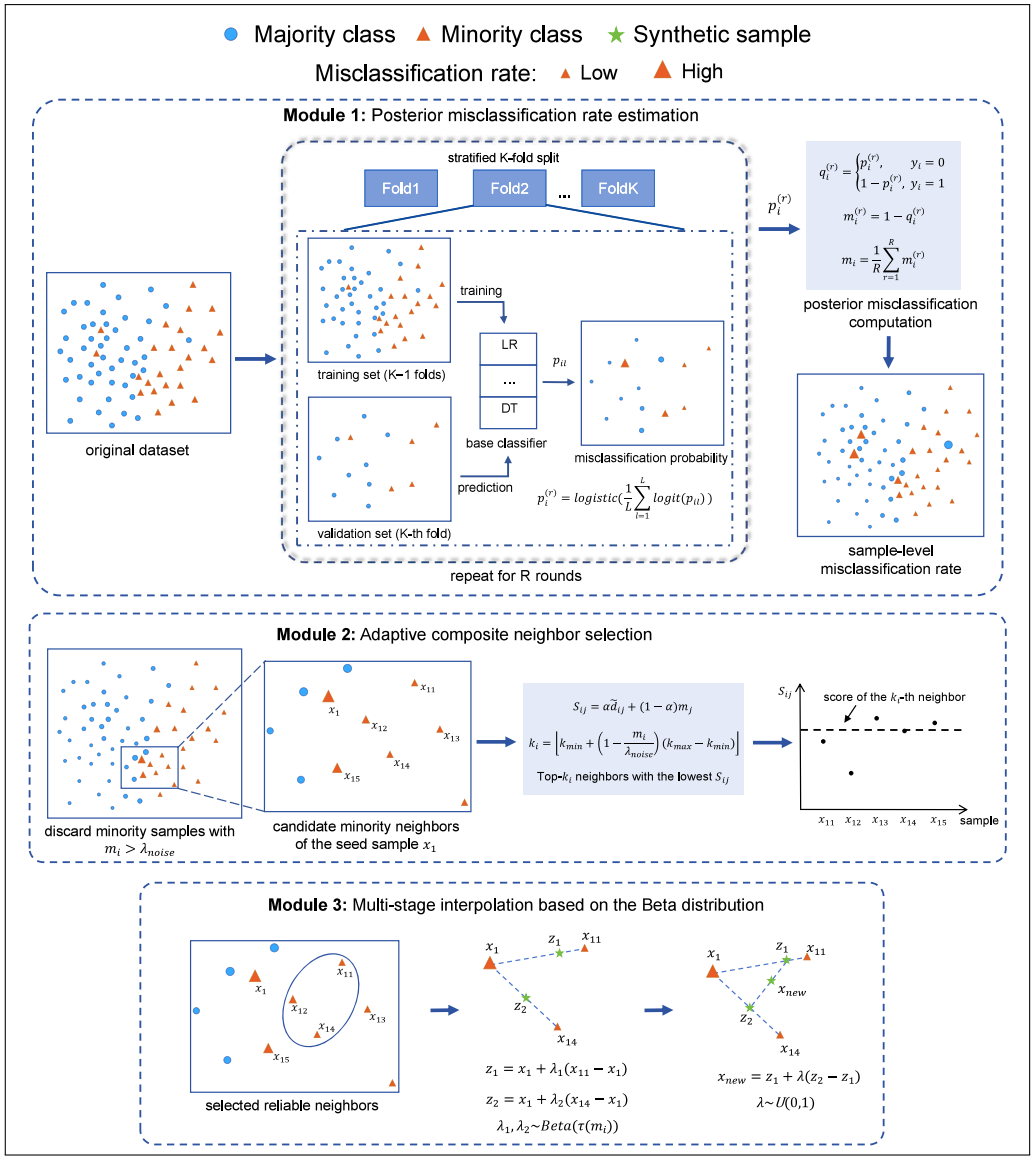


Fig. 2. The framework of the proposed RAN-SMOTE.

may be difficult to learn, the process is terminated when R reaches the upper limit max_R = 30. For sample x_i, the correct classification rate consistent with its label in round r is defined as

$$q_i^{(r)} = \begin{cases} p_i^{(r)}, & y_i = 1, \\ 1 - p_i^{(r)}, & y_i = 0. \end{cases} \tag{5}$$

Then, the misclassification rate of sample x_i is given by:

$$m_i^{(r)} = 1 - q_i^{(r)}. \tag{6}$$

Therefore, after R repeated rounds, the final posterior misclassification rate of each sample is defined as

$$m_i = \frac{1}{R} \sum_{r=1}^{R} m_i^{(r)}. \tag{7}$$

Since computational cost increases linearly with the number of repetitions, an adaptive scheme is designed to adjust R and balance efficiency with estimation stability. The procedure is as follows.

First, the Welford method (Welford, 1962) is used to recursively estimate the variance of each sample in the logit space across repeated rounds. Let the result of the rth round be denoted as the vector $\boldsymbol{z}^{(r)} = (z_1^{(r)}, \ldots, z_n^{(r)})$. The mean of each sample over the first $r-1$ rounds is denoted as $\bar{\boldsymbol{z}}^{(r-1)} = (\bar{z}_1^{(r-1)}, \ldots, \bar{z}_n^{(r-1)})$. The difference between the result of the rth round and the mean of the first $r-1$ rounds is denoted as $\boldsymbol{\delta} = \boldsymbol{z}^{(r)} - \bar{\boldsymbol{z}}^{(r-1)}$. The mean of each sample after the rth round is then recursively updated as $\bar{\boldsymbol{z}}^{(r)} = \bar{\boldsymbol{z}}^{(r-1)} + \frac{\boldsymbol{\delta}}{r}$. The variance accumulation term $\boldsymbol{M2}^{(r)}$ used for estimating the sample variance is defined as

$$\boldsymbol{M2}^{(r)} = \boldsymbol{M2}^{(r-1)} + \boldsymbol{\delta} \odot \left(\boldsymbol{z}^{(r)} - \bar{\boldsymbol{z}}^{(r)}\right). \tag{8}$$

Then, Eq. (9) is used to calculate the sample variance estimate $s_i^{2(r)}$ of the ith sample after the rth round. The obtained sample variance estimate is used to calculate the standard error of the sample mean by Eq. (10). Since the calculated standard error is in the logit space, Eq. (11) is used to map the standard error from the logit domain to the probability domain for interval estimation. The mapped standard error is then used to measure the reliability of the sample mean and determine whether early stopping can be performed.

$$s_i^{2(r)} = \begin{cases} \dfrac{M2_i^{(r)}}{r-1}, & r \geq 2, \\ 0, & r = 1 \end{cases} \tag{9}$$

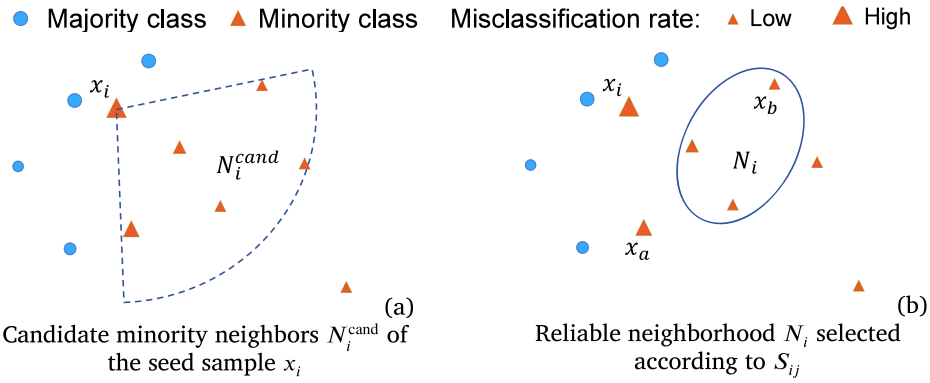


Fig. 3. Illustration of the adaptive composite neighborhood construction.

where $M2_i^{(r)}$ is the ith component of $\boldsymbol{M2}^{(r)}$.

$$SE\left(\bar{z}_i^{(r)}\right) = \sqrt{\frac{s_i^{2(r)}}{r}}, \tag{10}$$

$$SE\left(\bar{p}_i^{(r)}\right) \approx SE\left(\bar{z}_i^{(r)}\right) \cdot \bar{p}_i^{(r)}\left(1-\bar{p}_i^{(r)}\right), \tag{11}$$

where $\bar{p}_i^{(r)} = [1+\exp(-\bar{z}_i^{(r)})]^{-1}$ denotes the probability obtained by applying the logistic function to the running mean logit of sample x_i.

Since this paper focuses on the stability of classifier output probabilities for minority samples during repeated sampling, only the confidence interval half-widths of minority samples in the probability domain are considered. Specifically, a half-width not exceeding 0.05 at the 95% confidence level is used as the stability criterion. Thus, the number of minority samples that satisfy the stability requirement in the rth round is defined as

$$S^{(r)} = \sum_{i=1}^{n_{\min}} \mathbf{1}\left(1.96 \cdot SE\left(\bar{p}_i^{(r)}\right) \le 0.05\right), \tag{12}$$

where $\mathbf{1}(\cdot)$ denotes the indicator function, which equals 1 when the specified condition holds and 0 otherwise.

If all minority samples are required to reach stability, the number of repetitions may become too large due to some hard-to-learn samples, resulting in a high computational cost. Therefore, this paper uses the proportion of stable minority samples as the early stopping criterion. When $\frac{S^{(r)}}{n_{\min}} \ge 0.9$, the probability estimates of most minority samples are considered stable, and early stopping is triggered.

3.3. Adaptive composite neighbor selection

The proposed method maps the posterior misclassification rate of each minority sample to its corresponding neighborhood size. Let $\lambda_{\text{noise}} \in (0,1)$ denote the noise threshold, and let m_i denote the posterior misclassification rate of minority sample x_i obtained from Eq. (7). A minority sample is regarded as safe if its posterior misclassification rate is below λ_{noise}. Accordingly, the safe minority sample set is defined as $X_{\text{safe}} = \{x_i \in X_{\min} \mid m_i < \lambda_{\text{noise}}\}$. For samples in X_{safe} with lower misclassification rates, larger neighborhoods are used to improve the diversity of synthetic samples. For those with relatively higher misclassification rates, smaller neighborhoods are used to reduce the risk of generating noisy samples. The influence of λ_{noise} is examined in Section 4.2.1, and its value is set to 0.7 in the subsequent experiments based on the sensitivity analysis.

The lower and upper bounds of the adaptive neighborhood size k_i are denoted by $k_{\min}$ and $k_{\max}$, respectively, and are set to $k_{\min} = 3$ and $k_{\max} = \max\left(k_{\min}, \left\lfloor\sqrt{n_{\min}}\right\rfloor\right)$. According to the misclassification rate of each sample, Eq. (13) is used to adaptively select a different number of neighbors k_i for each sample.

$$k_i = \left\lfloor k_{\min} + \left(1 - \frac{m_i}{\lambda_{\text{noise}}}\right)(k_{\max} - k_{\min}) \right\rfloor. \tag{13}$$

Compared with traditional methods that select neighbors only based on Euclidean distance, a composite neighbor score is designed by integrating Euclidean distance and neighboring sample misclassification rates to select more reliable neighbors. The process is as follows.

Since sample distance and misclassification rate have different scales, the neighbor distance is first normalized. For a seed sample x_i, where $x_i \in X_{\text{safe}}$, $k_{\max}$ is used for neighbor search to obtain the candidate minority neighbor set N_i^{cand}, as shown in Fig. 3(a). For a neighboring sample x_{ij}, where $x_{ij} \in N_i^{\text{cand}}$, the distance between x_{ij} and the seed sample x_i is denoted as d_{ij}. For the candidate minority neighbor set N_i^{cand} of the seed sample x_i, let the minimum nearest neighbor distance be $d_i^{\min} = \min_{x_{ij} \in N_i^{\text{cand}}} d_{ij}$ and the maximum nearest neighbor distance be $d_i^{\max} = \max_{x_{ij} \in N_i^{\text{cand}}} d_{ij}$. Then, the normalized distance between the seed sample x_i and its neighboring sample x_{ij} is defined as

$$\tilde{d}_{ij} = (d_{ij} - d_i^{\min})/(d_i^{\max} - d_i^{\min}), \tag{14}$$

if $d_i^{\min} = d_i^{\max}$, set $\tilde{d}_{ij} = 0$.

To control the importance of distance and misclassification rate in the composite neighbor score, a weighted sum form is adopted. The composite neighbor score is defined as

$$S_{ij} = \eta \tilde{d}_{ij} + (1-\eta) m_j, \tag{15}$$

where m_j denotes the posterior misclassification rate of the candidate neighbor x_{ij}, and $\eta \in [0,1]$ controls the relative importance of normalized distance and misclassification rate. A larger η places more emphasis on normalized distance, whereas a smaller η places more emphasis on misclassification rate.

When samples have similar distances to the seed sample, those with lower misclassification rates are preferred as reliable neighbors, thereby avoiding nearby but unreliable samples. In this paper, the misclassification rate and distance are considered equally important, and thus $\eta = 0.5$ is used in the subsequent experiments. The candidate neighbors are sorted in ascending order according to S_{ij}, and the top k_i samples are selected to form the reliable neighbor set N_i of the seed sample x_i, as shown in Fig. 3(b). Although x_a is closer to x_i than x_b, its higher misclassification rate leads to a larger composite score, so x_a is excluded from N_i.

For sample weighting, the posterior misclassification rate is used to measure the learning difficulty of each minority sample. Samples with higher misclassification rates are more likely to lie near the decision boundary and are therefore assigned larger sampling weights to generate more synthetic samples. Therefore, for each minority sample x_i, a sampling weight ω_i is assigned using Eq. (16).

$$\omega_i = \frac{m_i}{\sum_{x_j \in X_{\text{safe}}} m_j} \tag{16}$$

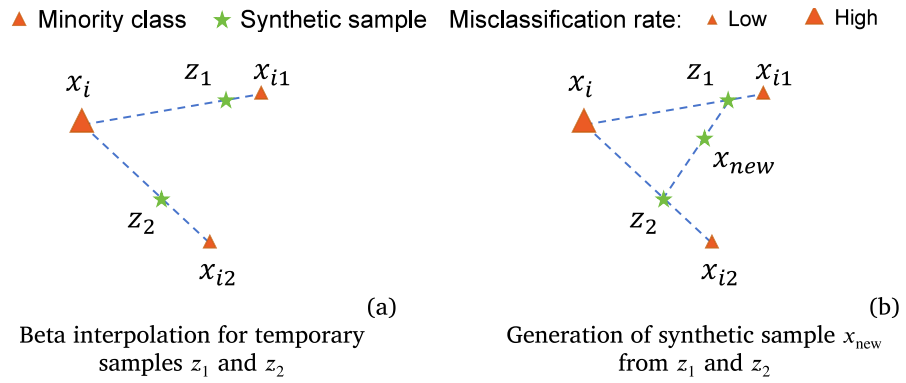


Fig. 4. Synthetic sample generation via Beta-guided triangular interpolation.

When x_i is selected as a seed sample, the expected number of synthetic samples generated from it is calculated as $n_i^{\text{syn}} = (n_{\text{maj}} - n_{\text{min}}) \cdot \omega_i$. Since the actual number of generated samples must be an integer, the integer part of each expected allocation value is first taken in the implementation, and the remaining sample quota is then assigned in descending order according to the fractional parts.

3.4. Triangular interpolation based on the beta distribution

Traditional interpolation-based oversampling algorithms generate samples along the line segment between a seed sample and one neighbor, which may cause sample overlap when the seed sample has a high sampling weight. To alleviate this issue, this paper introduces Beta distribution-based triangular interpolation.

A non-noisy minority sample x_i is selected as the seed sample. For each reliable neighbor $x_{ij} \in N_i$, the selection probability p_{ij} is calculated according to Eq. (17) based on its composite score S_{ij}, where a lower score indicates a higher probability of being selected for sample synthesis.

$$p_{ij} = \frac{1/S_{ij}}{\sum_{k \in N_i} 1/S_{ik}} \tag{17}$$

Since seed samples with higher misclassification rates are assigned more synthetic samples, interpolation should account for the relative reliability of the selected samples. The Beta distribution is well suited to our objective for three reasons. First, its support is bounded within $(0, 1)$, keeping each temporary sample on the line segment between the selected sample pair. Second, its parameters allow the expected interpolation position to favor the sample with a lower posterior misclassification rate. Third, the concentration parameter controls interpolation variability without changing this expected position. Suppose there are two samples x_a and x_b, with corresponding misclassification rates m_a and m_b. The weights induced by the misclassification rates are defined as:

$$\rho_a = \frac{1 - m_a}{2 - m_a - m_b}, \qquad \rho_b = \frac{1 - m_b}{2 - m_a - m_b}. \tag{18}$$

where $\rho_a, \rho_b \in (0, 1)$. A larger ρ_a indicates that sample x_a is more reliable, and the synthesis position is expected to be closer to x_a.

The interpolation coefficient is sampled from a Beta distribution, i.e., $\lambda \sim \text{Beta}(\alpha, \beta)$, $\lambda \in (0, 1)$, where $\alpha = \gamma \cdot \rho_b$, $\beta = \gamma \cdot \rho_a$, and γ is the concentration parameter. This parameter controls the variance of the Beta distribution without changing the expected interpolation position determined by the sample misclassification rates. Larger values of γ concentrate the interpolation coefficients more closely around their mean, whereas smaller values allow greater variability. The parameter sensitivity analysis in Section 4.2.1 supports $\gamma = 5$ as a reasonable default. Accordingly, $\gamma = 5$ is used in the remaining experiments.

Then, as shown in Fig. 4(a), two reliable neighbors x_{i1} and x_{i2} are selected according to the selection probability p_{ij} and paired with the seed sample. The interpolation coefficients $\lambda_1, \lambda_2 \sim \text{Beta}(\alpha, \beta)$, where $\lambda_1, \lambda_2 \in (0, 1)$, are sampled from a Beta distribution to synthesize two temporary samples:

$$z_1 = x_i + \lambda_1(x_{i1} - x_i), \qquad z_2 = x_i + \lambda_2(x_{i2} - x_i). \tag{19}$$

Finally, as shown in Fig. 4(b), where $\lambda_3 \in (0, 1)$, and interpolation is then performed between the two temporary samples z_1 and z_2:

$$x_{\text{new}} = z_1 + \lambda_3(z_2 - z_1). \tag{20}$$

3.5. Potential extension to multi-class imbalanced learning

In practical applications, class imbalance may occur among multiple classes. Although RAN-SMOTE is developed and evaluated for binary classification, its main components could be extended to multi-class imbalanced learning. Suppose that there are C classes and that the lth base learner outputs a probability vector $\boldsymbol{p}_{il}^{(r)} = (p_{il1}^{(r)}, \ldots, p_{ilC}^{(r)})$ for sample x_i in repetition r. To avoid numerical instability, probability estimates are clipped to $\tilde{p}_{ilc}^{(r)} = \min(\max(p_{ilc}^{(r)}, \epsilon), 1 - \epsilon)$ before taking logarithms, where ϵ is a small positive constant. A possible extension of the sample misclassification rate estimation to multi-class settings can be formulated as follows:

$$p_{ic}^{(r)} = \frac{\exp\left(\frac{1}{L}\sum_{l=1}^{L} \ln \tilde{p}_{ilc}^{(r)}\right)}{\sum_{h=1}^{C} \exp\left(\frac{1}{L}\sum_{l=1}^{L} \ln \tilde{p}_{ilh}^{(r)}\right)}, \tag{21}$$

Here, $p_{ic}^{(r)}$ denotes the aggregated probability that sample x_i belongs to class c in repetition r, and $p_{ilc}^{(r)}$ denotes the corresponding probability estimated by the lth base learner. For a sample with ground-truth label y_i, its posterior misclassification rate is defined as:

$$m_i = \frac{1}{R}\sum_{r=1}^{R}\left(1 - p_{i,y_i}^{(r)}\right). \tag{22}$$

This quantity represents the total probability assigned to incorrect classes, averaged over R repetitions, and provides an estimate of sample-level classification difficulty in the multi-class setting. When $C = 2$, it reduces to the posterior misclassification rate defined for binary classification, consistent with the proposed method.

In the subsequent sampling procedure, each minority class c requiring oversampling could be processed separately. The maximum number of neighbors $k_{\max,c}$ would be determined from the class sample size n_c, and potentially noisy samples would be filtered using their posterior misclassification rates m_i. Candidate and reliable neighborhoods would then be constructed exclusively from the retained samples of the same class, with neighborhood sizes bounded by the number of available same-class neighbors. Finally, the proposed composite neighbor scores and Beta distribution-guided triangular interpolation would be used to generate synthetic samples for that class.

The above discussion outlines a potential extension of RAN-SMOTE. The experiments in this study focus on binary classification, while the

implementation and systematic evaluation of the multi-class extension are left for future work.

3.6. Analysis of the RAN-SMOTE algorithm

Following the execution flow of RAN-SMOTE, this section first analyzes the time complexity of the three stages with reference to their corresponding procedures in Algorithms 1–3. Then, the overall time complexity of RAN-SMOTE is derived.

1. **Sample posterior misclassification rate estimation:** The algorithm first identifies $X_{\min}$ and X_{maj} from D and obtains their sizes $n_{\min}$ and n_{maj}. The posterior misclassification rate is then estimated by repeated stratified K-fold cross-validation. In each round, L heterogeneous base learners are trained and evaluated on the validation folds. Let $T_l(n,t)$ denote the cost of the lth learner on n samples with t features. The complexity of one round is $O(K\sum_{l=1}^{L} T_l(n,t))$. Updating the logit-domain mean and variance terms and counting stable samples requires $O(n)$. Therefore, if the final number of repeated rounds is R, the time complexity of this stage is $O\left(R\left(K\sum_{l=1}^{L} T_l(n,t)+n\right)\right)$.
2. **Adaptive composite neighbor construction:** After obtaining the average misclassification rate m_i, the algorithm detects noisy minority samples and determines the adaptive neighbor number k_i. Let $n'_{\min}$ be the number of remaining minority samples after noise removal. For each non-noisy sample, searching the candidate neighbor set costs $O(n'_{\min}t)$, while distance computation, normalization, and composite score calculation cost $O(k_{\max}t)$. Sorting the candidates costs $O(k_{\max}\log k_{\max})$. Thus, the cost for one sample is $O(n'_{\min}t + k_{\max}t + k_{\max}\log k_{\max})$. Since $k_{\max} = \max(k_{\min}, \lfloor\sqrt{n_{\min}}\rfloor)$ and $k_{\min}$ is constant, this can be written as $O(n'_{\min}t + \sqrt{n_{\min}}t + \sqrt{n_{\min}}\log\sqrt{n_{\min}})$. For all $n'_{\min}$ non-noisy samples, the total complexity of this stage is approximately $O((n'_{\min})^2 t)$.
3. **Triangular interpolation based on the Beta distribution:** After obtaining the reliable neighborhoods, the algorithm generates synthetic samples for each non-noisy minority sample according to n_i^{syn}. Let N_{syn} denote the total number of generated samples. Each sample generation involves selecting two neighbors, sampling Beta-distributed interpolation coefficients, generating two temporary samples, and performing final linear interpolation. Since these operations are linear in the feature dimension t, the cost per sample is $O(t)$, and the total complexity of this stage is $O(N_{\mathrm{syn}}t)$.

In summary, the total time complexity of the RAN-SMOTE algorithm is $O\left(R(K\sum_{l=1}^{L} T_l(n,t)+n) + (n'_{\min})^2 t + N_{\mathrm{syn}}t\right)$. Since R, K, and L are usually small constants, and $n'_{\min} \le n$ and $N_{\mathrm{syn}} \le n$, the above expression can be further simplified as $O\left(\sum_{l=1}^{L} T_l(n,t) + n^2 t\right)$. The source code of RAN-SMOTE is publicly available at https://github.com/zxy66677/ran-smote.

Algorithm 1 Sample posterior misclassification rate.

Require: Dataset D, heterogeneous base learner set $\mathcal{H} = \{h_l\}_{l=1}^{L}$
Ensure: Misclassification rate m_i for each $x_i \in D$, minority set $X_{\min}$
Split D into $X_{\min}$ and X_{maj}, with $n_{\min} = |X_{\min}|$ and $n_{\mathrm{maj}} = |X_{\mathrm{maj}}|$
$K \leftarrow \min\{\max(2, \lceil n_{\min}/5\rceil), 5\}$
Initialize $r \leftarrow 0$, min_R $\leftarrow 5$, max_R $\leftarrow 30$, $\bar{\boldsymbol{z}}^{(0)} \leftarrow \mathbf{0}$ and $\boldsymbol{M2}^{(0)} \leftarrow \mathbf{0}$
while $r <$ max_R **do**
  $r \leftarrow r + 1$
  Perform one round of stratified K-fold cross-validation and obtain $p_{il}^{(r)}$
  for each sample $x_i \in D$ **do**
    for $l = 1$ **to** L **do**
      $\tilde{p}_{il}^{(r)} \leftarrow \min\left(\max\left(p_{il}^{(r)}, \epsilon\right), 1-\epsilon\right)$, $z_{il}^{(r)} \leftarrow \mathrm{logit}\left(\tilde{p}_{il}^{(r)}\right)$
    end for
    $z_i^{(r)} \leftarrow \frac{1}{L}\sum_{l=1}^{L} z_{il}^{(r)}$, $p_i^{(r)} \leftarrow \frac{1}{1+\exp(-z_i^{(r)})}$
    $q_i^{(r)} \leftarrow p_i^{(r)}$ if $y_i = 1$; otherwise $q_i^{(r)} \leftarrow 1 - p_i^{(r)}$
    $m_i^{(r)} \leftarrow 1 - q_i^{(r)}$
  end for
  Update $\boldsymbol{M2}^{(r)}$, $s_i^{2(r)}$, $SE(\bar{z}_i^{(r)})$, and $SE(\bar{p}_i^{(r)})$ according to Eqs. (8)–(11)
  Compute $S^{(r)}$ under the 95% confidence level according to Eq. (12)
  if $r \ge$ min_R **and** $S^{(r)}/n_{\min} \ge 0.9$ **then**
    break
  end if
end while
Set $R \leftarrow r$
for each sample $x_i \in D$ **do**
  $m_i \leftarrow \frac{1}{R}\sum_{r=1}^{R} m_i^{(r)}$
end for
return $\{m_i\}_{i=1}^{|D|}$, $X_{\min}$

Algorithm 2 Adaptive composite neighbor selection.

Require: Dataset D, minority set $X_{\min}$, misclassification rates $\{m_i\}$, and noise threshold λ_{noise}
Ensure: Reliable neighbor sets $\{N_i\}$, synthetic numbers $\{n_i^{\mathrm{syn}}\}$, and safe minority set X_{safe}
Initialize $\eta \leftarrow 0.5$ and $X_{\mathrm{safe}} \leftarrow \varnothing$
for each $x_i \in X_{\min}$ **do**
  if $m_i < \lambda_{\mathrm{noise}}$ **then**
    $X_{\mathrm{safe}} \leftarrow X_{\mathrm{safe}} \cup \{x_i\}$
  end if
end for
for each $x_i \in X_{\mathrm{safe}}$ **do**
  Set k_i via Eq. (13) and construct the candidate neighbor set N_i^{cand}
  Compute the composite score S_{ij} according to Eqs. (14)–(15)
  Select the top k_i candidates with the smallest S_{ij} to form N_i
  $\omega_i \leftarrow m_i / \sum_{x_j \in X_{\mathrm{safe}}} m_j$, $n_i^{\mathrm{syn}} \leftarrow (n_{\mathrm{maj}} - n_{\min}) \cdot \omega_i$
end for
return $\{N_i\}$, $\{n_i^{\mathrm{syn}}\}$, $\{S_{ij}\}$, X_{safe}

4. Experimental analysis

To evaluate RAN-SMOTE on imbalanced classification tasks, this section introduces the experimental settings and comparative results, including datasets, evaluation metrics, classifiers, comparison algorithms, cross-validation protocols, parameter settings, and statistical tests.

4.1. Basic experimental settings

4.1.1. Datasets

This paper evaluates the proposed RAN-SMOTE using 31 public imbalanced datasets and compares it with several oversampling baseline methods. These datasets are obtained from the Knowledge Extraction based on Evolutionary Learning (KEEL) dataset repository.[1] Detailed information on each dataset is presented in Table 2, including the number of features (#Fea.), the number of samples (#Sam.), the number of majority samples (#Maj.), the number of minority samples (#Min.), and the imbalance ratio (IR).

In addition, five datasets covering different sample sizes, feature dimensions, and imbalance degrees are used to evaluate robustness to label noise under varied data conditions. Their characteristics are summarized in Table 3.

[1] https://sci2s.ugr.es/keel/imbalanced.php.

Table 2
Description of the datasets.

No.	Dataset	#Fea.	#Sam.	#Maj.	#Min.	IR
D1	abalone19	8	4174	4142	32	129.44
D2	dermatology-6	34	358	338	20	16.9
D3	ecoli-0-2-6-7_vs_3-5	7	224	202	22	9.18
D4	ecoli-0-6-7_vs_3-5	7	222	200	22	9.09
D5	ecoli-0-6-7_vs_5	6	220	200	20	10
D6	ecoli-0_vs_1	7	220	143	77	1.86
D7	ecoli1	7	336	259	77	3.36
D8	ecoli3	7	336	301	35	8.6
D9	glass-0-1-2-3_vs_4-5-6	9	214	163	51	3.2
D10	glass-0-4_vs_5	9	92	83	9	9.22
D11	glass0	9	214	144	70	2.06
D12	glass1	9	214	138	76	1.82
D13	glass4	9	214	201	13	15.46
D14	haberman	3	306	225	81	2.78
D15	iris0	4	150	100	50	2
D16	led7digit-0-2-4-5-6-7-8-9_vs_1	7	443	406	37	10.97
D17	newthyroid2	5	215	180	35	5.14
D18	pima	8	768	500	268	1.87
D19	poker-8-9_vs_6	10	1485	1460	25	58.4
D20	segment0	19	2308	1979	329	6.02
D21	shuttle-c0-vs-c4	9	1829	1706	123	13.87
D22	vehicle0	18	846	647	199	3.25
D23	wisconsin	9	683	444	239	1.86
D24	yeast-0-5-6-7-9_vs_4	8	528	477	51	9.35
D25	yeast-1-2-8-9_vs_7	8	947	917	30	30.57
D26	yeast-1-4-5-8_vs_7	8	693	663	30	22.1
D27	yeast-1_vs_7	7	459	429	30	14.3
D28	yeast3	8	1484	1321	163	8.1
D29	yeast4	8	1484	1433	51	28.1
D30	yeast5	8	1484	1440	44	32.73
D31	yeast6	8	1484	1449	35	41.4

Algorithm 3 Triangular interpolation based on the beta distribution.

Require: Dataset D, safe minority set X_{safe}, reliable neighbor sets $\{N_i\}$, scores $\{S_{ij}\}$, and synthetic numbers $\{n_i^{\text{syn}}\}$
Ensure: Output dataset D_{output}
Initialize $D_{\text{syn}} \leftarrow \emptyset$ and $\gamma \leftarrow 5$
for each sample $x_i \in X_{\text{safe}}$ **do**
  for $g = 1$ **to** n_i^{syn} **do**
    Randomly select neighbors x_{i1} and x_{i2} from N_i according to Eq. (17)
    Sample λ_1 and λ_2 from the Beta distribution.
    $z_1 \leftarrow x_i + \lambda_1(x_{i1} - x_i)$, $z_2 \leftarrow x_i + \lambda_2(x_{i2} - x_i)$
    Sample $\lambda_3 \sim U(0,1)$, $x_{\text{new}} \leftarrow z_1 + \lambda_3(z_2 - z_1)$
    $D_{\text{syn}} \leftarrow D_{\text{syn}} \cup \{x_{\text{new}}\}$
  end for
end for
$D_{\text{output}} \leftarrow D \cup D_{\text{syn}}$
return D_{output}

Table 3
Datasets used in the label-noise robustness experiments.

No.	Dataset	#Fea.	#Sam.	#Maj.	#Min.	IR
R1	ecoli2	7	336	284	52	5.46
R2	ionosphere	33	351	225	126	1.79
R3	pima	8	768	500	268	1.87
R4	thyroid	21	7200	6666	534	12.48
R5	wdbc	30	569	357	212	1.68

4.1.2. Evaluation metrics

To evaluate the performance of different oversampling methods, geometric mean (G-mean), area under the receiver operating characteristic curve (AUC), and F1-score are used as evaluation metrics. Let the minority class be the positive class, and the majority class be the negative class. The confusion matrix for binary classification is shown in Table 4.

Table 4
Confusion matrix for binary classification.

Actual values	Predicted values	
	Positive	Negative
Positive	True positive (TP)	False negative (FN)
Negative	False positive (FP)	True negative (TN)

Recall measures the proportion of correctly identified minority samples:

$$Recall = TP/(TP + FN). \tag{23}$$

Precision measures the proportion of correct minority class predictions:

$$Precision = TP/(TP + FP). \tag{24}$$

Specificity measures the proportion of correctly identified majority samples:

$$Specificity = TN/(TN + FP). \tag{25}$$

G-mean is the geometric mean of Recall and Specificity, reflecting the balanced recognition of minority and majority classes:

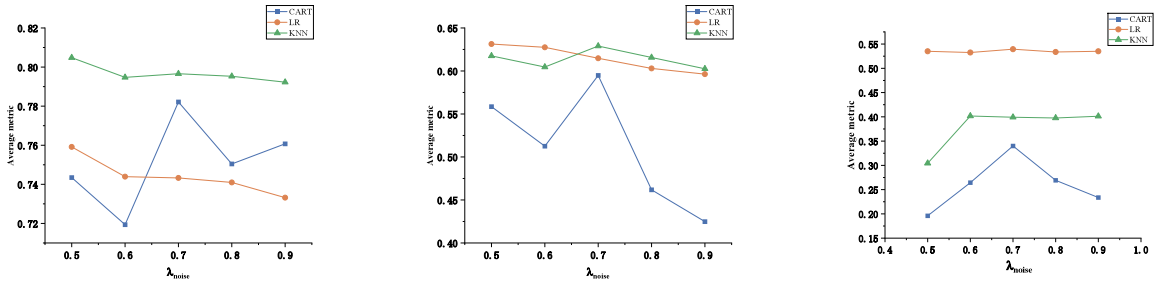
$$G\text{-}mean = \sqrt{Recall \cdot Specificity}. \tag{26}$$

F1 combines Precision and Recall to evaluate minority class recognition:

$$F1 = 2 \cdot Precision \cdot Recall/(Precision + Recall). \tag{27}$$

AUC is the area under the receiver operating characteristic (ROC) curve and measures the model's ranking ability between positive and negative samples, with false positive rate (FPR) and true positive rate (TPR) as the horizontal and vertical axes, respectively:

$$FPR = FP/(FP + TN), \qquad TPR = TP/(TP + FN). \tag{28}$$



(a) glass0 (b) yeast-1_vs_7 (c) abalone19

Fig. 5. Comparison of average metric values under different λ_{noise}.

Table 5
Classifier parameter settings.

Classifier	Parameters
CART	criterion = gini
LR	solver = liblinear, max_iter = 1000
KNN	neighbors = 5

4.1.3. Classifier settings

To evaluate RAN-SMOTE with different classifiers, the experiments use Classification and Regression Tree (CART), k-Nearest Neighbor (KNN), and Logistic Regression (LR). The specific parameters of the classifiers are shown in Table 5, and unspecified parameters are set to their default values.

4.1.4. Comparative algorithm settings

To evaluate the effectiveness of RAN-SMOTE, several representative oversampling methods are selected for comparison. The comparison methods are as follows:

(1) SMOTE (Chawla et al., 2002): A classical method that generates synthetic samples through nearest neighbor interpolation.

(2) Safe-Level-SMOTE (Bunkhumpornpat et al., 2009): It estimates sample safety using minority neighbors and generates samples in safer regions.

(3) ADASYN (He et al., 2008): It assigns more synthetic samples to minority samples with higher learning difficulty.

(4) Borderline-SMOTE (Han et al., 2005): It focuses on minority samples near decision boundaries to enhance boundary learning.

(5) MWMOTE (Barua et al., 2014): It identifies informative minority samples near decision boundaries and assigns weights based on their distances to majority samples for weighted sample generation.

(6) G-SMOTE (Douzas & Bacao, 2019): It expands linear interpolation to geometric regions and generates samples within hyperspheres around minority samples.

(7) ASN-SMOTE (Yi et al., 2022): It filters noisy minority samples using nearest neighbor information and adaptively selects neighbors with the help of nearest majority samples.

(8) VS-SMOTE (Qiu et al., 2025): It evaluates candidate subspaces and generates samples in high-value regions to improve synthetic sample quality.

(9) ISMOTE (Li et al., 2025b): It introduces random perturbation to expand the generation space and generate samples beyond the line segment between two minority samples.

The specific parameters used for each method are shown in Table 6. For RAN-SMOTE, the parameters defined in Section 3 are used. The influences of the noise threshold λ_{noise} and the concentration parameter γ are examined in the parameter sensitivity analysis. To ensure reliable evaluation, stratified five-fold cross-validation is used, with four folds for training and one fold for testing in each run. Oversampling is applied only to the training folds, and the final results are reported as the mean and standard deviation of the test metrics across five folds.

Table 6
Parameter settings of baseline oversampling methods.

Method	Parameters
SMOTE	$k = 5$
Safe-Level-SMOTE	$k = 5$
ADASYN	$k = 5$, $d_{th} = 0.9$
Borderline-SMOTE	$k = 5$
MWMOTE	$k_1 = 5$, $k_2 = 3$, $k_3 = 5$, $C_p = 3$, $C_f(th) = 5$, $CMAX = 2$
G-SMOTE	$k = 5$
ASN-SMOTE	$k = 7$
VS-SMOTE	$k = 5$, $k_{\min} = 3$, $k_{\max} = 5$
ISMOTE	$k = 5$

4.2. Experimental results and analysis

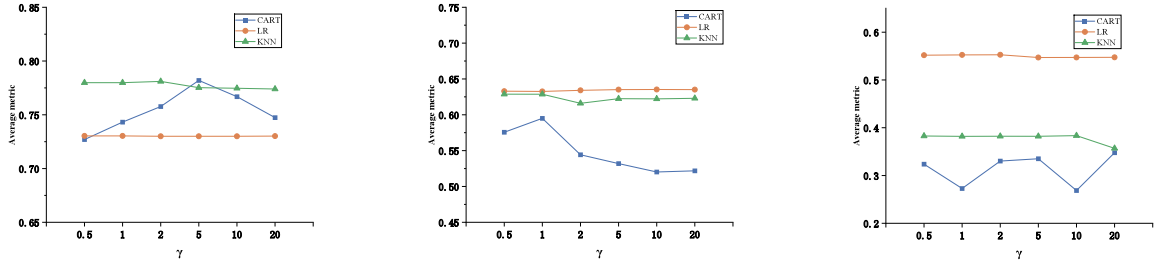
This section examines parameter sensitivity, compares classification performance, evaluates robustness to label noise and computational cost, and visualizes the distributions of synthetic samples.

4.2.1. Parameter sensitivity analysis

Sensitivity analyses are conducted for the noise threshold λ_{noise} and the Beta distribution concentration parameter γ. Three datasets are selected to represent different degrees of class imbalance: glass0 (IR = 2.06), yeast-1_vs_7 (IR = 14.3), and abalone19 (IR = 129.44). The experiments use CART, LR, and KNN, with performance summarized by the arithmetic mean of G-mean, AUC, and F1-score for each dataset and classifier. The two parameters are varied separately. When investigating λ_{noise}, its values are selected from $\{0.5, 0.6, 0.7, 0.8, 0.9\}$, with $\gamma = 5$. When investigating γ, its values are selected from $\{0.5, 1, 2, 5, 10, 20\}$, with $\lambda_{\text{noise}} = 0.7$. All other experimental settings remain unchanged.

Fig. 5(a) presents the results obtained on the low imbalance dataset glass0. When the CART classifier is used, $\lambda_{\text{noise}} = 0.7$ yields the highest performance. For LR and KNN, the performance associated with $\lambda_{\text{noise}} = 0.7$ remains comparable to that of the thresholds with stronger results. For the moderately imbalanced dataset yeast-1_vs_7, as shown in Fig. 5(b), the evaluated thresholds lead to similar performance under LR and KNN, whereas $\lambda_{\text{noise}} = 0.7$ achieves the highest performance under CART.

For the extremely imbalanced dataset abalone19, Fig. 5(c) indicates that $\lambda_{\text{noise}} = 0.5$ generally results in lower performance, with the exception of LR. This outcome may be attributed to the fact that, in highly imbalanced datasets, a large proportion of minority samples tend to exhibit relatively high misclassification rates. Consequently, a lower noise threshold may incorrectly identify normal minority samples as noise and remove them. In comparison, $\lambda_{\text{noise}} = 0.7$ achieves the highest result under CART and maintains relatively consistent performance under LR and KNN.



(a) glass0 (b) yeast-1_vs_7 (c) abalone19

Fig. 6. Comparison of average metric values under different γ.

Based on the above experimental results, $\lambda_{\text{noise}} = 0.7$ provides a good balance between noise filtering and minority sample preservation on datasets with different degrees of imbalance. Therefore, λ_{noise} is uniformly set to 0.7 in the subsequent experiments.

Next, we examine the sensitivity of RAN-SMOTE to the concentration parameter γ, as shown in Fig. 6. On glass0, Fig. 6(a) shows that LR and KNN remain relatively stable, whereas CART achieves its highest average performance at $\gamma = 5$. On yeast-1_vs_7, Fig. 6(b) shows only small variations for LR and KNN, while CART is more sensitive to γ. For the extremely imbalanced dataset abalone19, Fig. 6(c) indicates that LR remains relatively stable and KNN produces similar results except for a decrease at $\gamma = 20$, whereas CART exhibits larger fluctuations.

Overall, no single value of γ consistently achieves the best performance across all datasets and classifiers. Although $\gamma = 5$ yields the highest average performance across the nine dataset–classifier combinations, its advantage over $\gamma = 0.5$ and $\gamma = 2$ is small. These results support $\gamma = 5$ as a reasonable default with good overall performance. Therefore, $\gamma = 5$ is retained in the remaining experiments.

4.2.2. Comparative experimental results and analysis

This subsection compares the proposed method with SMOTE and its variants on 31 imbalanced datasets. For readability, Safe-Level-SMOTE and Borderline-SMOTE are abbreviated as SL-SMOTE and B-SMOTE. Tables 7–15 report the mean and standard deviation of each metric under different classifiers. The best result on each dataset is bolded, and the last row shows the average performance.

Specifically, Tables 7–9 report the CART results. RAN-SMOTE ranks first 26, 27, and 29 times for G-mean, AUC, and F1-score, respectively. It also achieves the highest averages on all three metrics, outperforming the second-best method by 2.42%, 2.28%, and 7.57%.

Tables 10–12 show the results under the LR classifier. RAN-SMOTE achieves the best performance 25, 19, and 16 times for G-mean, AUC, and F1-score, respectively. It also obtains the highest average G-mean and AUC, improving over the second-best method by 1.96% and 2.00%. For the F1-score, it is only 0.0015 lower than SMOTE while showing a smaller standard deviation, indicating more stable performance.

Tables 13–15 report the results under the KNN classifier. RAN-SMOTE achieves the best results 20, 21, and 19 times in terms of G-mean, AUC, and F1-score, respectively. Its average values are the best for all three metrics, improving upon the second-best method by 2.06%, 1.17%, and 2.90%, respectively.

RAN-SMOTE generally achieves the best overall performance across different classifiers and evaluation metrics. The results indicate that the proposed method can effectively improve the recognition ability of the minority class under different data scenarios. From the perspective of classifiers, RAN-SMOTE consistently outperforms the other methods under different classifiers, suggesting that its performance improvement does not depend on a specific classification model.

Next, the degree of imbalance is divided according to the value of IR: low imbalance ($IR \leq 5$), moderate imbalance ($5 < IR \leq 20$), high imbalance ($20 < IR \leq 100$), and extreme imbalance ($IR > 100$). Furthermore, Table 16 reports and analyzes the average metric values of the datasets under different imbalance levels.

On extremely imbalanced datasets, RAN-SMOTE significantly outperforms SMOTE across all metrics under the three classifiers.

On highly imbalanced datasets, RAN-SMOTE achieves substantial improvements over SMOTE on most metrics, except for the F1-score under the KNN and LR classifiers, where slight decreases are observed. This is because G-mean considers both the recall of the minority class and the specificity of the majority class, whereas F1-score depends only on the precision and recall of the minority class. RAN-SMOTE tends to generate more synthetic samples for samples with high misclassification rates, thereby improving the recognition performance of the minority class. However, this may also introduce more false positive samples, leading to a decrease in precision.

For the results under the KNN classifier, we find that RAN-SMOTE performs poorly on the poker-8-9_vs_6 dataset. A possible reason is that the algorithm preferentially assigns more synthetic samples to difficult-to-classify minority samples. Although this improves the recognition ability for the minority class, it may also cause a large number of majority samples to be misclassified as minority samples. As a result, the metric values on this dataset are relatively low, which affects the overall performance under highly imbalanced data scenarios.

For moderately and slightly imbalanced datasets, RAN-SMOTE also shows improvements over SMOTE in overall performance. Overall, as the degree of imbalance of the dataset increases, the improvement of RAN-SMOTE over SMOTE becomes more pronounced. Therefore, the proposed method has significant advantages on highly imbalanced datasets and also maintains a slight lead on datasets with a lower degree of imbalance. This indicates that the method can not only improve the classification performance on highly imbalanced datasets but is also applicable to relatively less imbalanced datasets, showing good general applicability.

In addition, when the CART classifier is used, RAN-SMOTE significantly outperforms SMOTE on highly imbalanced datasets. However, on some highly imbalanced datasets, such as yeast4, its G-mean is lower than that of ISMOTE, while its F1-score is higher than that of ISMOTE. This may be because these datasets have more severe boundary overlap. RAN-SMOTE assigns more synthetic samples to difficult-to-classify minority samples, which are often located in boundary regions. As a result, although the method can better identify minority samples, it may also misclassify majority samples, leading to a lower G-mean than ISMOTE.

4.2.3. Robustness to label noise

To evaluate the robustness of RAN-SMOTE to label noise, experiments are conducted on the five datasets listed in Table 3. The noise rates are set to $\{0\%, 5\%, 10\%, 15\%, 20\%\}$, where 0% denotes the setting without injected label noise. For each noise rate, the corresponding proportion of majority class samples in each training set is randomly selected and relabeled as the minority class. Label noise is introduced before oversampling, while the test labels remain unchanged.

Table 7
G-mean results of different oversampling methods under the CART classifier.

Dataset	SMOTE	SL-SMOTE	B-SMOTE	ADASYN	G-SMOTE	MWMOTE	ASN-SMOTE	ISMOTE	VS-SMOTE	RAN-SMOTE
D1	0.0704 ± 0.1644	0.1253 ± 0.1829	0.1021 ± 0.1725	0.2702 ± 0.2413	0.1323 ± 0.2084	0.1087 ± 0.1745	0.0788 ± 0.1577	0.3042 ± 0.2699	0.0487 ± 0.1320	**0.4133 ± 0.1763**
D2	0.9396 ± 0.0652	0.9396 ± 0.0652	0.9369 ± 0.0655	0.9369 ± 0.0655	0.9369 ± 0.0655	0.9396 ± 0.0652	0.9396 ± 0.0652	0.9164 ± 0.1214	0.9396 ± 0.0652	**0.9814 ± 0.0618**
D3	0.7834 ± 0.1470	0.7671 ± 0.1494	0.7499 ± 0.1317	0.7596 ± 0.1917	0.7777 ± 0.1497	0.7477 ± 0.1965	0.7698 ± 0.1404	0.7828 ± 0.1313	0.7737 ± 0.1709	**0.8188 ± 0.1016**
D4	0.7449 ± 0.2044	0.7228 ± 0.2013	0.7617 ± 0.2057	0.7713 ± 0.1289	0.7801 ± 0.1812	0.7222 ± 0.2079	0.7088 ± 0.2520	0.8040 ± 0.1452	0.7556 ± 0.1985	**0.8126 ± 0.1008**
D5	0.8093 ± 0.2164	0.7882 ± 0.1871	0.7461 ± 0.2245	0.8115 ± 0.1892	0.8207 ± 0.1676	0.7796 ± 0.2121	0.7844 ± 0.2172	0.8183 ± 0.1391	0.8231 ± 0.1955	**0.8587 ± 0.1272**
D6	0.9654 ± 0.0196	0.9645 ± 0.0364	0.9663 ± 0.0295	0.9563 ± 0.0271	0.9664 ± 0.0295	0.9596 ± 0.0388	0.9641 ± 0.0330	0.9665 ± 0.0330	0.9632 ± 0.0350	**0.9828 ± 0.0213**
D7	0.8338 ± 0.0705	0.8396 ± 0.0484	0.8179 ± 0.0629	0.8334 ± 0.0467	0.8261 ± 0.0747	0.8270 ± 0.0667	0.8363 ± 0.0631	0.8211 ± 0.0694	0.8273 ± 0.0857	**0.8633 ± 0.0586**
D8	0.6784 ± 0.1243	0.7237 ± 0.1272	0.6365 ± 0.1803	0.6931 ± 0.1644	0.6757 ± 0.1938	0.7035 ± 0.1207	0.6602 ± 0.1222	0.7465 ± 0.0947	0.6841 ± 0.1255	**0.7692 ± 0.1042**
D9	0.8925 ± 0.0602	0.8877 ± 0.0562	0.8783 ± 0.0736	0.8728 ± 0.0641	0.8919 ± 0.0638	0.8834 ± 0.0824	0.8905 ± 0.0602	0.8943 ± 0.0523	0.8923 ± 0.0514	**0.9028 ± 0.0602**
D10	0.9913 ± 0.0140	0.9828 ± 0.0213	0.9913 ± 0.0139	0.9913 ± 0.0139	0.9913 ± 0.0139	0.9913 ± 0.0140	0.9516 ± 0.0855	0.9525 ± 0.1949	0.9913 ± 0.0140	**0.9940 ± 0.0121**
D11	0.7728 ± 0.0676	0.7653 ± 0.0682	0.7584 ± 0.0598	0.7566 ± 0.0683	0.7736 ± 0.0667	0.7733 ± 0.0719	0.7723 ± 0.0838	0.7640 ± 0.0670	0.7691 ± 0.0774	**0.8015 ± 0.0732**
D12	0.7111 ± 0.0731	0.7007 ± 0.0722	0.7155 ± 0.0778	0.7181 ± 0.0600	0.7197 ± 0.0685	0.7103 ± 0.0801	0.7051 ± 0.0766	0.7015 ± 0.0786	0.7191 ± 0.0783	**0.7359 ± 0.0786**
D13	0.7409 ± 0.2683	0.7918 ± 0.2186	0.7491 ± 0.2921	0.7245 ± 0.2772	0.7739 ± 0.2912	0.7053 ± 0.2417	0.7685 ± 0.2718	0.8211 ± 0.2800	0.7911 ± 0.2721	**0.8512 ± 0.2095**
D14	0.5175 ± 0.0746	0.5354 ± 0.0885	0.5368 ± 0.0924	0.5172 ± 0.1093	0.5198 ± 0.1030	0.5035 ± 0.0722	0.5232 ± 0.0979	0.5233 ± 0.0736	0.5324 ± 0.1024	**0.5475 ± 0.0848**
D15	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9990 ± 0.0050	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D16	0.8744 ± 0.1093	0.8667 ± 0.0960	0.8782 ± 0.0888	0.8768 ± 0.0839	0.8781 ± 0.0883	0.8629 ± 0.0986	0.8820 ± 0.0728	0.8799 ± 0.0977	0.8825 ± 0.0765	**0.8945 ± 0.0667**
D17	0.9170 ± 0.0683	0.9238 ± 0.0681	0.9384 ± 0.0845	0.9470 ± 0.0744	0.9485 ± 0.0757	0.9355 ± 0.0777	0.9268 ± 0.0728	0.9474 ± 0.0559	0.9473 ± 0.0465	**0.9525 ± 0.0587**
D18	0.6583 ± 0.0359	0.6505 ± 0.0450	0.6611 ± 0.0401	0.6519 ± 0.0367	0.6592 ± 0.0370	0.6507 ± 0.0554	0.6622 ± 0.0453	0.6624 ± 0.0468	0.6720 ± 0.0437	**0.6826 ± 0.0395**
D19	0.3558 ± 0.2661	0.3747 ± 0.2322	0.4366 ± 0.2802	0.6540 ± 0.3241	0.6630 ± 0.3499	0.4308 ± 0.3411	0.3778 ± 0.3066	**0.7323 ± 0.2626**	0.5634 ± 0.2945	0.6589 ± 0.3533
D20	0.9856 ± 0.0091	0.9860 ± 0.0115	0.9836 ± 0.0133	0.9857 ± 0.0119	0.9875 ± 0.0085	0.9878 ± 0.0108	0.9870 ± 0.0092	0.9901 ± 0.0065	0.9859 ± 0.0113	**0.9920 ± 0.0056**
D21	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9999 ± 0.0004	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D22	0.8974 ± 0.0262	0.9011 ± 0.0351	0.9066 ± 0.0297	0.9024 ± 0.0254	0.8993 ± 0.0281	0.8952 ± 0.0277	0.9031 ± 0.0324	**0.9203 ± 0.0235**	0.8991 ± 0.0291	0.9201 ± 0.0241
D23	0.9297 ± 0.0205	0.9315 ± 0.0239	0.9260 ± 0.0280	0.9312 ± 0.0221	0.9295 ± 0.0303	0.9291 ± 0.0176	0.9286 ± 0.0200	0.9374 ± 0.0253	0.9339 ± 0.0332	**0.9479 ± 0.0241**
D24	0.6226 ± 0.1591	0.6758 ± 0.1184	0.6586 ± 0.0859	0.6637 ± 0.0978	0.6477 ± 0.0925	0.6226 ± 0.1197	0.6094 ± 0.1415	0.6788 ± 0.1076	0.6394 ± 0.1076	**0.7035 ± 0.1296**
D25	0.4285 ± 0.2443	0.4464 ± 0.1987	0.3577 ± 0.2549	0.4012 ± 0.2373	0.3470 ± 0.2360	0.3384 ± 0.2259	0.4329 ± 0.1925	**0.5009 ± 0.2264**	0.3568 ± 0.2415	0.4011 ± 0.2232
D26	0.2434 ± 0.2443	0.3706 ± 0.2380	0.2463 ± 0.2450	0.3534 ± 0.2054	0.2655 ± 0.2370	0.3070 ± 0.2503	0.1336 ± 0.1972	0.3573 ± 0.2209	0.3105 ± 0.2694	**0.4503 ± 0.2064**
D27	0.5150 ± 0.1875	0.5631 ± 0.1668	0.5219 ± 0.1832	0.5092 ± 0.2226	0.4966 ± 0.1899	0.4505 ± 0.1894	0.5441 ± 0.1879	0.5732 ± 0.1670	0.5920 ± 0.1839	**0.6030 ± 0.1718**
D28	0.8086 ± 0.0481	0.8349 ± 0.0409	0.8347 ± 0.0527	0.8451 ± 0.0429	0.8388 ± 0.0489	0.8492 ± 0.0426	0.8264 ± 0.0507	0.8487 ± 0.0396	0.8370 ± 0.0438	**0.8930 ± 0.0455**
D29	0.5496 ± 0.1190	0.6216 ± 0.1422	0.5896 ± 0.1257	0.5877 ± 0.1441	0.5795 ± 0.1246	0.5601 ± 0.1025	0.5305 ± 0.1228	**0.6844 ± 0.1150**	0.5756 ± 0.1363	0.6606 ± 0.1038
D30	0.7800 ± 0.1329	0.8312 ± 0.0830	0.7995 ± 0.1147	0.8542 ± 0.0818	0.8186 ± 0.0963	0.8076 ± 0.0790	0.7575 ± 0.1278	0.8495 ± 0.1211	0.8126 ± 0.0931	**0.9139 ± 0.0757**
D31	0.6461 ± 0.1586	0.7077 ± 0.1188	0.6885 ± 0.1306	0.7060 ± 0.1197	0.6937 ± 0.1346	0.6822 ± 0.1240	0.6816 ± 0.1833	**0.7974 ± 0.0947**	0.6814 ± 0.1588	0.7550 ± 0.1594
Average	0.7311 ± 0.1096	0.7490 ± 0.1013	0.7346 ± 0.1110	0.7575 ± 0.1090	0.7496 ± 0.1115	0.7311 ± 0.1099	0.7270 ± 0.1127	0.7799 ± 0.1084	0.7484 ± 0.1088	**0.7988 ± 0.0954**

Table 8
AUC results of different oversampling methods under the CART classifier.

Dataset	SMOTE	SL-SMOTE	B-SMOTE	ADASYN	G-SMOTE	MWMOTE	ASN-SMOTE	ISMOTE	VS-SMOTE	RAN-SMOTE
D1	0.5099 ± 0.0402	0.5126 ± 0.0364	0.5156 ± 0.0349	0.5484 ± 0.0698	0.5206 ± 0.0542	0.5151 ± 0.0341	0.5095 ± 0.0318	0.5560 ± 0.0939	0.5052 ± 0.0270	**0.5899 ± 0.0596**
D2	0.9435 ± 0.0608	0.9435 ± 0.0608	0.9410 ± 0.0610	0.9410 ± 0.0610	0.9410 ± 0.0610	0.9435 ± 0.0608	0.9435 ± 0.0608	0.9267 ± 0.0983	0.9435 ± 0.0608	**0.9835 ± 0.0535**
D3	0.8121 ± 0.1170	0.7999 ± 0.1183	0.7822 ± 0.1022	0.7951 ± 0.1356	0.8055 ± 0.1185	0.7897 ± 0.1160	0.7963 ± 0.1077	0.8034 ± 0.1064	0.8063 ± 0.1328	**0.8372 ± 0.0771**
D4	0.7935 ± 0.1252	0.7765 ± 0.1240	0.8043 ± 0.1266	0.7955 ± 0.0996	0.8140 ± 0.1211	0.7755 ± 0.1260	0.7740 ± 0.1350	0.8265 ± 0.1167	0.7990 ± 0.1187	**0.8320 ± 0.0783**
D5	0.8485 ± 0.1313	0.8331 ± 0.1137	0.7980 ± 0.1243	0.8410 ± 0.1284	0.8460 ± 0.1106	0.8230 ± 0.1296	0.8265 ± 0.1309	0.8370 ± 0.1129	0.8535 ± 0.1157	**0.8755 ± 0.1041**
D6	0.9660 ± 0.0192	0.9649 ± 0.0358	0.9667 ± 0.0291	0.9572 ± 0.0262	0.9668 ± 0.0291	0.9603 ± 0.0378	0.9647 ± 0.0325	0.9669 ± 0.0328	0.9637 ± 0.0344	**0.9832 ± 0.0207**
D7	0.8411 ± 0.0574	0.8489 ± 0.0468	0.8242 ± 0.0575	0.8372 ± 0.0433	0.8324 ± 0.0670	0.8325 ± 0.0603	0.8429 ± 0.0577	0.8269 ± 0.0636	0.8352 ± 0.0712	**0.8661 ± 0.0566**
D8	0.7232 ± 0.0928	0.7568 ± 0.0985	0.7029 ± 0.1110	0.7386 ± 0.1137	0.7308 ± 0.1127	0.7402 ± 0.0933	0.7090 ± 0.0862	0.7664 ± 0.0779	0.7264 ± 0.0881	**0.7903 ± 0.0820**
D9	0.8971 ± 0.0546	0.8958 ± 0.0532	0.8839 ± 0.0658	0.8775 ± 0.0593	0.8967 ± 0.0572	0.8883 ± 0.0752	0.8945 ± 0.0565	0.8984 ± 0.0477	0.8957 ± 0.0491	**0.9064 ± 0.0566**
D10	0.9914 ± 0.0138	0.9832 ± 0.0208	0.9915 ± 0.0137	0.9915 ± 0.0137	0.9915 ± 0.0137	0.9914 ± 0.0138	0.9547 ± 0.0778	0.9726 ± 0.0974	0.9914 ± 0.0138	**0.9940 ± 0.0119**
D11	0.7780 ± 0.0671	0.7710 ± 0.0652	0.7656 ± 0.0539	0.7620 ± 0.0659	0.7791 ± 0.0625	0.7793 ± 0.0664	0.7796 ± 0.0753	0.7703 ± 0.0622	0.7767 ± 0.0721	**0.8039 ± 0.0726**
D12	0.7185 ± 0.0692	0.7120 ± 0.0648	0.7205 ± 0.0746	0.7248 ± 0.0568	0.7270 ± 0.0651	0.7196 ± 0.0743	0.7131 ± 0.0738	0.7109 ± 0.0716	0.7260 ± 0.0729	**0.7447 ± 0.0722**
D13	0.8050 ± 0.1600	0.8344 ± 0.1391	0.8191 ± 0.1687	0.7951 ± 0.1526	0.8381 ± 0.1622	0.7733 ± 0.1277	0.8255 ± 0.1605	0.8723 ± 0.1631	0.8477 ± 0.1548	**0.8835 ± 0.1284**
D14	0.5551 ± 0.0628	0.5552 ± 0.0801	0.5714 ± 0.0685	0.5544 ± 0.0703	0.5593 ± 0.0706	0.5396 ± 0.0567	0.5626 ± 0.0728	0.5461 ± 0.0587	0.5647 ± 0.0782	**0.5762 ± 0.0675**
D15	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9990 ± 0.0049	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D16	0.8836 ± 0.0991	0.8829 ± 0.0867	0.8841 ± 0.0812	0.8827 ± 0.0783	0.8840 ± 0.0812	0.8815 ± 0.0878	0.8878 ± 0.0702	0.8828 ± 0.0862	0.8871 ± 0.0728	**0.9008 ± 0.0612**
D17	0.9214 ± 0.0627	0.9302 ± 0.0598	0.9432 ± 0.0742	0.9508 ± 0.0665	0.9521 ± 0.0684	0.9402 ± 0.0676	0.9308 ± 0.0634	0.9490 ± 0.0534	0.9494 ± 0.0439	**0.9550 ± 0.0535**
D18	0.6650 ± 0.0334	0.6605 ± 0.0406	0.6661 ± 0.0364	0.6569 ± 0.0339	0.6649 ± 0.0341	0.6563 ± 0.0508	0.6688 ± 0.0417	0.6669 ± 0.0441	0.6778 ± 0.0398	**0.6869 ± 0.0378**
D19	0.5927 ± 0.0947	0.5880 ± 0.0854	0.6295 ± 0.1194	0.7631 ± 0.1672	0.7779 ± 0.1940	0.6467 ± 0.1597	0.6123 ± 0.1178	**0.7962 ± 0.1775**	0.6986 ± 0.1438	0.7782 ± 0.1952
D20	0.9857 ± 0.0090	0.9864 ± 0.0113	0.9837 ± 0.0130	0.9859 ± 0.0117	0.9876 ± 0.0084	0.9879 ± 0.0107	0.9871 ± 0.0091	0.9901 ± 0.0065	0.9860 ± 0.0111	**0.9920 ± 0.0055**
D21	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9999 ± 0.0004	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D22	0.8998 ± 0.0245	0.9028 ± 0.0320	0.9083 ± 0.0282	0.9040 ± 0.0241	0.9013 ± 0.0263	0.8975 ± 0.0256	0.9044 ± 0.0313	**0.9209 ± 0.0230**	0.9011 ± 0.0276	**0.9209 ± 0.0232**
D23	0.9305 ± 0.0197	0.9323 ± 0.0230	0.9269 ± 0.0272	0.9320 ± 0.0213	0.9304 ± 0.0298	0.9300 ± 0.0170	0.9294 ± 0.0195	0.9378 ± 0.0251	0.9346 ± 0.0327	**0.9485 ± 0.0232**
D24	0.6852 ± 0.0798	0.7156 ± 0.0998	0.6974 ± 0.0638	0.7010 ± 0.0673	0.6895 ± 0.0699	0.6750 ± 0.0792	0.6703 ± 0.0933	0.7118 ± 0.0796	0.6891 ± 0.0714	**0.7397 ± 0.0944**
D25	0.6082 ± 0.0899	0.5913 ± 0.0860	0.5789 ± 0.0857	0.5828 ± 0.0865	0.5663 ± 0.0785	0.5631 ± 0.0703	0.5971 ± 0.0754	**0.6176 ± 0.1074**	0.5762 ± 0.0817	0.5881 ± 0.0786
D26	0.5356 ± 0.0698	0.5494 ± 0.0960	0.5317 ± 0.0698	0.5469 ± 0.0694	0.5332 ± 0.0692	0.5527 ± 0.0870	0.5019 ± 0.0476	0.5425 ± 0.0776	0.5614 ± 0.0897	**0.5961 ± 0.0849**
D27	0.6321 ± 0.0767	0.6405 ± 0.0950	0.6280 ± 0.0997	0.6224 ± 0.1010	0.6119 ± 0.0862	0.5925 ± 0.0665	0.6461 ± 0.0823	0.6430 ± 0.0875	0.6756 ± 0.1013	**0.6790 ± 0.0933**
D28	0.8216 ± 0.0391	0.8513 ± 0.0391	0.8441 ± 0.0458	0.8522 ± 0.0372	0.8473 ± 0.0427	0.8558 ± 0.0374	0.8372 ± 0.0427	0.8548 ± 0.0351	0.8459 ± 0.0381	**0.8962 ± 0.0419**
D29	0.6468 ± 0.0694	0.6908 ± 0.0898	0.6704 ± 0.0779	0.6679 ± 0.0736	0.6624 ± 0.0739	0.6501 ± 0.0550	0.6364 ± 0.0634	**0.7265 ± 0.0834**	0.6639 ± 0.0833	0.7129 ± 0.0672
D30	0.8113 ± 0.1017	0.8464 ± 0.0691	0.8243 ± 0.0879	0.8667 ± 0.0701	0.8379 ± 0.0792	0.8276 ± 0.0636	0.7919 ± 0.0965	0.8647 ± 0.0864	0.8327 ± 0.0752	**0.9193 ± 0.0665**
D31	0.7163 ± 0.1018	0.7525 ± 0.0870	0.7404 ± 0.0890	0.7487 ± 0.0815	0.7434 ± 0.0893	0.7354 ± 0.0822	0.7443 ± 0.0967	**0.8153 ± 0.0769**	0.7390 ± 0.1034	0.7949 ± 0.1099
Average	0.7909 ± 0.0659	0.7971 ± 0.0664	0.7917 ± 0.0675	0.8008 ± 0.0673	0.8013 ± 0.0689	0.7891 ± 0.0656	0.7884 ± 0.0682	0.8129 ± 0.0727	0.8017 ± 0.0679	**0.8314 ± 0.0638**

Table 9
F1-score results of different oversampling methods under the CART classifier.

Dataset	SMOTE	SL-SMOTE	B-SMOTE	ADASYN	G-SMOTE	MWMOTE	ASN-SMOTE	ISMOTE	VS-SMOTE	RAN-SMOTE
D1	0.0225 ± 0.0551	0.0227 ± 0.0340	0.0392 ± 0.0668	0.0435 ± 0.0451	0.0303 ± 0.0516	0.0295 ± 0.0479	0.0248 ± 0.0520	0.0336 ± 0.0357	0.0164 ± 0.0458	**0.0898 ± 0.0596**
D2	0.9149 ± 0.0649	0.9149 ± 0.0649	0.9121 ± 0.0642	0.9121 ± 0.0642	0.9121 ± 0.0642	0.9149 ± 0.0649	0.9149 ± 0.0649	0.8073 ± 0.1442	0.9149 ± 0.0649	**0.9587 ± 0.0776**
D3	0.6616 ± 0.1782	0.5687 ± 0.1503	0.6006 ± 0.1587	0.5676 ± 0.2033	0.6222 ± 0.1880	0.5887 ± 0.1990	0.5975 ± 0.1717	0.5778 ± 0.1530	0.6291 ± 0.2121	**0.7254 ± 0.1323**
D4	0.6404 ± 0.1988	0.5458 ± 0.2150	0.6179 ± 0.2120	0.5788 ± 0.1473	0.6268 ± 0.1908	0.5991 ± 0.2225	0.5651 ± 0.2446	0.6363 ± 0.1562	0.6298 ± 0.1943	**0.7300 ± 0.1417**
D5	0.7153 ± 0.2239	0.5861 ± 0.1632	0.6375 ± 0.2330	0.6234 ± 0.1815	0.6756 ± 0.1818	0.6833 ± 0.2272	0.6488 ± 0.2214	0.6426 ± 0.1748	0.7083 ± 0.2072	**0.8115 ± 0.1632**
D6	0.9509 ± 0.0285	0.9503 ± 0.0464	0.9533 ± 0.0379	0.9381 ± 0.0341	0.9551 ± 0.0379	0.9461 ± 0.0482	0.9489 ± 0.0435	0.9522 ± 0.0440	0.9489 ± 0.0438	**0.9824 ± 0.0218**
D7	0.7459 ± 0.0877	0.7304 ± 0.0670	0.7194 ± 0.0817	0.7325 ± 0.0643	0.7268 ± 0.0939	0.7265 ± 0.0911	0.7540 ± 0.0789	0.7211 ± 0.0922	0.7322 ± 0.1064	**0.7822 ± 0.0899**
D8	0.5051 ± 0.1561	0.5000 ± 0.1319	0.4581 ± 0.1885	0.4945 ± 0.1773	0.4900 ± 0.1873	0.5218 ± 0.1482	0.4610 ± 0.1203	0.5326 ± 0.1283	0.4924 ± 0.1408	**0.5858 ± 0.1319**
D9	0.8408 ± 0.0676	0.8234 ± 0.0630	0.8208 ± 0.0926	0.8081 ± 0.0819	0.8392 ± 0.0758	0.8247 ± 0.1112	0.8247 ± 0.0746	0.8463 ± 0.0626	0.8394 ± 0.0666	**0.8467 ± 0.0739**
D10	0.9387 ± 0.1014	0.8827 ± 0.1532	0.9360 ± 0.1066	0.9360 ± 0.1066	0.9360 ± 0.1066	0.9387 ± 0.1014	0.8187 ± 0.1734	0.9120 ± 0.2046	0.9387 ± 0.1014	**0.9547 ± 0.0938**
D11	0.7019 ± 0.0832	0.6887 ± 0.0836	0.6826 ± 0.0732	0.6767 ± 0.0828	0.6995 ± 0.0812	0.7005 ± 0.0864	0.6977 ± 0.1029	0.6883 ± 0.0817	0.6957 ± 0.0945	**0.7316 ± 0.0914**
D12	0.6379 ± 0.0891	0.6229 ± 0.0893	0.6434 ± 0.0933	0.6456 ± 0.0738	0.6484 ± 0.0841	0.6392 ± 0.0955	0.6333 ± 0.0920	0.6258 ± 0.0963	0.6446 ± 0.0990	**0.6666 ± 0.0939**
D13	0.5988 ± 0.2464	0.6259 ± 0.2173	0.6590 ± 0.2913	0.5723 ± 0.2617	0.6334 ± 0.2761	0.5489 ± 0.2207	0.5347 ± 0.2398	0.6110 ± 0.2468	0.6837 ± 0.2770	**0.7714 ± 0.2154**
D14	0.3520 ± 0.0828	0.3713 ± 0.0951	0.3738 ± 0.1014	0.3566 ± 0.1027	0.3558 ± 0.1108	0.3352 ± 0.0807	0.3589 ± 0.1055	0.3573 ± 0.0812	0.3695 ± 0.1137	**0.3841 ± 0.0956**
D15	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9981 ± 0.0093	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D16	0.7361 ± 0.1501	0.6614 ± 0.1163	0.7536 ± 0.1318	0.7501 ± 0.1224	0.7540 ± 0.1340	0.6558 ± 0.1369	0.7205 ± 0.0977	0.7552 ± 0.1305	0.7447 ± 0.1051	**0.7834 ± 0.0745**
D17	0.8752 ± 0.0837	0.8609 ± 0.0857	0.8941 ± 0.1010	0.9198 ± 0.0885	0.9264 ± 0.0936	0.9109 ± 0.0941	0.8614 ± 0.0921	0.8880 ± 0.0902	0.9178 ± 0.0643	**0.9312 ± 0.0707**
D18	0.5663 ± 0.0446	0.5594 ± 0.0542	0.5703 ± 0.0490	0.5601 ± 0.0444	0.5684 ± 0.0445	0.5585 ± 0.0670	0.5713 ± 0.0552	0.5729 ± 0.0568	0.5832 ± 0.0525	**0.5962 ± 0.0486**
D19	0.1940 ± 0.1801	0.1412 ± 0.1090	0.2690 ± 0.2177	0.5234 ± 0.3249	0.5646 ± 0.3814	0.3138 ± 0.3193	0.2211 ± 0.2090	0.4821 ± 0.3429	0.4071 ± 0.2788	**0.5859 ± 0.3785**
D20	0.9704 ± 0.0119	0.9714 ± 0.0172	0.9721 ± 0.0179	0.9767 ± 0.0159	0.9751 ± 0.0127	0.9772 ± 0.0120	0.9700 ± 0.0144	0.9729 ± 0.0148	0.9737 ± 0.0171	**0.9826 ± 0.0116**
D21	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9984 ± 0.0055	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D22	0.8514 ± 0.0349	0.8363 ± 0.0430	0.8560 ± 0.0366	0.8495 ± 0.0330	0.8470 ± 0.0384	0.8445 ± 0.0328	0.8440 ± 0.0457	0.8621 ± 0.0303	0.8468 ± 0.0388	**0.8676 ± 0.0302**
D23	0.9118 ± 0.0241	0.9142 ± 0.0274	0.9074 ± 0.0333	0.9142 ± 0.0255	0.9126 ± 0.0369	0.9119 ± 0.0227	0.9115 ± 0.0247	0.9185 ± 0.0331	0.9173 ± 0.0422	**0.9313 ± 0.0281**
D24	0.4066 ± 0.1393	0.3945 ± 0.1127	0.4174 ± 0.0956	0.4108 ± 0.0955	0.4063 ± 0.1038	0.3836 ± 0.1182	0.3739 ± 0.1365	0.4174 ± 0.1066	0.4178 ± 0.1176	**0.4853 ± 0.1384**
D25	**0.1996 ± 0.1449**	0.1419 ± 0.0858	0.1450 ± 0.1171	0.1413 ± 0.1088	0.1273 ± 0.1051	0.1351 ± 0.1140	0.1896 ± 0.1281	0.1591 ± 0.1070	0.1566 ± 0.1369	0.1739 ± 0.1168
D26	0.0984 ± 0.1109	0.1023 ± 0.0782	0.0938 ± 0.0993	0.1187 ± 0.0864	0.0962 ± 0.0962	0.1297 ± 0.1310	0.0509 ± 0.0801	0.1057 ± 0.0769	0.1376 ± 0.1396	**0.1756 ± 0.1052**
D27	0.2875 ± 0.1305	0.2311 ± 0.0931	0.2737 ± 0.1530	0.2489 ± 0.1449	0.2395 ± 0.1265	0.2160 ± 0.1049	0.3123 ± 0.1370	0.2610 ± 0.1164	**0.3642 ± 0.1676**	0.3607 ± 0.1405
D28	0.6741 ± 0.0540	0.6410 ± 0.0465	0.7023 ± 0.0641	0.7058 ± 0.0594	0.7016 ± 0.0546	0.7045 ± 0.0537	0.6970 ± 0.0504	0.7004 ± 0.0566	0.7066 ± 0.0438	**0.7707 ± 0.0665**
D29	0.2903 ± 0.1169	0.2676 ± 0.0980	0.3073 ± 0.1075	0.2738 ± 0.0966	0.2851 ± 0.0988	0.2756 ± 0.0694	0.2704 ± 0.1087	0.3221 ± 0.0913	0.3116 ± 0.1329	**0.3545 ± 0.0870**
D30	0.6301 ± 0.1434	0.6299 ± 0.1102	0.6445 ± 0.1392	0.6962 ± 0.1024	0.6587 ± 0.1166	0.6632 ± 0.0976	0.5480 ± 0.1237	0.5835 ± 0.1450	0.6550 ± 0.1125	**0.7216 ± 0.1064**
D31	0.3950 ± 0.1438	0.3792 ± 0.1124	0.4143 ± 0.1238	0.3610 ± 0.0974	0.3829 ± 0.1101	0.4188 ± 0.1254	0.4349 ± 0.1662	0.3938 ± 0.0952	0.3891 ± 0.1255	**0.5285 ± 0.1617**
Average	0.6230 ± 0.1025	0.5989 ± 0.0892	0.6218 ± 0.1061	0.6237 ± 0.0991	0.6322 ± 0.1059	0.6160 ± 0.1046	0.6051 ± 0.1053	0.6238 ± 0.1032	0.6378 ± 0.1078	**0.6861 ± 0.0983**

Table 10
G-mean results of different oversampling methods under the LR classifier.

Dataset	SMOTE	SL-SMOTE	B-SMOTE	ADASYN	G-SMOTE	MWMOTE	ASN-SMOTE	ISMOTE	VS-SMOTE	RAN-SMOTE
D1	0.0000 ± 0.0000	0.7449 ± 0.0973	0.3643 ± 0.2962	0.7193 ± 0.1184	0.6999 ± 0.1226	0.5720 ± 0.2579	0.6672 ± 0.2598	0.7259 ± 0.1284	0.0309 ± 0.1050	**0.7451 ± 0.1047**
D2	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9997 ± 0.0015	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D3	0.8187 ± 0.1489	0.8038 ± 0.1455	0.8229 ± 0.1375	0.8253 ± 0.1174	0.8274 ± 0.1184	0.8060 ± 0.0844	0.8257 ± 0.1504	0.8153 ± 0.1485	0.8251 ± 0.1537	**0.8539 ± 0.1170**
D4	0.8239 ± 0.1267	0.8029 ± 0.1261	0.8360 ± 0.1602	0.8263 ± 0.1230	0.8327 ± 0.1327	0.7972 ± 0.1267	0.8262 ± 0.1214	0.8306 ± 0.1218	0.8399 ± 0.1227	**0.8625 ± 0.0897**
D5	**0.8714 ± 0.1114**	0.8590 ± 0.0924	0.8638 ± 0.1243	0.8468 ± 0.1008	0.8604 ± 0.0922	0.8562 ± 0.0928	0.8708 ± 0.1080	0.8671 ± 0.1043	**0.8716 ± 0.1073**	0.8679 ± 0.1098
D6	0.9757 ± 0.0301	0.9746 ± 0.0406	**0.9795 ± 0.0242**	0.9620 ± 0.0254	0.9759 ± 0.0278	0.9673 ± 0.0261	0.9701 ± 0.0374	0.9707 ± 0.0362	0.9722 ± 0.0341	0.9707 ± 0.0307
D7	0.8476 ± 0.0506	0.8556 ± 0.0573	0.8666 ± 0.0513	0.8751 ± 0.0433	0.8586 ± 0.0453	0.8611 ± 0.0552	0.8564 ± 0.0459	0.8465 ± 0.0605	0.8525 ± 0.0491	**0.9012 ± 0.0357**
D8	0.8250 ± 0.0906	0.8706 ± 0.0608	0.8812 ± 0.0575	0.8694 ± 0.0856	0.8758 ± 0.0852	0.8865 ± 0.0546	0.8737 ± 0.0749	0.8865 ± 0.0612	0.8770 ± 0.0655	**0.8879 ± 0.0473**
D9	0.8983 ± 0.0629	0.9008 ± 0.0477	0.9054 ± 0.0505	0.9060 ± 0.0488	0.9005 ± 0.0686	0.8945 ± 0.0490	0.9037 ± 0.0511	0.9027 ± 0.0549	0.8945 ± 0.0596	**0.9168 ± 0.0588**
D10	0.8372 ± 0.3231	**0.9324 ± 0.0901**	0.8961 ± 0.2086	0.8962 ± 0.2086	0.8705 ± 0.2727	0.8943 ± 0.2049	0.9033 ± 0.1957	0.8966 ± 0.2653	0.8907 ± 0.2079	0.9090 ± 0.1021
D11	0.7251 ± 0.0774	0.7176 ± 0.0777	0.7129 ± 0.0659	0.7236 ± 0.0770	0.7199 ± 0.0718	0.7189 ± 0.0748	0.7138 ± 0.0711	0.7249 ± 0.0771	0.7168 ± 0.0749	**0.7393 ± 0.0572**
D12	0.5966 ± 0.0769	0.6057 ± 0.0853	0.5836 ± 0.0652	0.5934 ± 0.0657	0.6107 ± 0.0568	0.6122 ± 0.0680	0.6035 ± 0.0851	0.5980 ± 0.0908	0.6078 ± 0.0718	**0.6368 ± 0.0675**
D13	0.5596 ± 0.3397	0.8555 ± 0.1387	0.7981 ± 0.2131	0.8469 ± 0.1892	0.8059 ± 0.2422	0.8547 ± 0.1546	0.8326 ± 0.1623	0.8670 ± 0.1498	0.8488 ± 0.1586	**0.9067 ± 0.1178**
D14	0.5471 ± 0.0948	0.6062 ± 0.0746	0.6111 ± 0.0744	0.6195 ± 0.0790	0.6169 ± 0.0901	0.6021 ± 0.0691	0.5874 ± 0.0853	0.6162 ± 0.0811	0.5890 ± 0.0882	**0.6441 ± 0.0602**
D15	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D16	0.8853 ± 0.1024	0.8665 ± 0.0589	0.8660 ± 0.0523	0.8629 ± 0.0658	0.8790 ± 0.0642	0.8587 ± 0.0554	0.8701 ± 0.0900	0.8908 ± 0.0642	0.8582 ± 0.0964	**0.9074 ± 0.0621**
D17	0.9678 ± 0.0463	0.9824 ± 0.0364	0.9755 ± 0.0441	0.9819 ± 0.0248	0.9780 ± 0.0341	0.9869 ± 0.0238	0.9809 ± 0.0269	0.9850 ± 0.0266	0.9826 ± 0.0273	**0.9927 ± 0.0090**
D18	0.7441 ± 0.0317	0.7434 ± 0.0276	0.7376 ± 0.0384	0.7375 ± 0.0255	0.7442 ± 0.0260	0.7321 ± 0.0282	0.7386 ± 0.0289	0.7412 ± 0.0365	0.7357 ± 0.0360	**0.7493 ± 0.0350**
D19	0.0000 ± 0.0000	0.3162 ± 0.2098	0.3106 ± 0.2232	0.3736 ± 0.1601	0.4011 ± 0.1446	0.3668 ± 0.1474	0.3287 ± 0.2006	0.4019 ± 0.1936	0.3184 ± 0.1709	**0.4490 ± 0.1673**
D20	0.9917 ± 0.0047	0.9928 ± 0.0053	0.9926 ± 0.0053	0.9937 ± 0.0054	0.9927 ± 0.0066	0.9924 ± 0.0070	0.9926 ± 0.0064	0.9923 ± 0.0063	0.9923 ± 0.0071	**0.9946 ± 0.0048**
D21	0.9958 ± 0.0083	0.9959 ± 0.0082	0.9958 ± 0.0083	0.9946 ± 0.0101	0.9963 ± 0.0079	0.9975 ± 0.0068	0.9959 ± 0.0083	0.9949 ± 0.0108	0.9959 ± 0.0082	**1.0000 ± 0.0000**
D22	0.9640 ± 0.0152	0.9639 ± 0.0146	0.9633 ± 0.0165	0.9654 ± 0.0142	0.9667 ± 0.0152	0.9647 ± 0.0189	0.9646 ± 0.0138	0.9659 ± 0.0181	**0.9671 ± 0.0155**	0.9660 ± 0.0214
D23	0.9644 ± 0.0138	0.9653 ± 0.0168	0.9679 ± 0.0139	**0.9697 ± 0.0140**	0.9653 ± 0.0167	0.9669 ± 0.0180	0.9638 ± 0.0173	0.9686 ± 0.0128	0.9642 ± 0.0163	0.9685 ± 0.0124
D24	0.7040 ± 0.1362	0.7662 ± 0.0476	0.7743 ± 0.0591	0.7673 ± 0.0561	0.7735 ± 0.0558	0.7700 ± 0.0655	0.7644 ± 0.0728	0.7742 ± 0.0569	0.7805 ± 0.0749	**0.7961 ± 0.0667**
D25	0.2483 ± 0.2261	0.6817 ± 0.0956	0.6187 ± 0.1785	0.6997 ± 0.1052	0.7020 ± 0.1075	0.6965 ± 0.0974	0.5714 ± 0.2526	0.6822 ± 0.1282	0.5324 ± 0.1708	**0.7075 ± 0.0958**
D26	0.0163 ± 0.0797	0.6252 ± 0.1768	0.5603 ± 0.1867	0.6379 ± 0.1342	0.6534 ± 0.1351	0.6510 ± 0.1155	0.3855 ± 0.2498	0.6489 ± 0.1123	0.4943 ± 0.2606	**0.6632 ± 0.0970**
D27	0.4105 ± 0.2270	0.7323 ± 0.0885	0.7020 ± 0.1601	0.7329 ± 0.1363	0.7220 ± 0.1350	0.7339 ± 0.0902	0.6591 ± 0.1460	0.7340 ± 0.0922	0.7110 ± 0.1154	**0.7533 ± 0.0856**
D28	0.8628 ± 0.0354	0.8867 ± 0.0257	0.8926 ± 0.0343	0.9005 ± 0.0246	0.8844 ± 0.0286	0.8866 ± 0.0305	0.8820 ± 0.0291	0.8874 ± 0.0291	0.8833 ± 0.0295	**0.9011 ± 0.0216**
D29	0.4639 ± 0.1847	0.8022 ± 0.0741	0.7928 ± 0.0782	0.7947 ± 0.0698	0.7898 ± 0.0745	0.7930 ± 0.0817	0.7551 ± 0.0790	0.7920 ± 0.0794	0.7955 ± 0.1001	**0.8135 ± 0.0474**
D30	0.7557 ± 0.1211	0.9657 ± 0.0269	0.9592 ± 0.0381	0.9654 ± 0.0306	0.9674 ± 0.0248	**0.9718 ± 0.0227**	0.9596 ± 0.0404	0.9646 ± 0.0181	0.9655 ± 0.0319	0.9625 ± 0.0082
D31	0.6177 ± 0.1788	0.8761 ± 0.0716	0.8515 ± 0.0905	0.8640 ± 0.0680	0.8765 ± 0.0668	0.8666 ± 0.0783	0.8258 ± 0.0998	0.8710 ± 0.0863	0.8429 ± 0.1009	**0.8809 ± 0.0718**
Average	0.7070 ± 0.0950	0.8288 ± 0.0683	0.8091 ± 0.0890	0.8307 ± 0.0718	0.8306 ± 0.0764	0.8245 ± 0.0711	0.8088 ± 0.0906	0.8336 ± 0.0758	0.7947 ± 0.0826	**0.8499 ± 0.0582**

Table 11
AUC results of different oversampling methods under the LR classifier.

Dataset	SMOTE	SL-SMOTE	B-SMOTE	ADASYN	G-SMOTE	MWMOTE	ASN-SMOTE	ISMOTE	VS-SMOTE	RAN-SMOTE
D1	0.8196 ± 0.0676	0.8451 ± 0.0644	0.7402 ± 0.1047	0.8430 ± 0.0608	0.8304 ± 0.0702	0.7710 ± 0.0938	0.8214 ± 0.0982	0.8367 ± 0.0666	0.7570 ± 0.1299	**0.8586 ± 0.0479**
D2	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D3	0.8438 ± 0.1703	0.8280 ± 0.1723	0.8321 ± 0.1613	0.8446 ± 0.1606	0.8426 ± 0.1555	0.8061 ± 0.1228	0.8378 ± 0.1741	0.8368 ± 0.1797	0.8452 ± 0.1780	**0.9114 ± 0.1062**
D4	0.8415 ± 0.1525	0.8215 ± 0.1401	0.8394 ± 0.1496	0.8467 ± 0.1499	0.8519 ± 0.1444	0.8109 ± 0.1399	0.8370 ± 0.1608	0.8362 ± 0.1557	0.8414 ± 0.1584	**0.9129 ± 0.0886**
D5	0.9020 ± 0.1192	0.8832 ± 0.1277	0.9032 ± 0.1305	0.8701 ± 0.1374	0.8900 ± 0.1244	0.8920 ± 0.1214	0.9027 ± 0.1222	0.9015 ± 0.1220	0.8975 ± 0.1268	**0.9058 ± 0.1070**
D6	0.9952 ± 0.0084	0.9945 ± 0.0092	**0.9965 ± 0.0059**	0.9953 ± 0.0065	0.9953 ± 0.0076	0.9954 ± 0.0083	0.9947 ± 0.0090	0.9947 ± 0.0092	0.9956 ± 0.0073	0.9964 ± 0.0070
D7	**0.9581 ± 0.0180**	0.9508 ± 0.0298	0.9499 ± 0.0231	0.9523 ± 0.0241	0.9535 ± 0.0241	0.9511 ± 0.0297	0.9575 ± 0.0189	0.9504 ± 0.0303	0.9511 ± 0.0302	0.9478 ± 0.0228
D8	0.9222 ± 0.0558	0.9106 ± 0.0615	0.9158 ± 0.0677	0.9106 ± 0.0618	0.9138 ± 0.0609	0.9158 ± 0.0631	0.9289 ± 0.0493	0.9163 ± 0.0592	0.9196 ± 0.0564	**0.9423 ± 0.0345**
D9	0.9705 ± 0.0270	0.9689 ± 0.0285	0.9672 ± 0.0324	0.9652 ± 0.0312	0.9700 ± 0.0275	0.9631 ± 0.0263	0.9631 ± 0.0327	0.9647 ± 0.0347	**0.9720 ± 0.0247**	0.9592 ± 0.0369
D10	0.9856 ± 0.0325	**0.9940 ± 0.0236**	0.9923 ± 0.0211	0.9935 ± 0.0207	0.9928 ± 0.0219	0.9928 ± 0.0241	0.9867 ± 0.0294	0.9918 ± 0.0257	**0.9940 ± 0.0236**	0.9880 ± 0.0239
D11	0.8106 ± 0.0544	0.8135 ± 0.0747	0.8095 ± 0.0566	0.8107 ± 0.0784	0.8146 ± 0.0520	0.8105 ± 0.0512	0.8146 ± 0.0759	**0.8152 ± 0.0509**	0.8151 ± 0.0723	0.8237 ± 0.0539
D12	0.6475 ± 0.0795	0.6487 ± 0.0928	0.6169 ± 0.0713	0.6336 ± 0.0697	**0.6596 ± 0.0697**	0.6586 ± 0.0746	0.6510 ± 0.0917	0.6438 ± 0.0930	0.6498 ± 0.0824	0.6587 ± 0.0529
D13	0.8937 ± 0.1227	0.8581 ± 0.1554	0.8784 ± 0.1586	0.8589 ± 0.1794	0.8801 ± 0.1575	0.8651 ± 0.1683	0.8694 ± 0.1540	0.8499 ± 0.1661	0.8636 ± 0.1619	**0.9252 ± 0.0972**
D14	0.6608 ± 0.0588	0.6646 ± 0.0562	0.6573 ± 0.0807	0.6625 ± 0.0795	0.6677 ± 0.0799	0.6337 ± 0.0823	0.6524 ± 0.0685	0.6700 ± 0.0873	0.6679 ± 0.0653	**0.6906 ± 0.0521**
D15	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D16	0.9558 ± 0.0359	0.9435 ± 0.0412	0.9384 ± 0.0360	0.9451 ± 0.0343	0.9497 ± 0.0328	0.9370 ± 0.0463	0.9463 ± 0.0350	0.9528 ± 0.0345	0.9486 ± 0.0417	**0.9559 ± 0.0316**
D17	0.9995 ± 0.0017	0.9994 ± 0.0018	0.9990 ± 0.0037	0.9995 ± 0.0020	0.9995 ± 0.0015	0.9992 ± 0.0019	0.9997 ± 0.0011	0.9992 ± 0.0019	0.9997 ± 0.0011	**1.0000 ± 0.0000**
D18	0.8313 ± 0.0305	0.8311 ± 0.0308	0.8287 ± 0.0362	0.8284 ± 0.0224	0.8297 ± 0.0227	0.8250 ± 0.0258	0.8299 ± 0.0247	**0.8317 ± 0.0308**	0.8315 ± 0.0311	0.8280 ± 0.0366
D19	0.3243 ± 0.1468	0.3724 ± 0.1253	0.4268 ± 0.1163	0.3738 ± 0.1206	0.3784 ± 0.1265	0.3910 ± 0.0954	0.3548 ± 0.1194	0.4098 ± 0.1579	0.3589 ± 0.1074	**0.4655 ± 0.1204**
D20	0.9987 ± 0.0011	0.9993 ± 0.0011	0.9996 ± 0.0006	0.9995 ± 0.0008	0.9992 ± 0.0011	0.9992 ± 0.0012	0.9992 ± 0.0012	0.9992 ± 0.0011	0.9992 ± 0.0012	**0.9998 ± 0.0004**
D21	0.9918 ± 0.0164	0.9923 ± 0.0155	0.9918 ± 0.0165	0.9902 ± 0.0175	0.9935 ± 0.0149	0.9966 ± 0.0113	0.9919 ± 0.0161	0.9916 ± 0.0193	0.9919 ± 0.0161	**1.0000 ± 0.0000**
D22	0.9949 ± 0.0024	0.9944 ± 0.0032	0.9938 ± 0.0028	0.9945 ± 0.0026	0.9945 ± 0.0027	**0.9954 ± 0.0029**	0.9945 ± 0.0031	0.9945 ± 0.0036	0.9950 ± 0.0031	0.9945 ± 0.0047
D23	0.9951 ± 0.0039	**0.9957 ± 0.0027**	0.9949 ± 0.0027	0.9945 ± 0.0046	0.9951 ± 0.0038	0.9951 ± 0.0031	**0.9957 ± 0.0026**	0.9952 ± 0.0038	0.9956 ± 0.0028	**0.9957 ± 0.0045**
D24	0.8492 ± 0.0804	0.8349 ± 0.0549	0.8328 ± 0.0655	0.8332 ± 0.0632	0.8384 ± 0.0595	0.8409 ± 0.0646	0.8155 ± 0.0696	0.8436 ± 0.0581	0.8424 ± 0.0627	**0.8521 ± 0.0800**
D25	**0.7911 ± 0.0917**	0.7744 ± 0.0952	0.7694 ± 0.0936	0.7909 ± 0.0876	0.7846 ± 0.0931	0.7801 ± 0.0831	0.7388 ± 0.1129	0.7879 ± 0.0908	0.7555 ± 0.0861	**0.7911 ± 0.0767**
D26	0.7202 ± 0.1020	0.7146 ± 0.1416	0.7022 ± 0.1182	0.7129 ± 0.1190	0.7174 ± 0.1161	0.7074 ± 0.1062	0.5886 ± 0.1331	0.7088 ± 0.1075	0.6974 ± 0.1183	**0.7338 ± 0.0922**
D27	0.8483 ± 0.0741	0.8412 ± 0.0657	0.8333 ± 0.0874	0.8387 ± 0.0865	0.8360 ± 0.0864	0.8420 ± 0.0743	0.8150 ± 0.0713	0.8325 ± 0.0701	0.8475 ± 0.0640	**0.8559 ± 0.0667**
D28	**0.9672 ± 0.0112**	0.9651 ± 0.0126	0.9646 ± 0.0124	0.9657 ± 0.0117	0.9658 ± 0.0115	0.9660 ± 0.0119	0.9657 ± 0.0125	0.9655 ± 0.0120	0.9663 ± 0.0121	0.9661 ± 0.0127
D29	0.8751 ± 0.0607	0.8651 ± 0.0705	0.8628 ± 0.0630	0.8691 ± 0.0624	0.8674 ± 0.0634	0.8692 ± 0.0753	0.8335 ± 0.0695	0.8689 ± 0.0567	0.8657 ± 0.0795	**0.8812 ± 0.0542**
D30	0.9869 ± 0.0053	0.9862 ± 0.0061	0.9843 ± 0.0085	0.9855 ± 0.0070	0.9860 ± 0.0063	0.9858 ± 0.0069	0.9859 ± 0.0060	0.9862 ± 0.0061	0.9865 ± 0.0061	**0.9867 ± 0.0066**
D31	0.9442 ± 0.0408	0.9426 ± 0.0433	0.9340 ± 0.0432	0.9379 ± 0.0442	0.9403 ± 0.0405	0.9376 ± 0.0400	**0.9457 ± 0.0479**	0.9421 ± 0.0460	0.9449 ± 0.0522	0.9453 ± 0.0437
Average	0.8754 ± 0.0485	0.8694 ± 0.0488	0.8663 ± 0.0537	0.8695 ± 0.0532	0.8704 ± 0.0502	0.8657 ± 0.0512	0.8627 ± 0.0571	0.8688 ± 0.0557	0.8648 ± 0.0604	**0.8929 ± 0.0444**

Table 12
F1-score results of different oversampling methods under the LR classifier.

Dataset	SMOTE	SL-SMOTE	B-SMOTE	ADASYN	G-SMOTE	MWMOTE	ASN-SMOTE	ISMOTE	VS-SMOTE	RAN-SMOTE
D1	0.0000 ± 0.0000	0.0468 ± 0.0108	0.0474 ± 0.0421	0.0506 ± 0.0144	0.0470 ± 0.0132	0.0478 ± 0.0256	0.0428 ± 0.0192	0.0476 ± 0.0158	0.0046 ± 0.0158	**0.0577 ± 0.0152**
D2	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9956 ± 0.0218	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D3	0.7112 ± 0.1777	0.5569 ± 0.1431	0.6740 ± 0.1726	0.5537 ± 0.1457	0.6250 ± 0.1332	0.5275 ± 0.1080	0.6729 ± 0.1841	0.6179 ± 0.1520	0.6725 ± 0.1740	**0.7288 ± 0.1635**
D4	0.7325 ± 0.1730	0.5649 ± 0.1642	0.6967 ± 0.1760	0.5639 ± 0.1405	0.6525 ± 0.1642	0.5132 ± 0.1242	0.6875 ± 0.1802	0.6623 ± 0.1545	0.7221 ± 0.1685	**0.7345 ± 0.1289**
D5	**0.7810 ± 0.1332**	0.6064 ± 0.1400	0.7219 ± 0.1641	0.5639 ± 0.1332	0.6665 ± 0.1324	0.5820 ± 0.1463	0.7346 ± 0.1478	0.7138 ± 0.1383	0.7402 ± 0.1464	0.6875 ± 0.1491
D6	0.9707 ± 0.0347	0.9709 ± 0.0454	**0.9751 ± 0.0301**	0.9432 ± 0.0368	0.9700 ± 0.0339	0.9538 ± 0.0340	0.9657 ± 0.0405	0.9649 ± 0.0412	0.9675 ± 0.0373	0.9679 ± 0.0408
D7	0.7440 ± 0.0625	0.7433 ± 0.0755	0.7487 ± 0.0678	**0.7562 ± 0.0613**	0.7499 ± 0.0609	0.7460 ± 0.0734	0.7548 ± 0.0585	0.7353 ± 0.0787	0.7487 ± 0.0709	**0.7761 ± 0.0622**
D8	0.6590 ± 0.1284	0.6022 ± 0.0710	0.6308 ± 0.0962	0.5962 ± 0.1151	0.6184 ± 0.1118	0.6371 ± 0.0738	**0.6716 ± 0.0879**	0.6452 ± 0.0729	0.6558 ± 0.0881	0.5772 ± 0.0930
D9	0.8510 ± 0.0801	0.8494 ± 0.0663	0.8497 ± 0.0663	0.8477 ± 0.0693	0.8493 ± 0.0878	0.8247 ± 0.0690	0.8513 ± 0.0668	0.8549 ± 0.0689	0.8393 ± 0.0785	**0.8558 ± 0.0921**
D10	0.7949 ± 0.3320	0.7655 ± 0.1894	0.7690 ± 0.2393	0.7683 ± 0.2415	0.7875 ± 0.2853	0.7568 ± 0.2471	0.7032 ± 0.2361	0.7629 ± 0.2654	**0.7655 ± 0.2364**	0.7122 ± 0.2034
D11	0.6408 ± 0.0840	0.6369 ± 0.0822	0.6359 ± 0.0665	0.6470 ± 0.0783	0.6359 ± 0.0791	0.6369 ± 0.0795	0.6319 ± 0.0760	0.6401 ± 0.0847	0.6334 ± 0.0818	**0.6639 ± 0.0560**
D12	0.5607 ± 0.0824	0.5455 ± 0.0958	0.5341 ± 0.0641	0.5384 ± 0.0712	0.5534 ± 0.0653	0.5614 ± 0.0735	0.5492 ± 0.0950	0.5410 ± 0.0960	0.5511 ± 0.0830	**0.5855 ± 0.0713**
D13	0.4133 ± 0.2738	0.5231 ± 0.1424	0.5153 ± 0.1899	0.5436 ± 0.1816	0.5089 ± 0.1827	0.5238 ± 0.1557	0.5074 ± 0.1559	0.5347 ± 0.1411	0.5397 ± 0.1601	**0.6313 ± 0.1771**
D14	0.4151 ± 0.1125	0.4564 ± 0.0867	0.4576 ± 0.0882	0.4673 ± 0.0915	0.4711 ± 0.1044	0.4449 ± 0.0785	0.4396 ± 0.0962	0.4662 ± 0.0970	0.4499 ± 0.1022	**0.4986 ± 0.0676**
D15	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D16	**0.7798 ± 0.1257**	0.5594 ± 0.0908	0.5749 ± 0.1256	0.5729 ± 0.1272	0.6628 ± 0.1267	0.5117 ± 0.0818	0.6746 ± 0.1453	0.7309 ± 0.1184	0.6103 ± 0.1138	0.7616 ± 0.0685
D17	0.9476 ± 0.0542	0.9609 ± 0.0543	0.9411 ± 0.0643	0.9484 ± 0.0518	0.9567 ± 0.0483	0.9602 ± 0.0531	0.9537 ± 0.0454	0.9622 ± 0.0508	0.9614 ± 0.0436	**0.9660 ± 0.0412**
D18	0.6704 ± 0.0372	0.6693 ± 0.0321	0.6638 ± 0.0449	0.6635 ± 0.0295	0.6706 ± 0.0308	0.6566 ± 0.0325	0.6640 ± 0.0353	0.6668 ± 0.0432	0.6604 ± 0.0429	**0.6776 ± 0.0414**
D19	0.0000 ± 0.0000	0.0219 ± 0.0168	**0.0333 ± 0.0307**	0.0220 ± 0.0124	0.0247 ± 0.0143	0.0213 ± 0.0100	0.0202 ± 0.0146	0.0289 ± 0.0161	0.0177 ± 0.0111	0.0311 ± 0.0163
D20	0.9884 ± 0.0071	0.9873 ± 0.0092	0.9699 ± 0.0171	0.9778 ± 0.0127	0.9884 ± 0.0091	0.9884 ± 0.0100	0.9875 ± 0.0099	0.9887 ± 0.0093	0.9890 ± 0.0097	**0.9891 ± 0.0068**
D21	0.9950 ± 0.0088	0.9959 ± 0.0082	0.9958 ± 0.0084	0.9934 ± 0.0132	0.9959 ± 0.0091	0.9974 ± 0.0069	0.9951 ± 0.0104	0.9941 ± 0.0137	0.9959 ± 0.0082	**1.0000 ± 0.0000**
D22	0.9293 ± 0.0214	0.9175 ± 0.0277	0.9207 ± 0.0251	0.9216 ± 0.0240	0.9254 ± 0.0239	0.9249 ± 0.0255	0.9144 ± 0.0237	0.9211 ± 0.0324	0.9264 ± 0.0262	**0.9366 ± 0.0308**
D23	0.9530 ± 0.0178	0.9544 ± 0.0188	0.9532 ± 0.0184	**0.9558 ± 0.0184**	0.9545 ± 0.0209	0.9534 ± 0.0210	0.9530 ± 0.0191	0.9577 ± 0.0163	0.9528 ± 0.0185	0.9552 ± 0.0172
D24	**0.5600 ± 0.1557**	0.4161 ± 0.0500	0.4683 ± 0.0710	0.4040 ± 0.0533	0.4342 ± 0.0652	0.4285 ± 0.0725	0.4770 ± 0.0797	0.4436 ± 0.0552	0.5206 ± 0.0714	0.4667 ± 0.0661
D25	0.1686 ± 0.1636	0.1323 ± 0.0325	0.1772 ± 0.0765	0.1418 ± 0.0352	0.1478 ± 0.0405	0.1440 ± 0.0346	0.1569 ± 0.0857	0.1359 ± 0.0434	0.1485 ± 0.0660	0.1522 ± 0.0421
D26	0.0100 ± 0.0490	0.1390 ± 0.0550	0.1483 ± 0.0713	0.1409 ± 0.0431	0.1520 ± 0.0459	0.1521 ± 0.0440	0.0927 ± 0.0622	0.1488 ± 0.0432	**0.1610 ± 0.0938**	0.1583 ± 0.0380
D27	0.2696 ± 0.1752	0.2926 ± 0.0623	0.3474 ± 0.1109	0.3005 ± 0.0750	0.2977 ± 0.0752	0.3012 ± 0.0619	0.2905 ± 0.0855	0.2959 ± 0.0742	**0.3625 ± 0.1140**	0.3069 ± 0.0692
D28	**0.7547 ± 0.0478**	0.6601 ± 0.0509	0.6689 ± 0.0512	0.6517 ± 0.0388	0.6748 ± 0.0424	0.6677 ± 0.0436	0.7176 ± 0.0461	0.6777 ± 0.0455	0.7087 ± 0.0532	0.6401 ± 0.0470
D29	0.3091 ± 0.1552	0.2581 ± 0.0374	0.3211 ± 0.0552	0.2440 ± 0.0366	0.2608 ± 0.0375	0.2717 ± 0.0419	0.2760 ± 0.0777	0.2735 ± 0.0500	**0.3412 ± 0.0668**	0.2898 ± 0.0375
D30	0.5648 ± 0.1482	0.5518 ± 0.0721	0.5508 ± 0.0746	0.5577 ± 0.0552	**0.5727 ± 0.0733**	0.5684 ± 0.0583	0.5334 ± 0.0525	0.5405 ± 0.0651	0.5726 ± 0.0777	0.4606 ± 0.0598
D31	**0.4588 ± 0.1739**	0.3064 ± 0.0408	0.3668 ± 0.0752	0.2416 ± 0.0331	0.3119 ± 0.0432	0.3130 ± 0.0659	0.4091 ± 0.1045	0.3168 ± 0.0612	0.3943 ± 0.0822	0.3166 ± 0.0692
Average	**0.6333 ± 0.0973**	0.6029 ± 0.0636	0.6243 ± 0.0776	0.5993 ± 0.0658	0.6183 ± 0.0697	0.6005 ± 0.0630	0.6235 ± 0.0755	0.6216 ± 0.0692	0.6327 ± 0.0756	0.6318 ± 0.0636

Table 13
G-mean results of different oversampling methods under the KNN classifier.

Dataset	SMOTE	SL-SMOTE	B-SMOTE	ADASYN	G-SMOTE	MWMOTE	ASN-SMOTE	ISMOTE	VS-SMOTE	RAN-SMOTE
D1	0.0163 ± 0.0800	0.1970 ± 0.2094	0.0157 ± 0.0770	**0.4991 ± 0.1753**	0.3193 ± 0.2468	0.1496 ± 0.2052	0.1993 ± 0.2157	0.4670 ± 0.2050	0.0000 ± 0.0000	0.4537 ± 0.2194
D2	**1.0000 ± 0.0000**	0.9973 ± 0.0036	0.9996 ± 0.0018	0.9984 ± 0.0031	0.9979 ± 0.0033	0.9976 ± 0.0040	0.9961 ± 0.0043	0.9979 ± 0.0039	0.9976 ± 0.0040	0.9985 ± 0.0300
D3	0.7639 ± 0.2080	0.8023 ± 0.1296	0.7854 ± 0.1421	**0.8425 ± 0.1263**	0.8395 ± 0.1314	0.7838 ± 0.2045	0.7822 ± 0.1052	0.8211 ± 0.1535	0.8216 ± 0.1546	0.8348 ± 0.1273
D4	0.7938 ± 0.1490	0.7979 ± 0.1574	0.8026 ± 0.1307	**0.8413 ± 0.1337**	0.8394 ± 0.1374	0.7972 ± 0.1606	0.7711 ± 0.1589	0.8252 ± 0.1447	0.8174 ± 0.1702	0.8379 ± 0.1345
D5	0.8287 ± 0.1298	0.8240 ± 0.1249	0.8357 ± 0.1379	0.8464 ± 0.1057	0.8391 ± 0.1139	0.8289 ± 0.1264	0.8312 ± 0.1297	0.8377 ± 0.1224	0.8473 ± 0.1195	**0.8860 ± 0.1099**
D6	0.9599 ± 0.0376	0.9590 ± 0.0430	0.9641 ± 0.0327	0.9550 ± 0.0296	0.9619 ± 0.0330	0.9627 ± 0.0246	0.9558 ± 0.0366	0.9638 ± 0.0409	0.9613 ± 0.0439	**0.9867 ± 0.0186**
D7	0.8598 ± 0.0752	0.8767 ± 0.0625	0.8758 ± 0.0548	0.8751 ± 0.0369	0.8740 ± 0.0469	0.8725 ± 0.0395	0.8671 ± 0.0735	0.8702 ± 0.0649	0.8660 ± 0.0685	**0.8942 ± 0.0477**
D8	0.7844 ± 0.0914	0.8567 ± 0.0604	0.8493 ± 0.1013	0.8475 ± 0.0918	0.8477 ± 0.0923	0.8339 ± 0.0607	0.8306 ± 0.0847	0.8572 ± 0.0640	0.8480 ± 0.0586	**0.8779 ± 0.0534**
D9	0.8929 ± 0.0591	0.9073 ± 0.0547	0.9076 ± 0.0487	0.9007 ± 0.0543	0.9040 ± 0.0599	0.8901 ± 0.0622	0.8919 ± 0.0608	0.8970 ± 0.0701	0.8966 ± 0.0772	**0.9573 ± 0.0341**
D10	0.8808 ± 0.2199	0.8902 ± 0.2027	0.8859 ± 0.2132	0.9083 ± 0.2068	0.9307 ± 0.1091	0.8961 ± 0.2108	0.9135 ± 0.0958	0.8757 ± 0.2708	0.9468 ± 0.0976	**0.9865 ± 0.0198**
D11	0.7881 ± 0.0631	0.7765 ± 0.0523	0.7827 ± 0.0613	0.7896 ± 0.0524	**0.7927 ± 0.0538**	0.7868 ± 0.0545	0.7899 ± 0.0659	0.7829 ± 0.0473	0.7875 ± 0.0761	0.7839 ± 0.0526
D12	0.7531 ± 0.0810	0.7449 ± 0.0677	0.7391 ± 0.0698	0.7534 ± 0.0782	0.7460 ± 0.0538	0.7543 ± 0.0479	0.7464 ± 0.0867	0.7510 ± 0.0793	0.7419 ± 0.0821	**0.7990 ± 0.0622**
D13	0.7811 ± 0.2626	0.9031 ± 0.0908	0.8910 ± 0.2019	0.8905 ± 0.1180	0.8795 ± 0.1211	0.8228 ± 0.2693	0.9134 ± 0.1127	0.8780 ± 0.1286	0.8839 ± 0.1130	**0.9304 ± 0.1097**
D14	0.5373 ± 0.0711	0.5866 ± 0.0700	0.5724 ± 0.0730	0.5828 ± 0.0757	0.5763 ± 0.0704	0.5574 ± 0.0654	0.5417 ± 0.0574	0.5829 ± 0.0741	0.5678 ± 0.0798	**0.5956 ± 0.0580**
D15	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D16	0.4848 ± 0.2059	0.4766 ± 0.1554	0.5064 ± 0.2079	0.5105 ± 0.1963	0.5270 ± 0.1964	0.4762 ± 0.1547	0.4818 ± 0.2084	0.5094 ± 0.1505	0.5105 ± 0.2211	**0.8995 ± 0.0697**
D17	0.9569 ± 0.0666	0.9622 ± 0.0601	0.9644 ± 0.0566	0.9744 ± 0.0442	**0.9820 ± 0.0315**	0.9547 ± 0.0653	0.9700 ± 0.0529	0.9714 ± 0.0598	0.9714 ± 0.0605	0.9613 ± 0.0433
D18	0.7101 ± 0.0295	**0.7128 ± 0.0280**	0.7078 ± 0.0336	0.7074 ± 0.0340	0.7086 ± 0.0409	0.7105 ± 0.0391	0.7063 ± 0.0339	0.7101 ± 0.0358	0.7091 ± 0.0390	0.6964 ± 0.0377
D19	0.7660 ± 0.1432	0.6812 ± 0.1509	0.8266 ± 0.1717	**0.9739 ± 0.0408**	0.9600 ± 0.0534	0.8978 ± 0.1284	0.8696 ± 0.0863	0.9709 ± 0.0331	0.9173 ± 0.0730	0.5954 ± 0.1885
D20	0.9906 ± 0.0078	0.9913 ± 0.0063	0.9924 ± 0.0078	0.9929 ± 0.0066	0.9934 ± 0.0065	0.9921 ± 0.0068	0.9900 ± 0.0087	**0.9937 ± 0.0061**	0.9933 ± 0.0068	0.9902 ± 0.0063
D21	0.9934 ± 0.0097	0.9934 ± 0.0097	0.9938 ± 0.0103	0.9938 ± 0.0103	0.9942 ± 0.0101	0.9934 ± 0.0097	0.9933 ± 0.0098	0.9928 ± 0.0103	0.9933 ± 0.0098	**0.9959 ± 0.0083**
D22	0.9274 ± 0.0236	0.9260 ± 0.0185	0.9236 ± 0.0230	0.9255 ± 0.0213	0.9322 ± 0.0197	0.9327 ± 0.0143	0.9208 ± 0.0189	0.9275 ± 0.0212	0.9267 ± 0.0147	**0.9398 ± 0.0192**
D23	0.9699 ± 0.0147	0.9678 ± 0.0136	0.9684 ± 0.0153	0.9710 ± 0.0139	0.9686 ± 0.0174	0.9722 ± 0.0144	0.9665 ± 0.0167	0.9655 ± 0.0148	0.9692 ± 0.0131	**0.9797 ± 0.0123**
D24	0.6233 ± 0.1608	0.7142 ± 0.0727	0.7228 ± 0.0837	0.6999 ± 0.0993	0.7116 ± 0.1006	0.7105 ± 0.0677	0.6449 ± 0.0978	0.7313 ± 0.0716	0.7037 ± 0.0912	**0.8118 ± 0.0830**
D25	0.3353 ± 0.2544	0.5273 ± 0.1834	0.5449 ± 0.2106	**0.6429 ± 0.1308**	0.5852 ± 0.1944	0.5811 ± 0.1553	0.4006 ± 0.2481	0.6395 ± 0.1321	0.4565 ± 0.2009	0.6296 ± 0.1769
D26	0.2149 ± 0.2518	0.5544 ± 0.1314	0.4589 ± 0.1757	0.5893 ± 0.1780	0.5699 ± 0.1771	0.5428 ± 0.2131	0.0922 ± 0.1871	**0.5946 ± 0.1870**	0.4128 ± 0.2074	0.5770 ± 0.1297
D27	0.5837 ± 0.1219	0.6550 ± 0.1185	0.5933 ± 0.1953	0.6951 ± 0.1493	**0.7128 ± 0.1472**	0.6941 ± 0.1322	0.5826 ± 0.1533	0.6983 ± 0.1240	0.6195 ± 0.2236	0.6966 ± 0.1114
D28	0.8561 ± 0.0331	0.8678 ± 0.0312	0.8721 ± 0.0280	0.8812 ± 0.0270	0.8754 ± 0.0261	0.8790 ± 0.0265	0.8550 ± 0.0366	0.8755 ± 0.0387	0.8739 ± 0.0334	**0.9070 ± 0.0241**
D29	0.6043 ± 0.0922	0.7070 ± 0.1105	0.6825 ± 0.1173	0.7462 ± 0.0945	0.6883 ± 0.1135	0.7023 ± 0.1061	0.5895 ± 0.1217	0.7406 ± 0.0717	0.6811 ± 0.0922	**0.7820 ± 0.0643**
D30	0.8752 ± 0.0671	0.9518 ± 0.0360	0.9136 ± 0.0595	0.9381 ± 0.0473	0.9221 ± 0.0635	0.9097 ± 0.0724	0.8939 ± 0.0729	0.9492 ± 0.0428	0.9130 ± 0.0679	**0.9589 ± 0.0466**
D31	0.7513 ± 0.1210	0.8239 ± 0.0887	0.7918 ± 0.1056	0.7976 ± 0.0918	0.7901 ± 0.0915	0.7840 ± 0.0994	0.7953 ± 0.1043	0.8295 ± 0.0769	0.8010 ± 0.1008	**0.8528 ± 0.0853**
Average	0.7511 ± 0.1010	0.7946 ± 0.0821	0.7861 ± 0.0919	0.8248 ± 0.0798	0.8151 ± 0.0827	0.7957 ± 0.0916	0.7672 ± 0.0886	0.8228 ± 0.0821	0.7883 ± 0.0839	**0.8418 ± 0.0704**

Table 14
AUC results of different oversampling methods under the KNN classifier.

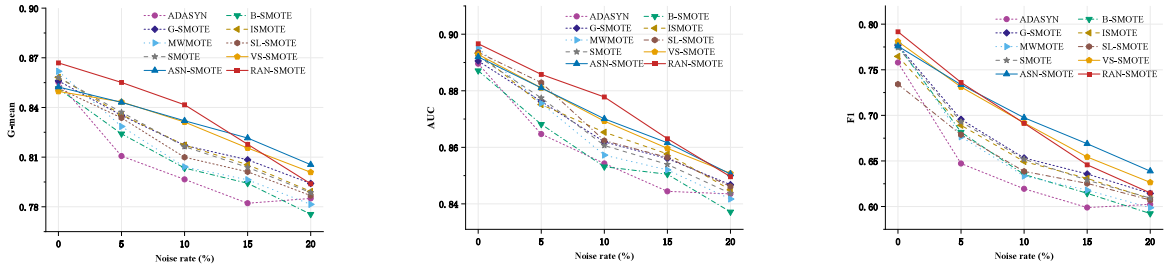
Dataset	SMOTE	SL-SMOTE	B-SMOTE	ADASYN	G-SMOTE	MWMOTE	ASN-SMOTE	ISMOTE	VS-SMOTE	RAN-SMOTE
D1	0.5508 ± 0.0548	0.5729 ± 0.0699	0.5449 ± 0.0544	**0.6640 ± 0.0987**	0.5915 ± 0.0831	0.5576 ± 0.0737	0.5524 ± 0.0587	0.6498 ± 0.0890	0.5312 ± 0.0575	0.6632 ± 0.1101
D2	**1.0000 ± 0.0000**	0.9997 ± 0.0014	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9997 ± 0.0015	0.9994 ± 0.0020	0.9985 ± 0.0030	0.9988 ± 0.0027	0.9988 ± 0.0027	**1.0000 ± 0.0000**
D3	0.8906 ± 0.1268	0.8973 ± 0.1138	0.9011 ± 0.0972	0.8989 ± 0.1101	0.9043 ± 0.1079	0.8881 ± 0.1264	**0.9093 ± 0.0895**	0.8999 ± 0.1362	0.8918 ± 0.1300	0.8898 ± 0.0730
D4	0.9074 ± 0.1124	0.8993 ± 0.1037	0.9000 ± 0.1026	0.9055 ± 0.0972	0.9085 ± 0.1026	0.9024 ± 0.1117	0.9033 ± 0.1123	**0.9096 ± 0.1085**	0.9028 ± 0.1219	0.8848 ± 0.0964
D5	0.8977 ± 0.0942	0.8949 ± 0.1016	0.9017 ± 0.0966	0.9014 ± 0.0958	0.8991 ± 0.0900	0.8874 ± 0.0916	0.8934 ± 0.1000	0.9035 ± 0.1042	0.9059 ± 0.0910	**0.9165 ± 0.0925**
D6	0.9754 ± 0.0228	0.9757 ± 0.0298	0.9812 ± 0.0225	0.9767 ± 0.0246	0.9803 ± 0.0221	0.9810 ± 0.0193	0.9721 ± 0.0231	0.9746 ± 0.0306	0.9755 ± 0.0355	**0.9867 ± 0.0187**
D7	0.9230 ± 0.0548	0.9308 ± 0.0466	0.9200 ± 0.0420	0.9222 ± 0.0329	0.9269 ± 0.0359	0.9294 ± 0.0360	0.9304 ± 0.0487	0.9269 ± 0.0448	0.9292 ± 0.0524	**0.9432 ± 0.0342**
D8	0.8889 ± 0.0645	0.9009 ± 0.0506	0.8955 ± 0.0737	0.8865 ± 0.0690	0.8882 ± 0.0690	0.8779 ± 0.0619	0.9013 ± 0.0578	0.8955 ± 0.0573	0.8873 ± 0.0609	**0.9191 ± 0.0474**
D9	0.9484 ± 0.0444	0.9572 ± 0.0404	0.9670 ± 0.0300	0.9621 ± 0.0385	0.9575 ± 0.0404	0.9466 ± 0.0441	0.9520 ± 0.0473	0.9607 ± 0.0421	0.9500 ± 0.0413	**0.9765 ± 0.0240**
D10	0.9564 ± 0.1229	0.9679 ± 0.0773	0.9269 ± 0.1372	0.9772 ± 0.0662	0.9810 ± 0.0541	0.9351 ± 0.1344	0.9661 ± 0.0825	0.9797 ± 0.0644	0.9721 ± 0.0711	**0.9940 ± 0.0236**
D11	0.8775 ± 0.0612	0.8803 ± 0.0540	0.8809 ± 0.0599	0.8825 ± 0.0565	0.8807 ± 0.0526	0.8818 ± 0.0580	**0.8852 ± 0.0645**	0.8827 ± 0.0543	0.8791 ± 0.0665	0.8685 ± 0.0518
D12	0.8420 ± 0.0704	0.8482 ± 0.0435	0.8281 ± 0.0656	0.8373 ± 0.0649	0.8411 ± 0.0491	0.8409 ± 0.0440	0.8359 ± 0.0695	0.8348 ± 0.0644	0.8381 ± 0.0716	**0.9000 ± 0.0391**
D13	0.9415 ± 0.0939	0.9334 ± 0.0914	0.9382 ± 0.0837	0.9351 ± 0.0842	0.9280 ± 0.0884	0.9283 ± 0.0861	0.9335 ± 0.0849	0.9321 ± 0.0839	0.9345 ± 0.0792	**0.9510 ± 0.0803**
D14	0.6257 ± 0.0578	0.6238 ± 0.0620	0.6265 ± 0.0688	0.6173 ± 0.0663	0.6120 ± 0.0607	0.6054 ± 0.0555	0.6296 ± 0.0643	0.6256 ± 0.0739	0.6287 ± 0.0585	**0.6389 ± 0.0478**
D15	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D16	0.6714 ± 0.1194	0.6495 ± 0.0805	0.6669 ± 0.1173	0.6661 ± 0.1168	0.6676 ± 0.1159	0.6513 ± 0.0789	0.6861 ± 0.1430	0.6525 ± 0.0812	0.6924 ± 0.1426	**0.9071 ± 0.0628**
D17	0.9949 ± 0.0147	0.9937 ± 0.0085	0.9921 ± 0.0173	0.9931 ± 0.0167	0.9948 ± 0.0165	0.9917 ± 0.0148	0.9904 ± 0.0215	0.9914 ± 0.0211	0.9913 ± 0.0218	**0.9976 ± 0.0048**
D18	0.7674 ± 0.0343	0.7705 ± 0.0301	0.7687 ± 0.0333	0.7639 ± 0.0334	0.7716 ± 0.0378	0.7717 ± 0.0328	0.7709 ± 0.0377	0.7694 ± 0.0311	**0.7732 ± 0.0280**	0.7527 ± 0.0391
D19	0.9293 ± 0.0742	0.8800 ± 0.0939	0.9334 ± 0.0681	**0.9899 ± 0.0351**	0.9737 ± 0.0488	0.9171 ± 0.0965	0.9359 ± 0.0605	0.9851 ± 0.0353	0.9474 ± 0.0512	0.7290 ± 0.1202
D20	0.9980 ± 0.0052	0.9982 ± 0.0044	**0.9983 ± 0.0042**	0.9981 ± 0.0039	0.9982 ± 0.0039	0.9976 ± 0.0053	0.9978 ± 0.0043	**0.9983 ± 0.0045**	0.9982 ± 0.0044	0.9970 ± 0.0032
D21	0.9951 ± 0.0088	0.9951 ± 0.0088	0.9960 ± 0.0081	0.9960 ± 0.0081	0.9955 ± 0.0084	0.9950 ± 0.0088	0.9950 ± 0.0088	0.9948 ± 0.0090	0.9951 ± 0.0088	**0.9967 ± 0.0076**
D22	**0.9822 ± 0.0087**	0.9792 ± 0.0078	0.9751 ± 0.0094	0.9773 ± 0.0094	0.9786 ± 0.0096	0.9801 ± 0.0081	0.9784 ± 0.0084	0.9793 ± 0.0101	0.9794 ± 0.0087	0.9806 ± 0.0098
D23	**0.9861 ± 0.0124**	0.9852 ± 0.0127	0.9839 ± 0.0113	0.9818 ± 0.0110	0.9850 ± 0.0111	0.9834 ± 0.0092	0.9859 ± 0.0120	0.9858 ± 0.0097	0.9846 ± 0.0110	0.9835 ± 0.0105
D24	0.7933 ± 0.0556	0.7965 ± 0.0701	0.8057 ± 0.0681	0.7912 ± 0.0725	0.7825 ± 0.0781	0.7861 ± 0.0609	0.7806 ± 0.0692	0.8085 ± 0.0614	0.7875 ± 0.0660	**0.8432 ± 0.0768**
D25	0.7087 ± 0.0918	0.6874 ± 0.0944	0.7063 ± 0.1171	**0.7271 ± 0.1008**	0.7025 ± 0.1113	0.6795 ± 0.1095	0.6671 ± 0.0968	0.6913 ± 0.1157	0.6894 ± 0.1026	0.7187 ± 0.1026
D26	0.6373 ± 0.1113	0.6291 ± 0.0956	0.6683 ± 0.1029	0.6650 ± 0.1262	0.6657 ± 0.1135	0.6596 ± 0.1274	0.6453 ± 0.0972	0.6415 ± 0.1241	0.6814 ± 0.1059	**0.6852 ± 0.0945**
D27	0.7676 ± 0.0842	0.7341 ± 0.1040	0.7370 ± 0.1213	0.7423 ± 0.1081	0.7456 ± 0.1021	0.7349 ± 0.1038	0.7438 ± 0.1015	0.7506 ± 0.0999	0.7752 ± 0.1020	**0.7785 ± 0.0718**
D28	0.9180 ± 0.0275	0.9232 ± 0.0255	0.9179 ± 0.0231	0.9209 ± 0.0234	0.9154 ± 0.0226	0.9214 ± 0.0244	0.9166 ± 0.0310	0.9263 ± 0.0326	0.9144 ± 0.0313	**0.9356 ± 0.0194**
D29	0.7850 ± 0.0601	0.8190 ± 0.0697	0.7858 ± 0.0885	0.8115 ± 0.0676	0.7691 ± 0.0768	0.7718 ± 0.0895	0.7630 ± 0.0810	0.8336 ± 0.0436	0.7715 ± 0.0876	**0.8442 ± 0.0538**
D30	0.9701 ± 0.0383	0.9810 ± 0.0250	0.9711 ± 0.0267	0.9793 ± 0.0252	0.9724 ± 0.0363	0.9742 ± 0.0339	0.9694 ± 0.0363	0.9826 ± 0.0156	0.9675 ± 0.0368	**0.9876 ± 0.0110**
D31	0.8674 ± 0.0799	0.8745 ± 0.0736	0.8504 ± 0.0809	0.8592 ± 0.0722	0.8434 ± 0.0754	0.8540 ± 0.0802	0.8601 ± 0.0743	0.8909 ± 0.0710	0.8621 ± 0.0769	**0.9055 ± 0.0642**
Average	0.8709 ± 0.0583	0.8703 ± 0.0545	0.8700 ± 0.0591	0.8784 ± 0.0560	0.8729 ± 0.0557	0.8655 ± 0.0590	0.8693 ± 0.0577	0.8792 ± 0.0556	0.8721 ± 0.0589	**0.8895 ± 0.0481**

Table 15
F1-score results of different oversampling methods under the KNN classifier.

Dataset	SMOTE	SL-SMOTE	B-SMOTE	ADASYN	G-SMOTE	MWMOTE	ASN-SMOTE	ISMOTE	VS-SMOTE	RAN-SMOTE
D1	0.0114 ± 0.0560	0.0244 ± 0.0278	0.0089 ± 0.0435	0.0473 ± 0.0245	0.0476 ± 0.0453	0.0281 ± 0.0421	0.0301 ± 0.0366	0.0412 ± 0.0227	0.0000 ± 0.0000	**0.0517 ± 0.0344**
D2	**1.0000 ± 0.0000**	0.9600 ± 0.0533	0.9933 ± 0.0264	0.9756 ± 0.0460	0.9689 ± 0.0499	0.9653 ± 0.0580	0.9431 ± 0.0616	0.9698 ± 0.0562	0.9653 ± 0.0580	0.9756 ± 0.0460
D3	0.6873 ± 0.2098	0.6172 ± 0.1411	0.6637 ± 0.1677	0.6395 ± 0.1399	0.6680 ± 0.1474	0.6214 ± 0.1935	0.6125 ± 0.1138	0.6596 ± 0.1684	0.6760 ± 0.1819	**0.7366 ± 0.1562**
D4	0.7149 ± 0.1854	0.6162 ± 0.2002	0.6866 ± 0.1648	0.6424 ± 0.1715	0.6773 ± 0.1851	0.6257 ± 0.1897	0.6243 ± 0.1868	0.6776 ± 0.1896	0.6784 ± 0.2172	**0.7359 ± 0.1626**
D5	0.7337 ± 0.1932	0.6362 ± 0.1723	0.6985 ± 0.1814	0.6115 ± 0.1307	0.6433 ± 0.1368	0.6666 ± 0.1742	0.6825 ± 0.1918	0.6864 ± 0.1656	0.6833 ± 0.1852	**0.8179 ± 0.1451**
D6	0.9535 ± 0.0441	0.9554 ± 0.0467	0.9609 ± 0.0366	0.9391 ± 0.0395	0.9567 ± 0.0389	0.9540 ± 0.0336	0.9530 ± 0.0394	0.9608 ± 0.0431	0.9579 ± 0.0472	**0.9865 ± 0.0191**
D7	0.7738 ± 0.1012	0.7783 ± 0.0836	0.7686 ± 0.0740	0.7583 ± 0.0583	0.7759 ± 0.0644	0.7606 ± 0.0516	0.7928 ± 0.0966	0.7724 ± 0.0858	0.7750 ± 0.0950	**0.7929 ± 0.0720**
D8	0.5998 ± 0.0908	0.5847 ± 0.0977	0.6159 ± 0.1185	0.5694 ± 0.0972	0.5894 ± 0.1026	0.5733 ± 0.0764	**0.6617 ± 0.1045**	0.6067 ± 0.0774	0.6187 ± 0.0730	0.6570 ± 0.0989
D9	0.8498 ± 0.0725	0.8600 ± 0.0710	0.8648 ± 0.0635	0.8465 ± 0.0714	0.8618 ± 0.0756	0.8337 ± 0.0916	0.8492 ± 0.0839	0.8492 ± 0.0891	0.8516 ± 0.1099	**0.9120 ± 0.0647**
D10	0.8240 ± 0.2472	0.7110 ± 0.2314	0.7742 ± 0.2543	0.7955 ± 0.2510	0.8063 ± 0.2089	0.7909 ± 0.2517	0.6742 ± 0.1806	0.7587 ± 0.2929	0.8455 ± 0.1899	**0.9000 ± 0.1461**
D11	0.7121 ± 0.0655	0.7001 ± 0.0572	0.7080 ± 0.0642	0.7141 ± 0.0536	**0.7195 ± 0.0574**	0.7117 ± 0.0578	0.7137 ± 0.0731	0.7065 ± 0.0521	0.7147 ± 0.0874	0.7072 ± 0.0620
D12	0.6874 ± 0.0973	0.6755 ± 0.0822	0.6705 ± 0.0834	0.6876 ± 0.0921	0.6788 ± 0.0671	0.6884 ± 0.0601	0.6788 ± 0.1055	0.6842 ± 0.0960	0.6751 ± 0.0999	**0.7431 ± 0.0755**
D13	0.6573 ± 0.2631	0.6304 ± 0.1119	0.7050 ± 0.1972	0.6481 ± 0.1561	0.6511 ± 0.1814	0.6152 ± 0.2513	0.7028 ± 0.1877	0.6233 ± 0.1591	0.6569 ± 0.1415	**0.7276 ± 0.1624**
D14	0.3726 ± 0.0788	0.4283 ± 0.0776	0.4101 ± 0.0813	0.4227 ± 0.0831	0.4148 ± 0.0793	0.3938 ± 0.0714	0.3813 ± 0.0640	0.4234 ± 0.0815	0.4093 ± 0.0900	**0.4369 ± 0.0669**
D15	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
D16	0.3680 ± 0.1818	0.3166 ± 0.1559	0.3822 ± 0.1932	0.3793 ± 0.1882	0.4041 ± 0.1919	0.3133 ± 0.1489	0.3542 ± 0.1870	0.3858 ± 0.1627	0.3898 ± 0.2044	**0.7885 ± 0.0840**
D17	0.9313 ± 0.0818	0.9133 ± 0.0909	0.9273 ± 0.0794	0.9359 ± 0.0647	**0.9537 ± 0.0521**	0.9206 ± 0.0763	0.9261 ± 0.0785	0.9333 ± 0.0878	0.9338 ± 0.0913	0.9280 ± 0.0663
D18	0.6327 ± 0.0317	**0.6357 ± 0.0320**	0.6311 ± 0.0378	0.6309 ± 0.0381	0.6295 ± 0.0478	0.6339 ± 0.0449	0.6252 ± 0.0403	0.6313 ± 0.0415	0.6290 ± 0.0460	0.6190 ± 0.0422
D19	**0.7056 ± 0.1727**	0.2037 ± 0.0723	0.6200 ± 0.1740	0.5666 ± 0.0883	0.6653 ± 0.1210	0.6689 ± 0.1452	0.4226 ± 0.1125	0.5096 ± 0.0698	0.5852 ± 0.1003	0.1665 ± 0.0793
D20	0.9743 ± 0.0111	0.9670 ± 0.0137	**0.9781 ± 0.0149**	0.9755 ± 0.0151	0.9740 ± 0.0142	0.9733 ± 0.0142	0.9658 ± 0.0139	0.9737 ± 0.0140	0.9728 ± 0.0127	0.9668 ± 0.0179
D21	0.9933 ± 0.0098	0.9933 ± 0.0098	0.9938 ± 0.0104	0.9934 ± 0.0112	0.9938 ± 0.0111	0.9933 ± 0.0098	0.9925 ± 0.0115	0.9854 ± 0.0202	0.9925 ± 0.0115	**0.9958 ± 0.0084**
D22	**0.8597 ± 0.0366**	0.8356 ± 0.0335	0.8362 ± 0.0343	0.8303 ± 0.0353	0.8499 ± 0.0335	0.8451 ± 0.0276	0.8349 ± 0.0301	0.8490 ± 0.0324	0.8433 ± 0.0270	0.8570 ± 0.0303
D23	0.9570 ± 0.0202	0.9548 ± 0.0181	0.9524 ± 0.0213	0.9548 ± 0.0207	0.9559 ± 0.0230	0.9559 ± 0.0213	0.9539 ± 0.0220	0.9516 ± 0.0193	0.9564 ± 0.0167	**0.9662 ± 0.0194**
D24	0.4488 ± 0.1469	0.3895 ± 0.0675	0.4530 ± 0.0860	0.3762 ± 0.0924	0.4139 ± 0.0980	0.4061 ± 0.0687	0.4070 ± 0.1063	0.4190 ± 0.0670	0.4478 ± 0.0994	**0.5646 ± 0.1013**
D25	0.1996 ± 0.1877	0.1337 ± 0.0597	0.2355 ± 0.1281	0.1547 ± 0.0502	0.1736 ± 0.0794	0.1718 ± 0.0818	0.2365 ± 0.1649	0.1620 ± 0.0556	**0.2837 ± 0.1446**	0.2182 ± 0.0861
D26	0.1143 ± 0.1484	0.1311 ± 0.0579	0.1523 ± 0.0799	0.1579 ± 0.0705	0.1795 ± 0.0847	0.1787 ± 0.0991	0.0385 ± 0.0779	0.1713 ± 0.0790	0.1768 ± 0.1133	**0.1812 ± 0.0760**
D27	**0.4025 ± 0.1536**	0.2479 ± 0.0742	0.3099 ± 0.1242	0.2750 ± 0.0853	0.3274 ± 0.1002	0.3216 ± 0.1135	0.3454 ± 0.1292	0.3024 ± 0.0956	0.3378 ± 0.1429	0.3460 ± 0.0916
D28	0.7338 ± 0.0539	0.6451 ± 0.0481	0.6908 ± 0.0414	0.6492 ± 0.0429	0.6830 ± 0.0411	0.6710 ± 0.0457	0.7103 ± 0.0477	0.6747 ± 0.0651	0.7007 ± 0.0453	**0.7407 ± 0.0448**
D29	0.4006 ± 0.1064	0.2502 ± 0.0699	0.3474 ± 0.1060	0.2919 ± 0.0651	0.2976 ± 0.0922	0.3040 ± 0.0923	0.3478 ± 0.1309	0.2909 ± 0.0542	0.3621 ± 0.1141	**0.4140 ± 0.0694**
D30	0.6932 ± 0.0792	0.5828 ± 0.0704	0.5958 ± 0.0728	0.5996 ± 0.0663	0.6078 ± 0.0868	0.5954 ± 0.0773	0.5786 ± 0.0977	0.5855 ± 0.0572	0.5779 ± 0.0748	**0.6839 ± 0.0881**
D31	**0.5270 ± 0.1169**	0.3487 ± 0.0585	0.4716 ± 0.1056	0.2891 ± 0.0514	0.3532 ± 0.0781	0.3985 ± 0.0902	0.5245 ± 0.1257	0.3243 ± 0.0601	0.4459 ± 0.0872	0.4956 ± 0.0977
Average	0.6619 ± 0.1046	0.6041 ± 0.0770	0.6486 ± 0.0925	0.6244 ± 0.0774	0.6426 ± 0.0837	0.6316 ± 0.0890	0.6311 ± 0.0936	0.6313 ± 0.0826	0.6498 ± 0.0938	**0.6811 ± 0.0747**

Table 16
Performance comparison between SMOTE and RAN-SMOTE under different imbalance levels.

Imbalance Level	Metric	CART			LR			KNN		
		SMOTE	Our	Gain	SMOTE	Our	Gain	SMOTE	Our	Gain
Extremely Imbalance ($IR > 100$)	G-mean	0.0704	0.4133	487.1%	0.0000	0.7451	–	0.0163	0.4537	2683.4%
	AUC	0.5099	0.5899	15.7%	0.8196	0.8586	4.8%	0.5508	0.6632	20.4%
	F1	0.0225	0.0898	299.1%	0.0000	0.0577	–	0.0114	0.0517	353.5%
Highly Imbalance ($20 < IR \leq 100$)	G-mean	0.5006	0.6400	27.8%	0.3503	0.7461	113.0%	0.5912	0.7326	23.9%
	AUC	0.6518	0.7316	12.2%	0.7736	0.8006	3.5%	0.8163	0.8117	−0.6%
	F1	0.3012	0.4233	40.5%	0.2519	0.2348	−6.8%	0.4401	0.3599	−18.2%
Moderately Imbalance ($5 < IR \leq 20$)	G-mean	0.8151	0.8660	6.2%	0.8253	0.9024	9.3%	0.8087	0.9010	11.4%
	AUC	0.8462	0.8828	4.3%	0.9285	0.9440	1.7%	0.9015	0.9294	3.1%
	F1	0.7089	0.7751	9.3%	0.7419	0.7287	−1.8%	0.7192	0.7772	8.1%
Low Imbalance ($IR \leq 5$)	G-mean	0.8179	0.8384	2.5%	0.8263	0.8493	2.8%	0.8396	0.8633	2.8%
	AUC	0.8251	0.8437	2.3%	0.8864	0.8895	0.3%	0.8928	0.9031	1.2%
	F1	0.7559	0.7789	3.0%	0.7735	0.7917	2.4%	0.7797	0.8021	2.9%



(a) G-mean (b) AUC (c) F1-score

Fig. 7. Classification performance at different label noise rates.

Fig. 7 shows the G-mean, AUC, and F1-score values at different noise rates. For each method and noise rate, these values are averaged over the 15 combinations of five datasets and three classifiers.

Overall, the three evaluation metrics generally decrease as the label noise rate increases, indicating that label noise adversely affects classification performance in the oversampling experiments. For G-mean, RAN-SMOTE achieves the highest average values at noise rates of 0%, 5%, and 10%, and the second highest value at 15%. For AUC, RAN-SMOTE maintains the highest average values at noise rates from 0% to 15% and ranks third at 20%. For F1-score, RAN-SMOTE achieves the highest values at 0% and 5%, but performs worse than ASN-SMOTE and VS-SMOTE at the remaining three noise rates. These results indicate that higher label noise rates pose a greater challenge to minority class recognition.

To further assess performance stability across different dataset and classifier combinations, Fig. 8 shows the distributions of relative performance drops as the label noise rate increases from 0% to 10%. The relative performance drop is defined as

$$RD = \frac{S_0 - S_{10}}{S_0} \times 100\%, \tag{29}$$

where S_0 and S_{10} denote the values of the same evaluation metric at noise rates of 0% and 10%, respectively, with the method, dataset, and classifier held fixed. A smaller relative drop indicates better performance retention under the evaluated label noise condition.

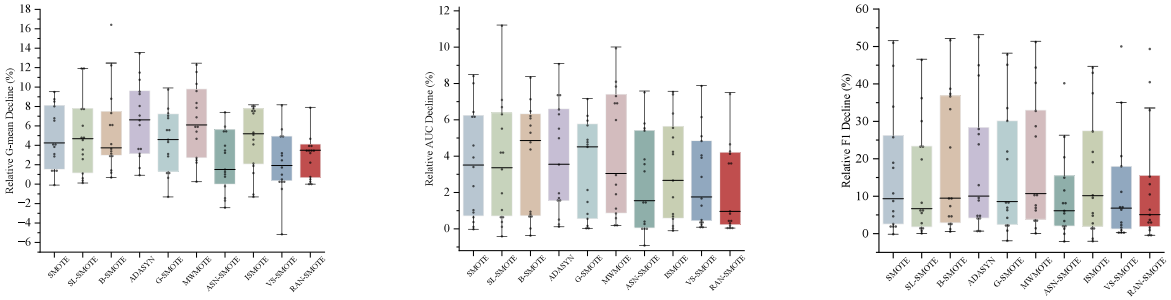
For G-mean, RAN-SMOTE does not achieve the lowest median relative drop, but its narrow interquartile range indicates relatively consistent performance degradation. For AUC, RAN-SMOTE achieves a lower median relative drop than the competing methods, with a relatively concentrated distribution. For F1-score, RAN-SMOTE also exhibits a relatively low median drop, although larger decreases occur in some combinations, indicating that its ability to retain performance varies across dataset and classifier combinations.

Considering the results at different noise rates, RAN-SMOTE maintains favorable G-mean and AUC performance at noise rates of 5% and 10%. This may be attributed to posterior misclassification rate estimation, which helps identify and filter potentially noisy samples before sample generation, and adaptive neighbor selection, which reduces the participation of unreliable samples in interpolation. Higher noise rates may make misclassification rate estimation and reliable neighbor selection more difficult, leading to performance deterioration. Overall, the experimental results support the robustness of RAN-SMOTE to low-to-moderate label noise, while indicating room for improvement under higher label noise levels.

4.2.4. *Runtime comparison*

To evaluate the practical computational cost, we compare the oversampling runtime of RAN-SMOTE with that of the baseline methods on the 31 datasets. During posterior misclassification rate estimation in RAN-SMOTE, the training and prediction tasks for different cross-validation folds and heterogeneous base learners within the same repetition can be executed independently. These tasks can therefore be parallelized using multiple processes to reduce elapsed runtime. Each method is executed five times on each training fold, and the runtimes are averaged across repetitions and folds for each dataset.

Table 17 reports the mean runtime of each baseline method and of RAN-SMOTE under serial and six-process configurations. Methods such as SMOTE, ADASYN, and G-SMOTE mainly rely on nearest neighbor searches and relatively simple sample generation operations, resulting in shorter runtimes. VS-SMOTE additionally evaluates candidate generation spaces and selects high-value regions, introducing extra computational overhead. In comparison, RAN-SMOTE requires repeated cross-validation and the training and prediction of multiple heterogeneous base learners for posterior misclassification rate estimation, leading to higher computational costs. Parallel execution with six worker pro-



(a) G-mean (b) AUC (c) F1-score

Fig. 8. Distributions of relative performance drops as the label noise rate increases from 0% to 10%.

Table 17
Mean oversampling runtime on 31 datasets (seconds).

NO.	SMOTE	SL-SMOTE	B-SMOTE	ADASYN	G-SMOTE	MWMOTE	ASN-SMOTE	ISMOTE	VS-SMOTE	RAN-SMOTE (serial)	RAN-SMOTE (6 processes)	Time reduction (%)
D1	0.0085	0.0553	0.0092	0.0135	0.0040	0.0379	0.0733	0.0047	0.0472	530.6625	81.5990	84.62
D2	0.0033	0.0035	0.0033	0.0041	0.0027	0.0115	0.0108	0.0027	0.0120	64.9175	10.5124	83.81
D3	0.0032	0.0027	0.0038	0.0036	0.0028	0.0162	0.0087	0.0032	0.0082	249.3379	39.8216	84.03
D4	0.0031	0.0028	0.0039	0.0036	0.0029	0.0163	0.0094	0.0029	0.0087	240.3155	38.3583	84.04
D5	0.0029	0.0028	0.0032	0.0036	0.0026	0.0132	0.0092	0.0028	0.0081	232.4356	37.4080	83.91
D6	0.0031	0.0028	0.0047	0.0036	0.0028	0.0181	0.0206	0.0024	0.0114	53.2583	7.4072	86.09
D7	0.0032	0.0027	0.0048	0.0041	0.0028	0.0469	0.0171	0.0029	0.0146	187.7303	26.3410	85.97
D8	0.0033	0.0030	0.0043	0.0039	0.0027	0.0202	0.0119	0.0029	0.0105	196.5903	28.4321	85.54
D9	0.0028	0.0024	0.0036	0.0033	0.0026	0.0130	0.0112	0.0027	0.0085	247.6455	46.3945	81.27
D10	0.0027	0.0021	0.0029	0.0029	0.0019	0.0075	0.0049	0.0021	0.0046	89.4873	20.7675	76.79
D11	0.0029	0.0023	0.0034	0.0034	0.0025	0.0364	0.0109	0.0024	0.0088	220.7445	44.1913	79.98
D12	0.0027	0.0024	0.0036	0.0034	0.0027	0.0396	0.0088	0.0027	0.0081	239.2232	49.4334	79.34
D13	0.0024	0.0026	0.0028	0.0026	0.0020	0.0082	0.0060	0.0026	0.0057	101.6573	23.1933	77.18
D14	0.0024	0.0022	0.0033	0.0028	0.0024	0.0699	0.0081	0.0025	0.0099	69.5763	14.8924	78.60
D15	0.0026	0.0021	0.0025	0.0022	0.0021	0.0059	0.0114	0.0023	0.0060	33.4226	7.1393	78.64
D16	0.0031	0.0031	0.0036	0.0031	0.0023	0.0147	0.0101	0.0025	0.0083	88.3806	18.8613	78.66
D17	0.0024	0.0023	0.0034	0.0030	0.0025	0.0114	0.0089	0.0025	0.0067	57.5422	12.2127	78.78
D18	0.0032	0.0035	0.0059	0.0049	0.0027	0.3866	0.0283	0.0029	0.0635	62.5441	13.2698	78.78
D19	0.0041	0.0091	0.0050	0.0048	0.0023	0.0207	0.0179	0.0026	0.0269	206.2425	49.8033	75.85
D20	0.0043	0.0115	0.0080	0.0066	0.0031	0.5270	0.0495	0.0028	0.0907	73.2596	15.6326	78.66
D21	0.0042	0.0124	0.0042	0.0063	0.0028	0.0326	0.0360	0.0024	0.0296	37.9283	8.2796	78.17
D22	0.0028	0.0067	0.0059	0.0048	0.0024	0.1980	0.0214	0.0024	0.0314	61.2349	13.3328	78.23
D23	0.0031	0.0039	0.0073	0.0049	0.0030	0.0296	0.0396	0.0025	0.0310	37.1981	8.4052	77.40
D24	0.0029	0.0031	0.0034	0.0033	0.0022	0.0285	0.0087	0.0021	0.0097	179.7981	39.5464	78.01
D25	0.0034	0.0056	0.0040	0.0039	0.0028	0.0194	0.0104	0.0021	0.0137	218.9415	47.9079	78.12
D26	0.0030	0.0043	0.0038	0.0036	0.0025	0.0154	0.0078	0.0022	0.0105	154.7573	33.8367	78.14
D27	0.0030	0.0031	0.0030	0.0035	0.0023	0.0178	0.0070	0.0022	0.0083	181.0802	39.6964	78.08
D28	0.0037	0.0097	0.0069	0.0059	0.0030	0.2061	0.0316	0.0052	0.0316	64.8639	14.0800	78.29
D29	0.0039	0.0100	0.0047	0.0047	0.0024	0.0396	0.0163	0.0022	0.0207	167.4134	36.5709	78.16
D30	0.0041	0.0097	0.0048	0.0048	0.0025	0.0336	0.0184	0.0022	0.0199	161.0846	35.2527	78.12
D31	0.0036	0.0097	0.0049	0.0051	0.0025	0.0262	0.0160	0.0022	0.0194	148.8452	32.5372	78.14

cesses reduces the overall mean runtime of RAN-SMOTE by approximately 80%, although it remains higher than that of the other compared methods. Together with the classification performance and robustness results, these findings indicate that the performance improvements achieved by RAN-SMOTE come at an additional computational cost.

4.2.5. *Visualization analysis of synthetic sample distribution*

To more intuitively evaluate the quality of synthetic samples, three datasets with different degrees of imbalance are selected: wisconsin ($IR = 1.86$), yeast6 ($IR = 41.4$), and abalone19 ($IR = 129.44$). PCA is applied to reduce the data features to two dimensions, and synthetic samples are generated on the reduced data using different oversampling methods. Figs. 9–14 show the two-dimensional sample distributions generated by different oversampling methods, with red, blue, and green points denoting majority, minority, and synthetic samples, respectively.

The visualization results on the wisconsin dataset are shown in Figs. 9-10. The two classes exhibit a certain degree of overlap. Most oversampling methods generate samples in relatively safe minority regions, which contribute little to improving boundary discrimination. Some methods focus excessively on overlapping regions and may generate samples biased toward the majority class under noise interference. In contrast, RAN-SMOTE generates samples mainly in minority boundary regions while avoiding excessive deviation toward majority class regions, thereby preserving local minority characteristics and improving minority class recognition.

The visualization results on the yeast6 dataset are shown in Figs. 11-12. This dataset is highly imbalanced, with sparse minority samples and relatively dense majority samples. As shown in the figure, SMOTE, G-SMOTE, and ISMOTE tend to generate samples toward the majority class region on the left under the influence of minority class noise. ADASYN, Borderline-SMOTE, and MWMOTE focus on boundary minority samples, but some generated samples still fall into overlap-

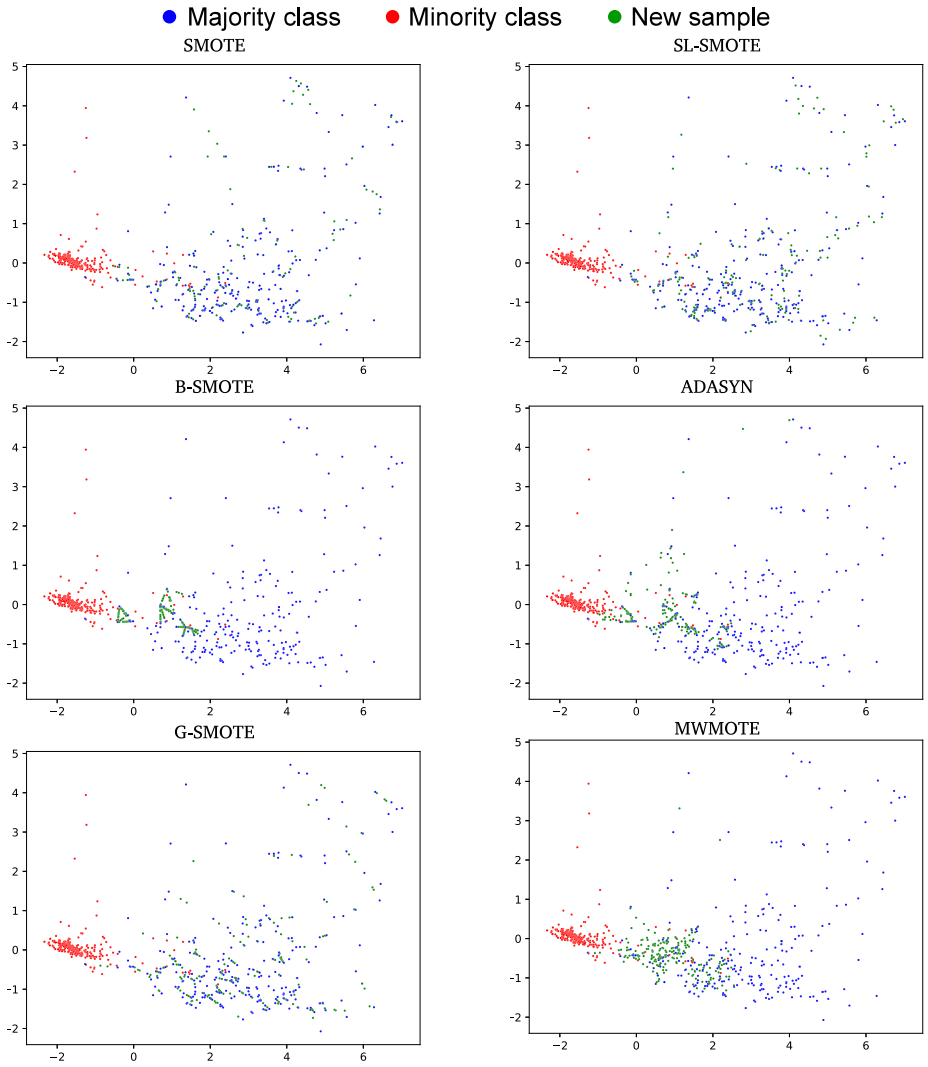


Fig. 9. Visualization of synthetic samples on the wisconsin dataset.

ping regions with limited diversity. Safe-Level-SMOTE, ASN-SMOTE, and VS-SMOTE generate samples mainly in safe minority regions, reducing noisy samples but providing limited support for critical boundary regions. In contrast, RAN-SMOTE generates samples in key boundary regions while suppressing noisy and cross-class synthesis, thereby enhancing minority class boundaries and maintaining better sample diversity.

The visualization results on the abalone19 dataset are shown in Figs. 13-14. This dataset contains very few minority samples, and its PCA-reduced distribution shows a band-like structure. SMOTE, ADASYN, ISMOTE, and G-SMOTE generate cross-cluster samples, causing scattered synthetic distributions that deviate from the original data. Safe-Level-SMOTE, Borderline-SMOTE, MWMOTE, ASN-SMOTE, and VS-SMOTE are more conservative, but some generated samples highly overlap with the original minority samples. In contrast, RAN-SMOTE generates samples along the minority distribution, reducing cross-cluster synthesis and producing a more reasonable synthetic distribution.

From the visualization results in Figs. 9–14, it can be concluded that RAN-SMOTE can generate more reasonable samples under different imbalance scenarios. It not only enhances the minority class boundary but also improves the diversity of synthetic samples, while reducing noisy samples and cross-cluster synthesis. These results further verify the effectiveness and robustness of the proposed method in improving classification performance under imbalanced scenarios.

4.3. Statistical test analysis

To verify the statistical reliability of the differences among different methods, this paper adopts the Friedman rank test, the Nemenyi test, and the Holm post-hoc test for analysis.

The Friedman test is a nonparametric statistical test used to compare multiple algorithms across multiple datasets. It is used to determine whether there are significant overall differences among the methods. Suppose there are N datasets and K algorithms. The average rank of the jth algorithm is defined as:

$$R_j = \frac{1}{N}\sum_{i=1}^{N} r_{ij}, \qquad j = 1, 2, \ldots, K. \tag{30}$$

Here, r_{ij} denotes the rank of the jth algorithm on the ith dataset. The Friedman test statistic is defined as:

$$\chi_F^2 = \frac{12N}{K(K+1)}\left(\sum_{j=1}^{K} R_j^2 - \frac{K(K+1)^2}{4}\right). \tag{31}$$

The χ_F^2 values calculated using the CART classifier are 96.83, 86.09, and 95.63 for G-mean, AUC, and F1-score, respectively. The χ_F^2 values calculated using the LR classifier are 83.71, 55.41, and 36.48 for

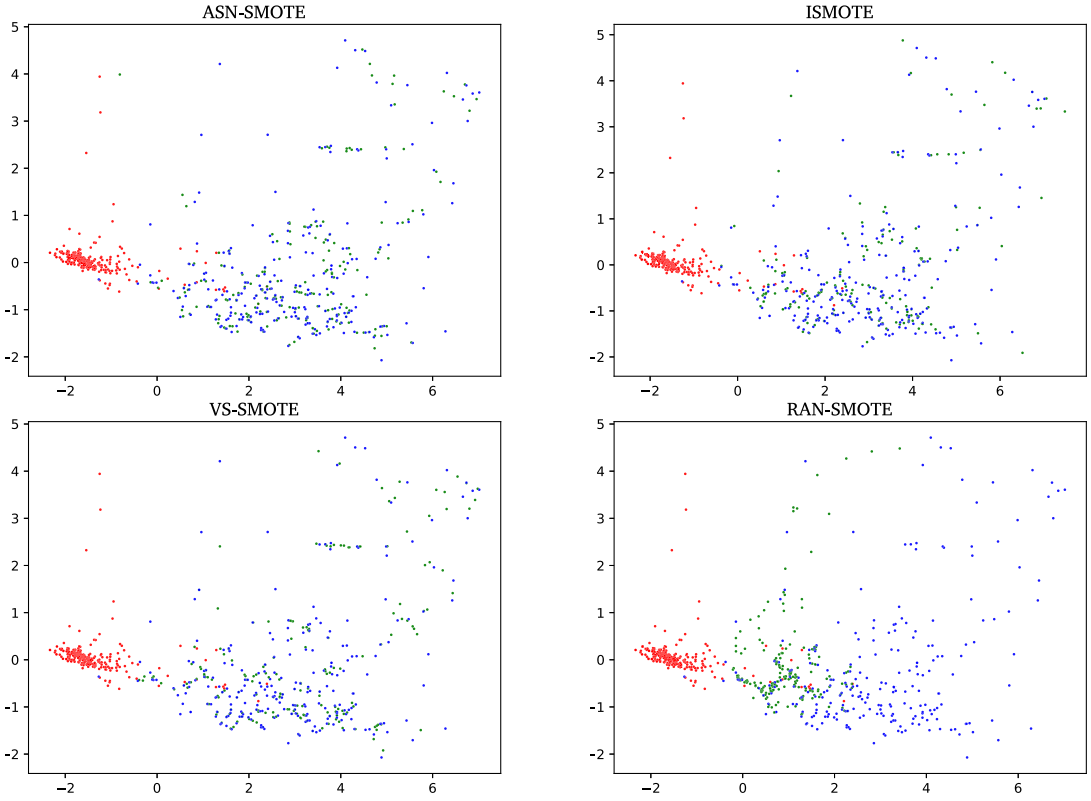


Fig. 10. Visualization of synthetic samples on the wisconsin dataset continued.

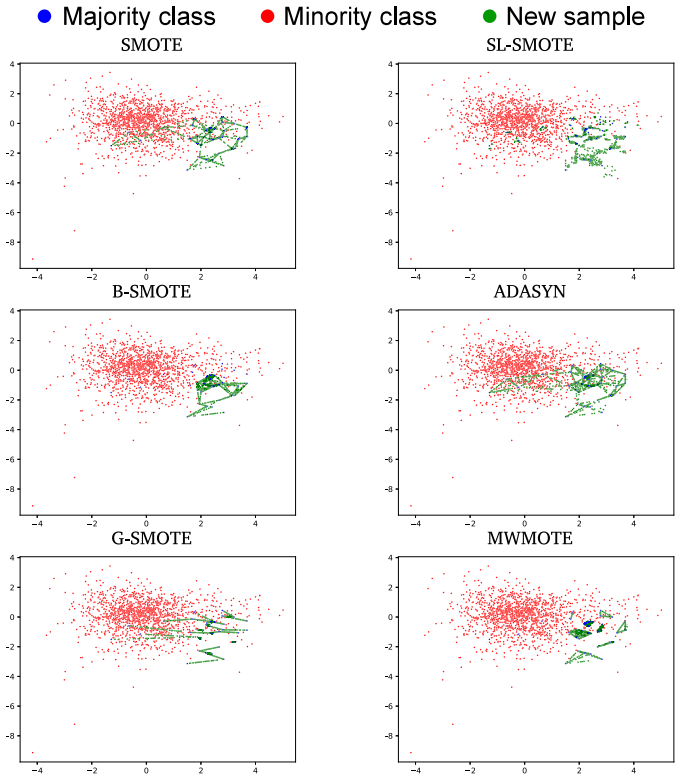


Fig. 11. Visualization of synthetic samples on the yeast6 dataset.

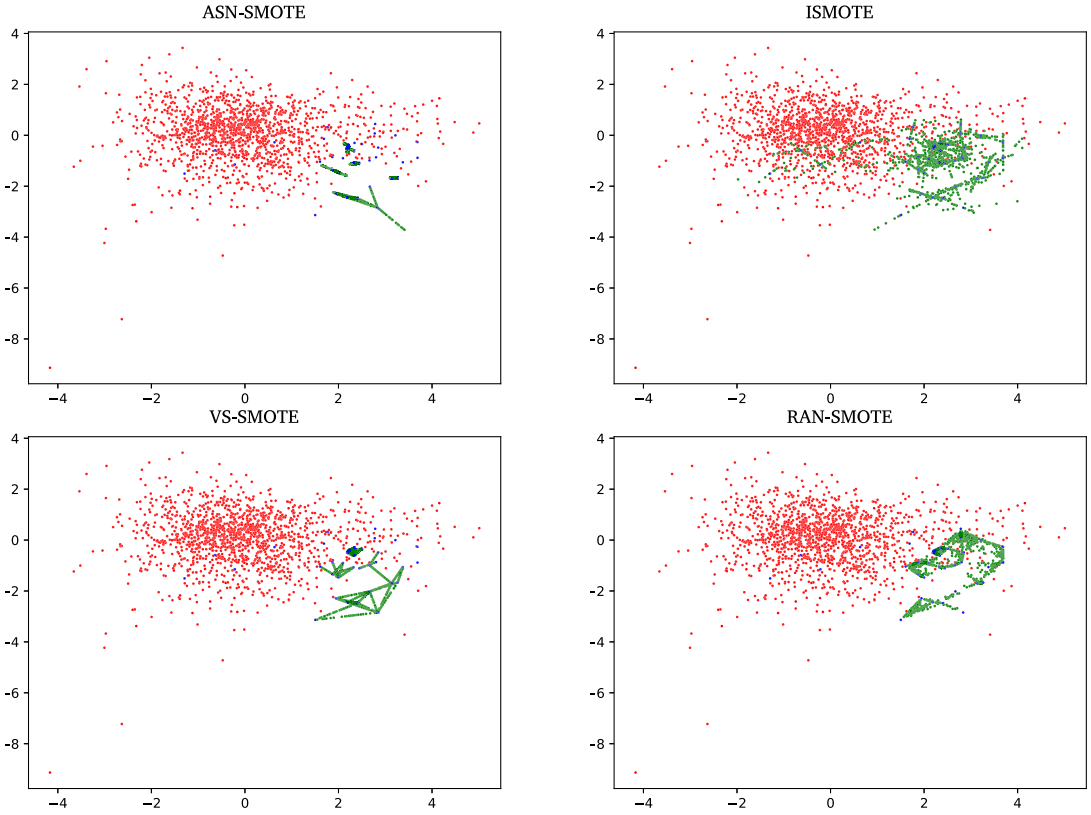


Fig. 12. Visualization of synthetic samples on the yeast6 dataset continued.

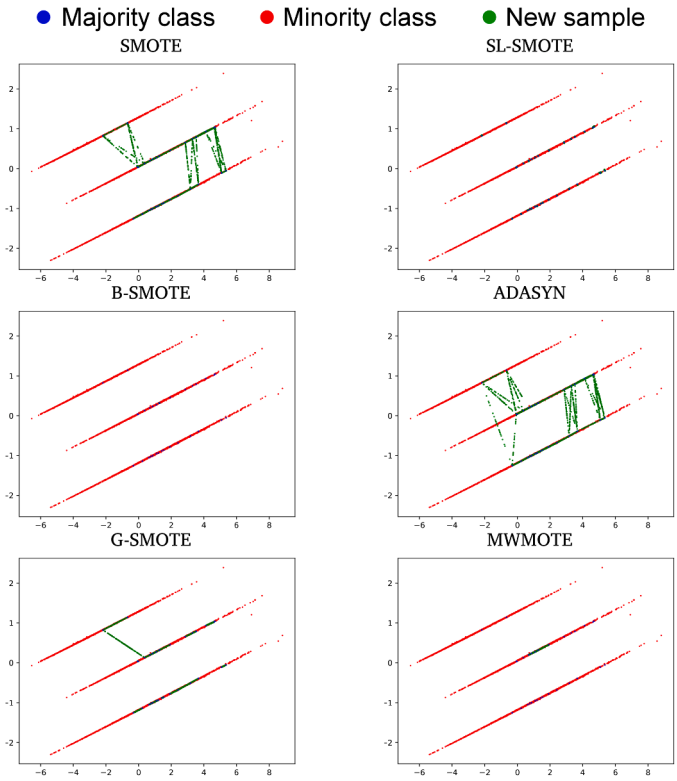


Fig. 13. Visualization of synthetic samples on the abalone19 dataset.

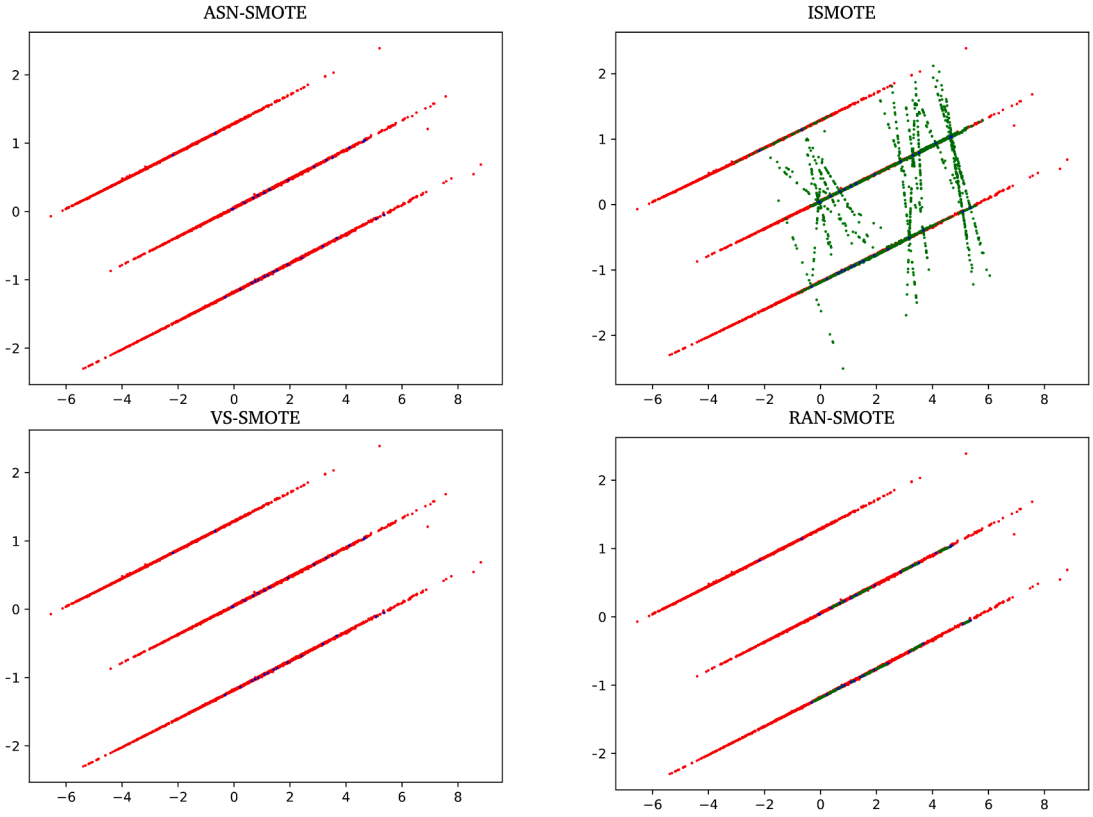


Fig. 14. Visualization of synthetic samples on the abalone19 dataset continued.

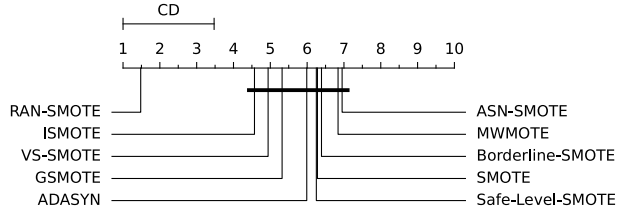


Fig. 15. CD diagram of the CART results.

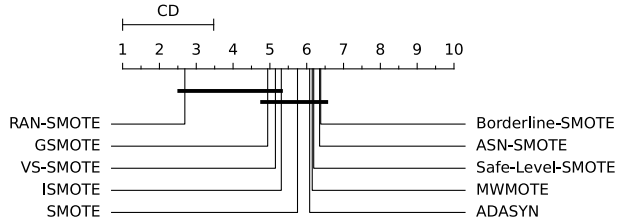


Fig. 16. CD diagram of the LR results.

G-mean, AUC, and F1-score, respectively. The χ_F^2 values calculated using the KNN classifier are 84.29, 32.83, and 70.20 for G-mean, AUC, and F1-score, respectively. All these values are greater than the chi-square critical value of 16.92 for the Friedman test at the significance level of $\alpha = 0.05$ with degrees of freedom $df = K - 1 = 9$, indicating that there are significant performance differences among the compared algorithms.

The Nemenyi test is further used to calculate the critical difference (CD), which is defined as follows:

$$CD = q_\alpha \sqrt{\frac{K(K+1)}{6N}}. \tag{32}$$

When the significance level is $\alpha = 0.05$, the number of algorithms is $K = 10$, and the number of datasets is $N = 31$, we have $q_\alpha = 3.164$, and the calculated critical difference is $CD = 2.43$. If the difference between the average ranks of two algorithms exceeds the critical difference, their performance difference is considered statistically significant.Figs. 15–17 show the CD diagrams of the average ranks corresponding to the CART, LR, and KNN classifiers at the significance level of $\alpha = 0.05$.

In these figures, a smaller average rank indicates better performance, and algorithms connected by a horizontal line show no statistically significant difference. Overall, RAN-SMOTE outperforms the compared methods across classifiers, with a more evident advantage under CART.

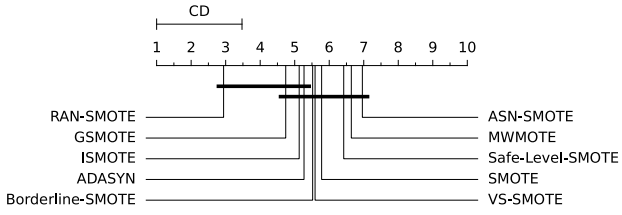


Fig. 17. CD diagram of the KNN results.

Although its differences from some methods under LR and KNN are not statistically significant, it still achieves stronger overall performance than traditional oversampling methods.

The Holm post-hoc test is then used to compare whether there are significant differences between RAN-SMOTE and the other methods. The statistic corresponding to the difference in average ranks between two methods is defined as:

$$z_j = \frac{\left|R_j - R_c\right|}{\sqrt{\frac{K(K+1)}{6N}}}. \tag{33}$$

Table 18
Holm post-hoc test results of RAN-SMOTE under different classifiers and evaluation metrics.

Algorithm	G-mean			AUC			F1-score		
	Rank	*p*-value	Decision	Rank	*p*-value	Decision	Rank	*p*-value	Decision
CART									
RAN-SMOTE	1.58	–	–	1.50	–	–	1.35	–	–
SMOTE	6.87	4.21E-11	Rejected	6.55	3.13E-10	Rejected	5.42	3.55E-07	Rejected
SL-SMOTE	5.52	1.24E-06	Rejected	5.58	4.48E-07	Rejected	7.66	2.00E-15	Rejected
B-SMOTE	6.66	2.36E-10	Rejected	6.65	1.56E-10	Rejected	5.85	2.43E-08	Rejected
ADASYN	5.89	1.07E-07	Rejected	5.97	3.13E-08	Rejected	6.11	3.68E-09	Rejected
G-SMOTE	5.24	4.69E-06	Rejected	5.26	3.07E-06	Rejected	5.45	3.55E-07	Rejected
MWMOTE	7.19	2.61E-12	Rejected	7.16	1.63E-12	Rejected	6.18	2.51E-09	Rejected
ASN-SMOTE	7.00	1.46E-11	Rejected	6.71	9.99E-11	Rejected	7.13	4.78E-13	Rejected
ISMOTE	3.77	4.34E-03	Rejected	4.47	1.14E-04	Rejected	5.47	3.55E-07	Rejected
VS-SMOTE	5.27	4.69E-06	Rejected	5.16	3.85E-06	Rejected	4.37	8.78E-05	Rejected
LR									
RAN-SMOTE	1.90	–	–	2.76	–	–	3.40	–	–
SMOTE	7.97	2.80E-14	Rejected	4.52	3.56E-02	Rejected	4.77	1.49E-01	Not rejected
SL-SMOTE	5.52	1.05E-05	Rejected	6.21	4.55E-05	Rejected	6.85	6.46E-05	Rejected
B-SMOTE	6.40	3.41E-08	Rejected	7.26	4.38E-08	Rejected	5.47	3.63E-02	Rejected
ADASYN	5.27	3.50E-05	Rejected	6.21	4.55E-05	Rejected	6.76	1.03E-04	Rejected
G-SMOTE	4.61	4.46E-04	Rejected	5.02	9.97E-03	Rejected	5.19	5.97E-02	Not rejected
MWMOTE	5.66	5.12E-06	Rejected	6.40	1.71E-05	Rejected	6.39	7.31E-04	Rejected
ASN-SMOTE	6.77	1.91E-09	Rejected	6.23	4.55E-05	Rejected	6.06	3.23E-03	Rejected
ISMOTE	4.74	4.46E-04	Rejected	5.82	2.70E-04	Rejected	5.37	4.20E-02	Rejected
VS-SMOTE	6.15	2.08E-07	Rejected	4.58	3.56E-02	Rejected	4.73	1.49E-01	Not rejected
KNN									
RAN-SMOTE	2.85	–	–	3.23	–	–	2.74	–	–
SMOTE	7.77	1.27E-09	Rejected	5.55	1.01E-02	Rejected	4.02	9.75E-02	Not rejected
SL-SMOTE	5.89	3.37E-04	Rejected	5.71	7.43E-03	Rejected	7.68	1.24E-09	Rejected
B-SMOTE	5.97	3.10E-04	Rejected	5.63	8.89E-03	Rejected	4.97	1.24E-02	Rejected
ADASYN	3.95	1.79E-01	Not rejected	5.27	1.55E-02	Rejected	6.60	3.76E-06	Rejected
G-SMOTE	4.16	1.79E-01	Not rejected	5.47	1.07E-02	Rejected	4.58	3.36E-02	Rejected
MWMOTE	6.23	8.18E-05	Rejected	7.13	3.48E-06	Rejected	6.56	4.00E-06	Rejected
ASN-SMOTE	7.84	8.21E-10	Rejected	6.39	3.15E-04	Rejected	6.65	3.09E-06	Rejected
ISMOTE	4.42	1.26E-01	Not rejected	4.79	4.19E-02	Rejected	6.19	3.59E-05	Rejected
VS-SMOTE	5.92	3.37E-04	Rejected	5.84	4.76E-03	Rejected	5.02	1.24E-02	Rejected

Table 18 presents the experimental results at the significance level of $p < 0.05$. The results show that RAN-SMOTE achieves the best ranking for all metrics under the CART, LR, and KNN classifiers, indicating the best overall performance among all methods.

When CART is used as the classifier, RAN-SMOTE rejects the null hypothesis in all cases, showing statistically significant differences. When LR is used as the classifier, significant differences are observed for all methods and metrics, except for the F1-score results of SMOTE, G-SMOTE, and VS-SMOTE. Under the KNN classifier, no significant differences are observed for the G-mean results of ADASYN, G-SMOTE, and ISMOTE, the AUC result of ISMOTE, and the F1-score result of SMOTE; all other comparisons show statistically significant differences. Overall, although RAN-SMOTE does not show a significant advantage over a few methods on certain metrics, especially ISMOTE, the proposed method still demonstrates better comprehensive performance than these methods.

5. Conclusion and future work

This paper proposes RAN-SMOTE to address the limitations of existing oversampling methods for imbalanced learning. By incorporating sample-level reliability estimation, adaptive neighborhood selection, and Beta distribution-guided triangular interpolation, the proposed method reduces the negative influence of noisy and unreliable minority samples, alleviates the limitation of fixed-size neighborhoods in complex and uneven local data distributions, and generates more diverse synthetic instances around informative minority samples near class boundaries. In this way, the proposed method enhances the representation of minority class boundary regions and helps classifiers learn a more reliable decision boundary. The experimental results demonstrate that our proposal achieves competitive performance across multiple evaluation metrics and effectively improves the classification of imbalanced data, confirming the effectiveness of the proposed method. Furthermore, statistical significance tests were performed, confirming that the approach provides meaningful improvements over most competing oversampling methods.

Despite these results, several limitations remain. Posterior misclassification rate estimation requires repeated cross-validation with multiple heterogeneous base learners, leading to additional computational costs. Although early stopping and parallel execution help reduce this cost, further improvements in efficiency are needed, particularly for large-scale and high-dimensional data. Future work will explore lightweight estimation strategies and further optimization of the parallel implementation. The label-noise experiments also show that the performance advantage of RAN-SMOTE diminishes at higher noise rates. Heavy label contamination may impair misclassification rate estimation and reliable neighbor selection. More robust probability estimation and noise identification will therefore be investigated. Finally, the current empirical evaluation focuses on binary imbalanced classification. Future work will implement and systematically evaluate the multi-class extension discussed in Section 3.5.

CRediT authorship contribution statement

Xianyou Zhong: Writing – original draft, Methodology, Formal analysis; **Yifeng Zheng:** Writing – review & editing, Supervision, Project administration, Funding acquisition, Conceptualization; **Wenjie Zhang:** Writing – review & editing, Supervision, Funding acquisition; **Baoya Wei:** Investigation, Data curation; **Weiqiang Liu:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is the results of the research project funded by Natural Science Foundation of China (Grant Nos. 62376114, 62372488), Natural Science Foundation of Zhangzhou (Grant No. ZZ2024J20), Natural Science Foundation of Fujian Province (Nos. 2026J001984, 2026J002035, 2025J01366).

References

Arafa, A., El-Fishawy, N., Badawy, M., & Radad, M. (2022). RN-SMOTE: Reduced noise SMOTE based on DBSCAN for enhancing imbalanced data classification. *Journal of King Saud University - Computer and Information Sciences*, *34*, 5059–5074. https://doi.org/10.1016/j.jksuci.2022.06.005

Barua, S., Islam, M. M., Yao, X., & Murase, K. (2014). MWMOTE–majority weighted minority oversampling technique for imbalanced data set learning. *IEEE Transactions on Knowledge and Data Engineering*, *26*, 405–425. https://doi.org/10.1109/TKDE.2012.232

Bunkhumpornpat, C., Sinapiromsaran, K., & Lursinsap, C. (2009). Safe-level-SMOTE: Safe-level-synthetic minority over-sampling technique for handling the class imbalanced problem. In *Advances in knowledge discovery and data mining* (pp. 475–482). Springer. https://doi.org/10.1007/978-3-642-01307-2_43

Cai, Q. N., Zhang, Z. L., Wu, Y. H., & Zhang, X. M. (2024). LSMOTE: A link-based synthetic minority oversampling technique for binary imbalanced datasets. *Neurocomputing*, *608*, 128372. https://doi.org/10.1016/j.neucom.2024.128372

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, *16*, 321–357. https://doi.org/10.1613/jair.953

Chen, H., Wei, J., Huang, H., Yuan, Y., & Wang, J. (2024a). Review of imbalanced fault diagnosis technology based on generative adversarial networks. *Journal of Computational Design and Engineering*, *11*, 99–124. https://doi.org/10.1093/jcde/qwae075

Chen, W., Yang, K., Yu, Z., Shi, Y., & Chen, C. L. P. (2024b). A survey on imbalanced learning: Latest research, applications and future directions. *Artificial Intelligence Review*, *57*, 137. https://doi.org/10.1007/s10462-024-10759-6

Dai, Q., Wang, L., Zhang, J., Ding, W., & Chen, L. (2025). GQEO: Nearest neighbor graph-based generalized quadrilateral element oversampling for class-imbalance problem. *Neural Networks*, *184*, 107107. https://doi.org/10.1016/j.neunet.2024.107107

Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2018). Credit card fraud detection: A realistic modeling and a novel learning strategy. *IEEE Transactions on Neural Networks and Learning Systems*, *29*, 3784–3797. https://doi.org/10.1109/TNNLS.2017.2736643

Douzas, G., & Bacao, F. (2019). Geometric SMOTE a geometrically enhanced drop-in replacement for SMOTE. *Information Sciences*, *501*, 118–135. https://doi.org/10.1016/j.ins.2019.06.007

Elreedy, D., & Atiya, A. F. (2019). A comprehensive analysis of synthetic minority oversampling technique (SMOTE) for handling class imbalance. *Information Sciences*, *505*, 32–64. https://doi.org/10.1016/j.ins.2019.07.070

Guan, H., Zhao, L., Dong, X., & Chen, C. (2023). Extended natural neighborhood for SMOTE and its variants in imbalanced classification. *Engineering Applications of Artificial Intelligence*, *124*, 106570. https://doi.org/10.1016/j.engappai.2023.106570

Guo, H., Li, Y., Shang, J., Gu, M., Huang, Y., & Gong, B. (2017). Learning from class-imbalanced data: Review of methods and applications. *Expert Systems with Applications*, *73*, 220–239. https://doi.org/10.1016/j.eswa.2016.12.035

Han, H., Wang, W. Y., & Mao, B. H. (2005). Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning. In *Advances in intelligent computing* (pp. 878–887). Springer. https://doi.org/10.1007/11538059_91

He, H., Bai, Y., Garcia, E. A., & Li, S. (2008). ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *Proceedings of the IEEE international joint conference on neural networks* (pp. 1322–1328). IEEE. https://doi.org/10.1109/IJCNN.2008.4633969

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, *21*, 1263–1284. https://doi.org/10.1109/TKDE.2008.239

Hida, G. S., & Nascimento, A. C. A. D. (2026). Overview of machine learning in class imbalance scenarios: Trends, challenges, and approaches. *Expert Systems with Applications*, *298*, 129592. https://doi.org/10.1016/j.eswa.2025.129592

Hu, S., Li, J., Li, Z., Zhang, Z., Feng, Y., & Law, K. L. E. (2026). Loyalty-SMOTE: Data synthesis algorithm for effective imbalanced data classification. *Neural Networks*, *199*, 108677. https://doi.org/10.1016/j.neunet.2026.108677

Idwan, S., Etaiwi, W., Rafayia, H., & Matar, I. (2025). A comprehensive review of statistical variants and enhancements of SMOTE oversampling method. *International Journal of Data Science and Analytics*, *20*, 6887–6904. https://doi.org/10.1007/s41060-025-00878-w

Li, H., Wang, S., Jiang, J., Deng, C., Ou, J., Zhou, Z., & Yu, D. (2024a). Augmenting the diversity of imbalanced datasets via multi-vector stochastic exploration oversampling. *Neurocomputing*, *583*, 127600. https://doi.org/10.1016/j.neucom.2024.127600

Li, J., Fong, S., Wong, R. K., & Chu, V. W. (2018). Adaptive multi-objective swarm fusion for imbalanced data classification. *Information Fusion*, *39*, 1–24. https://doi.org/10.1016/j.inffus.2017.03.007

Li, Y., Jin, J., Tao, H., Xiao, Y., Liang, J., & Chen, C. L. P. (2024b). Complemented subspace-based weighted collaborative representation model for imbalanced learning. *Applied Soft Computing*, *153*, 111319. https://doi.org/10.1016/j.asoc.2024.111319

Li, Y., Wang, S., Jin, J., Zhu, F., Zhao, L., Liang, J., & Chen, C. L. P. (2024c). Imbalanced complemented subspace representation with adaptive weight learning. *Expert Systems with Applications*, *249*, 123555. https://doi.org/10.1016/j.eswa.2024.123555

Li, Y., Wang, Y., Jin, J., Zhang, W., Tao, H., Wu, H., & Chen, C. L. P. (2025a). Imbalanced broad learning system with label relaxation and sample weight adaptation. *Applied Soft Computing*, *182*, 113543. https://doi.org/10.1016/j.asoc.2025.113543

Li, Y., Yang, Y., Song, P., Duan, L., & Ren, R. (2025b). An improved SMOTE algorithm for enhanced imbalanced data classification by expanding sample generation space. *Scientific Reports*, *15*, 23521. https://doi.org/10.1038/s41598-025-09506-w

Lin, K. Y., & Jamrus, T. (2024). Industrial data-driven modeling for imbalanced fault diagnosis. *Industrial Management & Data Systems*, *124*, 3108–3137. https://doi.org/10.1108/IMDS-12-2023-0927

Matharaarachchi, S., Domaratzki, M., & Muthukumarana, S. (2024). Enhancing SMOTE for imbalanced data with abnormal minority instances. *Machine Learning with Applications*, *18*, 100597. https://doi.org/10.1016/j.mlwa.2024.100597

Qiu, F., Yang, F., Xiong, Y., Chen, T., Xiao, P., Zheng, W., Chen, X., & Zhou, K. (2025). VS-SMOTE: Leveraging high-value spaces to balance noise control and data diversity for class imbalanced learning. *Neurocomputing*, *654*, 131321. https://doi.org/10.1016/j.neucom.2025.131321

Rezvani, S., Pourpanah, F., Lim, C. P., & Wu, Q. M. J. (2024). Methods for class-imbalanced learning with support vector machines: A review and an empirical evaluation. *Soft Computing*, *28*, 11873–11894. https://doi.org/10.1007/s00500-024-09931-5

Sáez, J. A., Luengo, J., Stefanowski, J., & Herrera, F. (2015). SMOTE-IPF: Addressing the noisy and borderline examples problem in imbalanced classification by a re-sampling method with filtering. *Information Sciences*, *291*, 184–203. https://doi.org/10.1016/j.ins.2014.08.051

Salmi, M., Atif, D., Oliva, D., Abraham, A., & Ventura, S. (2024). Handling imbalanced medical datasets: Review of a decade of research. *Artificial Intelligence Review*, *57*, 273. https://doi.org/10.1007/s10462-024-10884-2

Siddavatam, K. I., & Shinde, S. K. (2026). A hybrid literature review on handling imbalanced medical data: AI models and open issues. *Expert Systems with Applications*, *296*, 129004. https://doi.org/10.1016/j.eswa.2025.129004

Sun, P., Wang, Z., Jia, L., & Xu, Z. (2024a). SMOTE-kTLNN: A hybrid re-sampling method based on SMOTE and a two-layer nearest neighbor classifier. *Expert Systems with Applications*, *238*, 121848. https://doi.org/10.1016/j.eswa.2023.121848

Sun, Y., Kamel, M. S., Wong, A. K. C., & Wang, Y. (2007). Cost-sensitive boosting for classification of imbalanced data. *Pattern Recognition*, *40*, 3358–3378. https://doi.org/10.1016/j.patcog.2007.04.009

Sun, Z., Ying, W., Zhang, W., & Gong, S. (2024b). Undersampling method based on minority class density for imbalanced data. *Expert Systems with Applications*, *249*, 123328. https://doi.org/10.1016/j.eswa.2024.123328

Tao, L., Wang, Q., Zhu, Z., Yu, F., & Yin, X. (2024). NCLWO: Newton's cooling law-based weighted oversampling algorithm for imbalanced datasets with feature noise. *Neurocomputing*, *610*, 128538. https://doi.org/10.1016/j.neucom.2024.128538

Wang, S., Bao, Y., & Yang, S. (2025). HS-SMOTE: Oversampling method for multiple dynamic interpolations based on regular hexagon scoring mechanism. *Expert Systems with Applications*, *265*, 125855. https://doi.org/10.1016/j.eswa.2024.125855

Wang, W., & Liu, F. (2023). ADDPC-SMOTE: An oversampling algorithm based on density difference peak clustering and spatial distribution entropy. *IEEE Access*, *11*, 108152–108166. https://doi.org/10.1109/ACCESS.2023.3320265

Welford, B. P. (1962). Note on a method for calculating corrected sums of squares and products. *Technometrics*, *4*, 419–420. https://doi.org/10.1080/00401706.1962.10490022

Yi, X., Xu, Y., Hu, Q., Krishnamoorthy, S., Li, W., & Tang, Z. (2022). ASN-SMOTE: A synthetic minority oversampling method with adaptive qualified synthesizer selection. *Complex & Intelligent Systems*, *8*, 2247–2272. https://doi.org/10.1007/s40747-021-00638-w

Yu, H., Sun, C., Yang, X., Yang, W., Shen, J., & Qi, Y. (2016). ODOC-ELM: Optimal decision outputs compensation-based extreme learning machine for classifying imbalanced data. *Knowledge-Based Systems*, *92*, 55–70. https://doi.org/10.1016/j.knosys.2015.10.012

Zhang, A., Yu, H., Zhou, S., Huan, Z., & Yang, X. (2022). Instance weighted SMOTE by indirectly exploring the data distribution. *Knowledge-Based Systems*, *249*, 108919. https://doi.org/10.1016/j.knosys.2022.108919

Zhang, X., Gweon, H., & Provost, S. (2020). Threshold moving approaches for addressing the class imbalance problem and their application to multi-label classification. In *ICAIP 2020: 4th International conference on advances in image processing, Chengdu, China* (pp. 72–77). ACM. https://doi.org/10.1145/3441250.3441274

Zhou, C., & Zou, L. (2025). Adaptive synthetic oversampling with Poincaré distance and two-stage weight assignment. *Neurocomputing*, *647*, 130436. https://doi.org/10.1016/j.neucom.2025.130436

Zhou, Z. H., & Liu, X. Y. (2006). Training cost-sensitive neural networks with methods addressing the class imbalance problem. *IEEE Transactions on Knowledge and Data Engineering*, *18*(1), 63–77. https://doi.org/10.1109/TKDE.2006.17

Zhu, Q., Feng, J., & Huang, J. (2016). Natural neighbor: A self-adaptive neighborhood method without parameter k. *Pattern Recognition Letters*, *80*, 30–36.